

TRABAJO FIN DE GRADO

Trabajo fin de grado

Realizado por
María Quirós Quiroga

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Director 1
Director 2

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de junio/julio/diciembre, curso 20XX/YY



Escuela Técnica Superior de
Ingeniería Informática

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español

Palabras clave: palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: keyword 1, keyword 2, ..., keyword N

Índice general

1. Introducción	2
2. Estudio previo	6
2.1. Introducción	6
2.2. Objetivos	6
2.3. Metodología	7
2.4. Planificación	8
2.5. Presupuesto	12
2.6. Análisis de Riesgos	13
2.7. Conclusiones	16
3. Contexto	17
3.1. Introducción	17
3.2. Procesamiento de datos genómicos	17
3.3. Predicción de resistencia a antibióticos	21
3.4. Retos y necesidades	21
3.5. Conclusiones	22
4. Análisis del problema	23
4.1. Introducción	23
4.2. Situación actual	23
4.3. Usuarios y necesidades	28
4.4. Diseño conceptual de la solución	31
4.5. Catálogo de requisitos	35
4.6. Conclusiones	53
5. Diseño de la solución	54
5.1. Introducción	54
5.2. Visión general de la arquitectura	54
5.3. Descomposición en módulos	58
5.4. Modelo de datos y clases	60

5.5. Flujos principales del sistema	60
5.6. Decisiones de diseño	64
5.7. Diseño de la interfaz de usuario	67
5.8. Conclusiones	70
6. Implementación	71
6.1. Introducción	71
6.2. Tecnologías y herramientas utilizadas	71
6.3. Sprint 1 - Estudio previo y definición del sistema	74
6.4. Sprint 2 - Diseño de arquitectura y definición del framework	75
6.5. Sprint 3 - Desarrollo de la infraestructura base	77
6.6. Sprint 4 - Servicio de conversión de datos genómicos	82
6.7. Sprint 5 - Integración y generación de features	89
6.8. Sprint 6 - Módulo de predicción de resistencia	96
6.9. Sprint 7 - Arquitectura modular para modelos de predicción	99
6.10. Sprint 8 - Interfaz de usuario y notificaciones	107
6.11. Sprint 9 - Integración del sistema, pruebas y validación	107
6.12. Sprint 10 - Memoria, ajustes finales y puesta en producción	107
6.13. Conclusiones	107
7. Conclusiones	108
Bibliografía	109

Índice de extractos de código

Cod. 6.1	Segmento del script de setup.py utilizado para configurar el proyecto	79
Cod. 6.2	Segmento del código de integración de la herramienta de ensamblaje SPAdes dentro del sistema bioinformático	83
Cod. 6.3	Segmento del código de integración de la herramienta de ensamblaje Flye dentro del sistema bioinformático	83
Cod. 6.4	Segmento del código de integración de la herramienta de ensamblaje Raven dentro del sistema bioinformático	83
Cod. 6.5	Segmento del código de integración de la herramienta de anotado Bakta dentro del sistema bioinformático	85
Cod. 6.6	Ejemplo del código de la API del sistema bioinformático	87
Cod. 6.7	Ejemplo del código de la capa de servicios encargada de la comunicación con el sistema bioinformático	87
Cod. 6.8	Mecanismo de polling implementado para monitorizar el estado de los procesos bioinformáticos desde el sistema web	87
Cod. 6.9	Ejemplo simplificado de parser de JSON generado por Bakta	91
Cod. 6.10	Queryset personalizado para el modelo Gene	93
Cod. 6.11	Clase base para los parsers	93
Cod. 6.12	Registro de parsers	94
Cod. 6.13	Ejemplo de implementación de un parser concreto utilizando el sistema de registro mediante decoradores	94
Cod. 6.14	Interfaz Strategy para los parsers	95
Cod. 6.15	Modelo de predicción base_bakta_50	96
Cod. 6.16	Definición de las features utilizadas por el modelo base_bakta_50 . .	96
Cod. 6.17	Definición del método de carga para el modelo base_bakta_50	96
Cod. 6.18	Definición del método de predicción para el modelo base_bakta_50 .	98
Cod. 6.19	Función de cálculo de presencia o ausencia de genes a partir de los resultados almacenados en la base de datos	99
Cod. 6.20	Función de carga de archivos de pickle con información relevante para la generación de features	99

Cod. 6.21	Función de carga de archivos de pesos para modelos de torch	99
Cod. 6.22	Clase base para los modelos de predicción	102
Cod. 6.23	Registro de modelos de predicción	102
Cod. 6.24	Registro de modelos de predicción	104
Cod. 6.25	Llamada única de predicción	105
Cod. 6.26	Estructura de archivos recomendada para la integración de nuevos modelos de predicción dentro del módulo `ai_models`	105
Cod. 6.27	Función de predicción para múltiples antibióticos	105

Índice de figuras

Fig. 2.1	Diagrama de Gantt del proyecto	11
Fig. 3.1	Flujo de trabajo típico en el análisis genómico	18
Fig. 3.2	Resumen de flujo de secuenciación	19
Fig. 3.3	Resumen de flujo de ensamblaje	20
Fig. 3.4	Resumen de flujo de anotación	21
Fig. 4.1	Flujo de análisis genómico integrado del sistema.	33
Fig. 4.2	Diagrama de casos de uso para el flujo de análisis genómico integrado	36
Fig. 4.3	Diagrama de casos de uso para el flujo de notificaciones del sistema . .	41
Fig. 4.4	Diagrama de casos de uso para el flujo de gestión de procesos	42
Fig. 4.5	Diagrama Entidad-Relación	47
Fig. 5.1	Diagrama de arquitectura del sistema a alto nivel	54
Fig. 5.2	Diagrama de despliegue del sistema	57
Fig. 5.3	Diagrama de módulos del sistema web	58
Fig. 5.4	Diagrama de módulos del sistema bioinformático	60
Fig. 5.5	Diagrama de secuencia del flujo de ensamblaje o anotación	61
Fig. 5.6	Diagrama de secuencia del flujo de predicción de resistencia	62
Fig. 5.7	Diagrama de navegación de la interfaz	67
Fig. 5.8	Mockup de la interfaz de visualización de predicciones.	69

Índice de tablas

Tab. 2.1	Sprints del proyecto	9
Tab. 2.2	Tiempo estimado y real de los Sprints del proyecto	10
Tab. 2.3	Costes estimados del proyecto	12
Tab. 2.4	Costes reales del proyecto	13
Tab. 6.1	Endpoints expuestos por la API del sistema bioinformático para la comunicación con el sistema web	87

Índice

2.1	Añadir captura de p	7
4.1	Incluir un ejemplo d	24
4.2	Añadir capturas	25
4.3	Incluir un ejemplo d	25
4.4	Añadir citas a algu	25
4.5	Mencionar algo del d	35
4.6	A lo mejor hay que d	51
5.1	Quiero cambiar una c	60
6.1	Maybe añadir un dia	71
6.2	Añadir cosas intere	78
6.3	Meter una captura ej	78
6.4	Mejorar párrafo fin	81
6.5	El S6 y el S7 he de	95

Introducción

Este Trabajo de Fin de Grado se enmarca en el contexto del proyecto de investigación PID2022-139798OB-I00¹, en el cual la autora se encuentra actualmente contratada. Dicho proyecto tiene como objetivo el desarrollo de técnicas y herramientas de ingeniería de datos, con especial énfasis en tareas de predicción. En particular, este trabajo se sitúa dentro del paquete de trabajo P4 (Models for Antibiotic Resistance Prediction), orientado a la construcción y validación de modelos predictivos aplicados a datos biológicos, así como a su disponibilidad mediante herramientas accesibles para otros investigadores y profesionales del dominio.

Este proyecto es desarrollado por el grupo DEAL (Data Engineering Applications Lab) de la Universidad de Sevilla. Adicionalmente, se colabora con la Dra. Lorena López-Cerero y la Dra. Marina Pulido, investigadoras expertas en microbiología del Grupo de Resistencia Microbiana de la Universidad de Sevilla.

En este contexto, el presente trabajo se centra en facilitar el uso práctico de las técnicas desarrolladas, mediante el diseño de soluciones que permitan su ejecución, integración y explotación en entornos reales.

La resistencia bacteriana a los antibióticos constituye uno de los mayores retos actuales para la salud pública a nivel mundial. Este fenómeno amenaza con revertir décadas de avances médicos, haciendo que infecciones y enfermedades comunes, hoy en día tratables de forma sencilla, vuelvan a ser potencialmente mortales. En

¹https://investigacion.us.es/sisius/sis_proyecto.php?idproy=37783

la actualidad, se estima que causa más de 1,14 millones de muertes anuales, y se espera que esta cifra supere los 8 millones en 2050, situándose entre las principales causas de mortalidad, por delante de enfermedades como el cáncer.

La resistencia a los antibióticos surge cuando las bacterias mutan en respuesta a la presión ejercida por el uso, en muchos casos indebido, de estos fármacos. Este proceso favorece la aparición y propagación de microorganismos con mecanismos de resistencia, lo que reduce la eficacia de los tratamientos disponibles y da lugar a infecciones más difíciles de tratar. A pesar de la magnitud de este problema, el desarrollo de nuevos antibióticos se ha ralentizado significativamente en las últimas décadas, debido tanto a los elevados costes como a la complejidad asociada a su investigación. Como consecuencia, las bacterias resistentes continúan aumentando rápidamente, mientras que el número de nuevos antibióticos disponibles para combatirlos ha permanecido estancado.

En este contexto, las pruebas de sensibilidad antimicrobiana (antimicrobial susceptibility testing, AST) desempeñan un papel fundamental. Estas técnicas permiten determinar si un microorganismo es sensible o resistente a distintos antibióticos, orientando así la elección del tratamiento más adecuado. Tradicionalmente, los métodos de AST se basan en el aislamiento del microorganismo, su cultivo en laboratorio y su exposición a diferentes agentes antimicrobianos para evaluar su crecimiento o inhibición. Sin embargo, estos procedimientos presentan una limitación crucial: la necesidad de cultivo implica tiempos prolongados, oscilando entre varios días o incluso semanas. Este retraso en la obtención de resultados condiciona la toma de decisiones clínicas y puede demorar el inicio del tratamiento adecuado para el paciente, lo cual resulta especialmente crítico en casos de infecciones graves.

Ante estas limitaciones, la secuenciación del genoma completo, combinada con técnicas de predicción, ha emergido como una alternativa complementaria prometedora. Este enfoque permite analizar directamente la información genética del microorganismo para identificar patrones asociados a resistencia, reduciendo significativamente los tiempos de diagnóstico y facilitando una aproximación más rápida y eficaz al tratamiento.

No obstante, el desarrollo e implementación de este tipo de sistemas plantea desafíos significativos. En primer lugar, requiere la integración de múltiples disciplinas, como la biología molecular, la informática y el aprendizaje automático. Además, es necesario abordar la transformación de datos genómicos complejos en representaciones adecuadas para su procesamiento y análisis computacional. A esto se suma la necesidad de diseñar, entrenar y validar modelos de predicción robustos, capaces de generalizar correctamente ante nuevos datos y ofrecer resultados fiables y relevantes.

Por otro lado, la utilidad real de estos modelos y herramientas predictivas depende en gran medida de su integración efectiva en el entorno clínico. Esto implica no solo ofrecer predicciones precisas, sino también desarrollar interfaces accesibles e intuitivas que facilitan su uso por parte de los profesionales sanitarios, quienes no necesariamente cuentan con formación técnica en informática o análisis de datos.

En este proyecto, se aborda este reto mediante el desarrollo de una herramienta de apoyo a la toma de decisiones clínicas, que permite predecir la resistencia a los antibióticos de un microorganismo a partir de su secuencia genómica. Concretamente, se propone el desarrollo de un framework flexible y extensible que integra tanto el procesamiento de datos genómicos como la aplicación de modelos predictivos.

Por un lado, el sistema proporciona funcionalidades orientadas a la gestión y transformación de datos biológicos, permitiendo la conversión entre distintos formatos de archivos ampliamente utilizados en genómica, como FASTQ, FASTA y anotaciones generadas mediante la herramienta Bakta. Estas utilidades facilitan etapas fundamentales del flujo de trabajo, como son el ensamblaje y la anotación del genoma, simplificando procesos que habitual requieren el uso de múltiples herramientas independientes y conocimientos técnicos especializados.

Por otro lado, el framework ofrece una infraestructura flexible diseñada para la integración y despliegue de modelos predictivos orientados a la resistencia a antibióticos. En particular, se busca reducir la complejidad asociada a la incorporación de nuevos modelos, permitiendo que estos puedan añadirse de forma sencilla al sistema mediante una interfaz. Asimismo, se ofrece una interfaz de predicción que permite a los usuarios del ámbito de la salud acceder a las predicciones de resistencia de forma sencilla e incorpora los nuevos modelos automáticamente, de modo que estos quedan disponibles para su uso dentro de la plataforma sin necesidad de desarrollar componentes adicionales.

En definitiva, este proyecto no se limita al desarrollo de una solución específica, sino que propone una herramienta reutilizable y extensible que facilita la integración de nuevos modelos predictivos en el flujo de trabajo clínico. De este modo, contribuye a reducir la brecha existente entre el desarrollo de modelos de predicción y su aplicación efectiva en la práctica clínica.

Este documento se compone de seis capítulos, en los cuales se profundiza en distintos aspectos del proyecto llevado a cabo durante el desarrollo de este trabajo de fin de grado.

En primer lugar, el **Capítulo 2** se centra en el estudio previo al proyecto, en el que se exponen aspectos como los objetivos del proyecto, la metodología seguida y la planificación temporal y presupuestaria del mismo.

El **Capítulo 3** describe el contexto en el que se enmarca este proyecto, incluyendo una descripción del proceso de análisis genómico y la predicción de resistencia a antibióticos, así como una revisión de los retos y necesidades actuales en este ámbito.

El **Capítulo 4** se dedica al análisis del problema, detallando las herramientas disponibles en la situación actual, sus limitaciones, los usuarios y necesidades, el sistema propuesto y el conjunto de casos de uso, requisitos de información, requisitos funcionales y no funcionales que se han definido para el desarrollo del proyecto.

El **Capítulo 5** se centra en el diseño de la solución propuesta, incluyendo la arquitectura del sistema, los diagramas de clases, prototipos de interfaz y otros aspectos relevantes del diseño.

El **Capítulo 6** se dedica a la descripción de la implementación de la solución propuesta, describiendo las tecnologías utilizadas, la estructura del código y los aspectos más relevantes de la implementación.

Finalmente, el **Capítulo 7** recoge las conclusiones obtenidas a lo largo del desarrollo del proyecto, así como las posibles líneas de trabajo futuro que se podrían seguir a partir de este trabajo de fin de grado.

Estudio previo

2.1. Introducción

En este capítulo se exponen aspectos de gestión previos a la ejecución del proyecto detallando aspectos como los objetivos del mismo, la metodología seguida para su desarrollo y la planificación temporal y presupuestaria del proyecto.

2.2. Objetivos

El objetivo principal de este proyecto es diseñar y desarrollar un framework que integre el procesamiento y conversión de datos genómicos con la incorporación de modelos predictivos, con el fin de facilitar su uso por parte de personal sanitario en entornos reales.

Adicionalmente, se plantean los siguientes objetivos secundarios:

- Desarrollar un **sistema capaz de gestionar y convertir distintos formatos** de datos genómicos (FASTQ, FASTA y anotaciones generadas con Bakta), ofreciendo un entorno único en el que realizar los procesos de ensamblaje y anotación.
- Diseñar una **arquitectura modular** que permita la integración sencilla de modelos de predicción.

- Implementar un mecanismo para **automatizar la incorporación de modelos** y su disponibilidad dentro de la plataforma.
- Desarrollar una **interfaz intuitiva** que permita el uso del sistema de forma sencilla por parte del personal sanitario.
- Implementar un **sistema de notificaciones** que informe a los usuarios sobre el estado de ejecución de sus tareas y disponibilidad de resultados.
- **Optimizar la gestión de procesos** de alto coste computacional asociados al procesamiento y transformación de datos genómicos.
- Adquirir **experiencia en el desarrollo** de aplicaciones que integran bioinformática y aprendizaje automático.
- Integrar los conocimientos adquiridos a lo largo del grado en un **proyecto práctico** con aplicación en un **contexto real**.

2.3. Metodología

Para el desarrollo de este proyecto, se ha adoptado una metodología ágil adaptada a las características de un proyecto individual. Se ha priorizado la flexibilidad en la organización para poder adaptarse a las necesidades del software y a la carga lectiva del curso. Por ello, se ha optado por Scrumban, una metodología híbrida que combina la estructura de Scrum con la fluidez de Kanban.

Scrumban combina la planificación iterativa propia de Scrum con la gestión visual del flujo de trabajo ofrecido por Kanban. El desarrollo se ha dividido en iteraciones de duración variable (sprints), donde se definen objetivos concretos a corto plazo. Al inicio de cada iteración se establecen las metas y al finalizar se evalúan los resultados obtenidos, ajustando la planificación en función del progreso y cambios en las prioridades y los requisitos.

 **TODO:** Añadir captura de pantalla del tablero de Trello o algo para ilustrar la gestión de tareas

Como herramienta principal para la gestión de tareas se ha utilizado un tablero Kanban en Trello. Éste permite visualizar el estado del proyecto mediante columnas (*Todo*, *In Progress*, *Blocked* y *Done*), facilitando el seguimiento por parte de los tutores sin requerir reuniones diarias. Adicionalmente, se han establecido límites al trabajo en progreso (WIP), restringiendo el enfoque a una sola tarea activa para evitar la multitarea y favorecer la concentración y la finalización de tareas antes de iniciar nuevas.

Dada la naturaleza individual del proyecto y la necesidad de flexibilidad en la planificación, las reuniones diarias propuestas por Scrumban se han sustituido por sesiones semanales de seguimiento con los tutores. En estas reuniones se revisa el progreso, se resuelven dudas y bloqueos y se ajusta la planificación.

Este enfoque permite un desarrollo incremental y flexible, facilitando la entrega continua de funcionalidades sin depender de ciclos cerrados rígidos, adaptándose a las necesidades del proyecto y al ritmo académico.

2.4. Planificación

La planificación del proyecto se ha estructurado mediante un enfoque iterativo basado en sprints, tal y como se ha descrito en la sección de metodología. El proyecto se ha dividido en sprints de duración variable, con una planificación adaptada a la complejidad y necesidades de cada módulo.

Cada sprint ha estado orientado a un objetivo concreto, aportando cada uno valor al sistema. A continuación se detalla la planificación temporal del proyecto. En la siguiente tabla se resumen los sprints definidos, incluyendo sus fechas, duración y objetivos principales:

2. Estudio previo

Sprint	Fechas	Objetivo principal
1	25/11-02/12 2025	Estudio previo del contexto biológico y definición del sistema
2	02/12-16/12 2025	Diseño de arquitectura del sistema y definición del framework
3	16/12-13/01 2026	Desarrollo de la infraestructura base (autenticación, base de datos, Docker, etc.)
4	13/01-27/01 2026	Desarrollo del servicio de conversión de datos genómicos
5	27/01-10/02 2026	Procesamiento y parseo de datos anotados y almacenamiento en base de datos
6	10/02-03/03 2026	Desarrollo del módulo de predicción de resistencia mediante ML
7	03/03-17/03 2026	Diseño e implementación de arquitectura modular para modelos de predicción
8	17/03-14/04 2026	Desarrollo de la interfaz de usuario y sistema de notificaciones
9	14/04-21/04 2026	Integración del sistema, pruebas y validación
10	21/04-12/05 2026	Redacción de la memoria, ajustes finales y puesta en producción

Tabla 2.1: Sprints del proyecto

En segundo lugar, se detalla la planificación de la dedicación en horas para cada sprint, junto con la dedicación real registrada durante el desarrollo. Esto nos permite analizar las desviaciones producidas durante el desarrollo:

Sprint	Dedicación estimada (Hrs.)	Dedicación real (Hrs.)
1	15h	14h
2	25h	20h
3	35h	42h
4	45h	48h
5	30h	28h
6	30h	35h
7	25h	21h
8	50h	62h
9	15h	14h
10	40h	45h
Total	310h	329h

Tabla 2.2: Tiempo estimado y real de los Sprints del proyecto

Como puede observarse, la planificación inicial se ha mantenido en términos generales dentro de lo previsto, aunque se han producido desviaciones en determinados sprints. Destaca especialmente el sprint 8, centrado en el desarrollo de la interfaz de usuario, que requirió un esfuerzo superior al estimado para lograr una experiencia de uso intuitiva y accesible para los profesionales del ámbito de la salud.

Adicionalmente, la fase de desarrollo de la infraestructura también presentó una desviación relevante, debido a la complejidad asociada al despliegue del sistema y el servicio de conversión de datos genómicos mediante Docker y la incorporación de mecanismos de paralelización de tareas mediante la herramienta Celery.

Por el contrario, la fase de diseño de la arquitectura resultó requerir menor dedicación de lo previsto inicialmente. Esto se debe a la decisión de adoptar un enfoque modular y sencillo, evitando introducir complejidades innecesarias y priorizando la claridad y mantenibilidad del sistema.

Finalmente, se presenta el diagrama de Gantt del proyecto, en el que se representa de forma visual la distribución temporal de las principales tareas desarrolladas a lo largo de los distintos sprints:

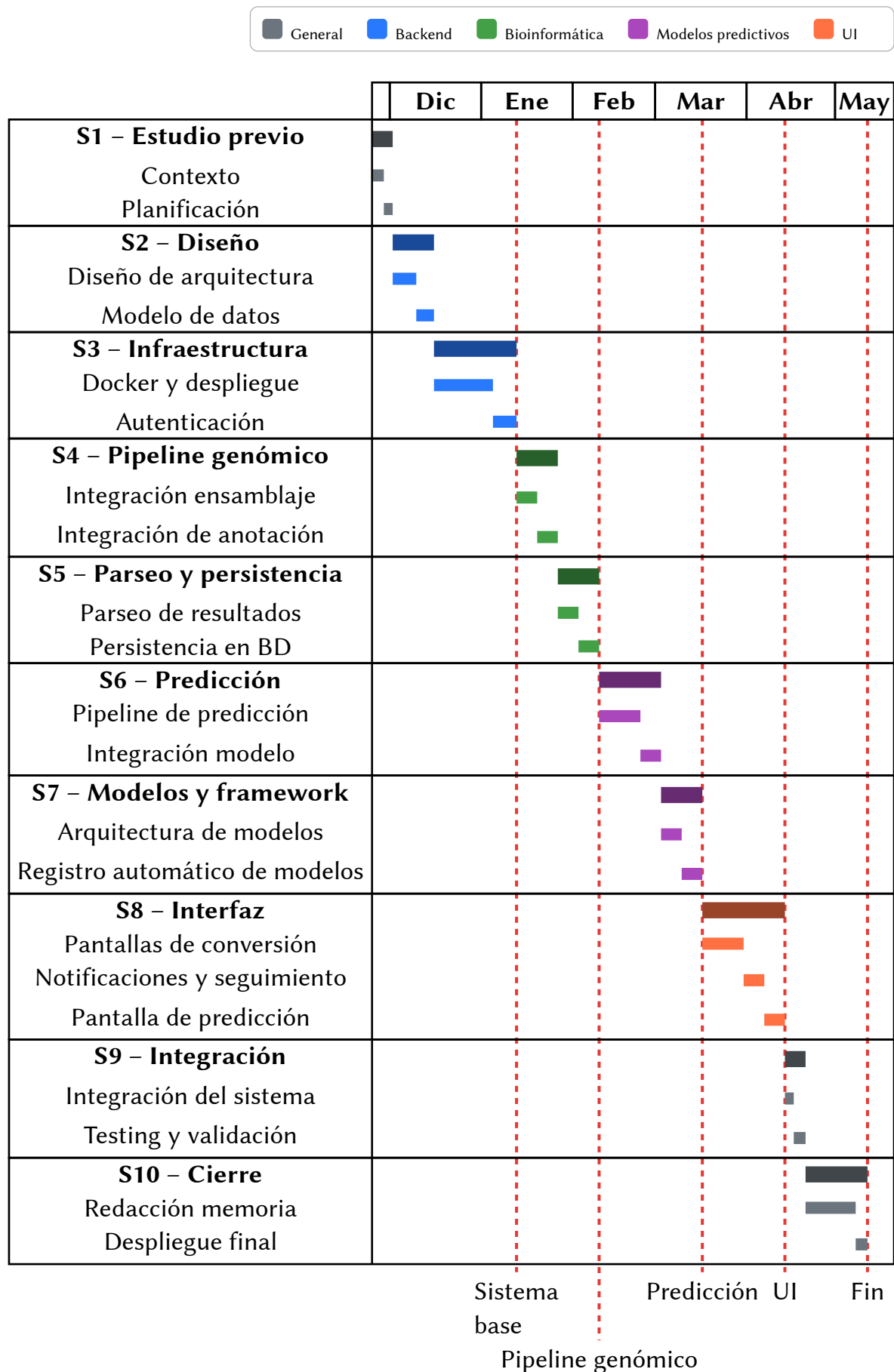


Figura 2.1: Diagrama de Gantt del proyecto

2.5. Presupuesto

El cálculo de los costes del proyecto tiene en cuenta varias partidas, cuyo importe depende en mayor o menor medida de la dedicación empleada.

En primer lugar, se consideran los costes de personal. Para su estimación se establece un coste de 23,85 euros por hora, tomando como referencia el salario medio de un desarrollador junior en España.

Por otro lado, dentro de los costes de equipo y suministros, se incluyen los costes asociados a la amortización del equipo informático utilizado. Este equipo tiene un coste inicial de 1.200 euros y una vida útil estimada de seis años, siendo su periodo de uso en el desarrollo de este proyecto de seis meses. Adicionalmente, se estima el consumo eléctrico del equipo en 200 W por hora de uso, con un coste medio de 0,20 euros por kWh. Asimismo, se considera el gasto correspondiente a la conexión a Internet, estimado en una tarifa mensual de 50 euros.

Finalmente, se establece una reserva para riesgos e imprevistos del 10% sobre el total de los costes anteriores.

Con todo ello, el coste total del proyecto asciende a 9.268,49 euros, desglosados de la siguiente forma:

Partida	Cálculo	Total
Coste de personal	23,85 €/h x 310 horas	8013,50 €
Amortización de equipo	1200 €/72 meses x 6 meses de proyecto	100 €
Suministro eléctrico	0,20 €/kWh x 200 W x 310h	12,40 €
Conexión a Internet	50 €/mes x 6 meses	300 €
	Total parcial	8425,90 €
	Reserva (10%)	842,59 €
	Total	9268,49 €

Tabla 2.3: Costes estimados del proyecto

Los costes incurridos finalmente durante la ejecución del proyecto han sido los siguientes:

Partida	Cálculo	Total
Coste de personal	25,85 €/h x 329 horas	8504,65 €
Amortización de equipo	1200 €/72 meses x 6 meses de proyecto	100 €
Suministro eléctrico	0,20 €/kWh x 200 W x 329h	13,16 €
Conexión a Internet	50 €/mes x 6 meses	300 €
	Gastos finales	8917,81 €
	Total previsto (con reserva)	9268,49 €
	Superávit en costes	350,68 €

Tabla 2.4: Costes reales del proyecto

Concluimos por tanto que el coste del proyecto asciende a 8.917,81 euros. El aumento en el coste respecto a lo planificado inicialmente se debe principalmente a la mayor dedicación de horas comentada en la sección de planificación. Sin embargo, gracias a la reserva se ha podido cubrir dicho aumento de costes, obteniendo un superávit de 350,68 euros, lo que representa aproximadamente un 3,78% del presupuesto total.

2.6. Análisis de Riesgos

El desarrollo del sistema propuesto, centrado en la integración del pipeline de análisis genómico y la incorporación de modelos de predicción de resistencia a antibióticos, presenta una serie de riesgos que podrían afectar a su viabilidad y correcta ejecución. A continuación, se detallan los principales riesgos identificados, junto con su probabilidad de ocurrencia, impacto potencial y estrategias de mitigación.

R-001	Integración del pipeline completo
Probabilidad	Media.
Impacto	Alto.
Descripción	La unificación en un único flujo de distintas etapas del análisis genómico, tradicionalmente desacopladas, puede incrementar la complejidad del desarrollo y dificultar su correcta integración.
Mitigación	• Desarrollo incremental por etapas independientes. • Uso de interfaces claras entre componentes. • Uso de herramientas consolidadas para cada fase.
R-002	Dependencia de herramientas externas
Probabilidad	Media.

R-002	Dependencia de herramientas externas
Impacto	Alto.
Descripción	El sistema depende de herramientas de bioinformática externas para el ensamblaje y la anotación, lo que introduce incertidumbre en cuanto a su compatibilidad, rendimiento y mantenimiento a lo largo del tiempo.
Mitigación	• Uso de herramientas consolidadas y ampliamente utilizadas en la comunidad. • Diseño flexible que permita sustituir herramientas sin afectar al resto del sistema. • Gestión adecuada de errores en ejecuciones externas.

R-003	Complejidad de la generación de features
Probabilidad	Media.
Impacto	Alto.
Descripción	La transformación de los resultados de anotación en características válidas para modelos predictivos puede resultar compleja.
Mitigación	• Definición clara del formato intermedio. • Validación de los datos generados. • Mecanismos extensibles basados en parsers.

R-004	Integración de modelos predictivos heterogéneos
Probabilidad	Media.
Impacto	Medio-Alto.
Descripción	El sistema pretende ser extensible a distintos modelos de predicción, lo que puede dificultar la definición de una interfaz común para su integración.
Mitigación	• Definición de contratos de entrada/salida claros. • Uso de adaptadores para integrar modelos. • Documentación clara para la incorporación de nuevos modelos. • Restricción inicial a modelos compatibles.

R-005	Gestión de tareas asíncronas
Probabilidad	Media.
Impacto	Medio-Alto.

R-005	Gestión de tareas asíncronas
Descripción	La ejecución asíncrona de tareas de larga duración puede aumentar la complejidad del sistema, especialmente la gestión de estados y la comunicación con el usuario.
Mitigación	• Definición clara de estados del proceso. • Implementación de mecanismos de monitoreo. • Uso de notificaciones para informar al usuario.

R-006	Rendimiento en procesamiento de datos genómicos
Probabilidad	Media.
Impacto	Alto.
Descripción	Las tareas de ensamblaje y anotación pueden requerir un alto consumo de recursos, lo que puede afectar al rendimiento global del sistema.
Mitigación	• Implementación de un servicio externo al sistema principal para el procesamiento de datos genómicos. • Uso de entornos controlados de ejecución. • Paralelización y uso de cola de tareas.

R-007	Facilidad de uso para usuarios no técnicos
Probabilidad	Media.
Impacto	Medio.
Descripción	La complejidad del dominio bioinformático puede dificultar el uso del sistema por parte de usuarios sin experiencia técnica..
Mitigación	• Diseño de interfaz simple. • Automatización del flujo completo. • Abstracción de detalles técnicos. • Soporte de ejecución por etapas.

R-008	Alcance del proyecto
Probabilidad	Alta.
Impacto	Alto.
Descripción	Debido al contexto y el carácter extensible del sistema, existe el riesgo de que el alcance del proyecto se expanda más allá de lo inicialmente previsto y asumible dentro del marco temporal y de recursos disponibles.

R-008	Alcance del proyecto
Mitigación	• Definición clara de los objetivos y funcionalidades a desarrollar. • Priorización de funcionalidades. • Desarrollo y planificación iterativa e incremental.

2.7. Conclusiones

En este capítulo se han expuesto los aspectos de gestión previos a la ejecución del proyecto, detallando los objetivos, la metodología adoptada y la planificación temporal y presupuestaria del proyecto. Se ha establecido un marco de trabajo que ha guiado la ejecución del mismo, permitiendo una gestión eficiente de los recursos y una adaptación flexible a las necesidades del proyecto a lo largo de su desarrollo.

Contexto

3.1. Introducción

En este capítulo se realiza una introducción al contexto biológico del proyecto, centrado en el procesamiento de datos genómicos y el uso de modelos predictivos para la predicción de resistencia a antibióticos. Este contexto es fundamental para comprender la relevancia del problema abordado y el vocabulario específico utilizado a lo largo de esta memoria.

3.2. Procesamiento de datos genómicos

El proyecto se apoya en dos pilares fundamentales: el procesamiento y conversión de datos genómicos, y la aplicación de modelos predictivos. Esta sección se centra en el primero de ellos, proporcionando una visión general del flujo de análisis genómico.

Los microorganismos, como bacterias y virus, poseen material genético que contiene información sobre su biología y comportamiento. La secuenciación del ADN y su posterior análisis permiten identificar genes y mutaciones que pueden estar asociados a la resistencia a antibióticos.

Para llevar a este análisis, es necesario seguir un flujo de trabajo compuesto por varias etapas: secuenciación, ensamblaje y anotación ([Figura 3.1](#)). Este proceso

transforma el ADN en datos crudos que posteriormente se convierten en información estructurada y biológicamente interpretable, que se puede utilizar en análisis más avanzados, como la predicción de resistencia a antibióticos.

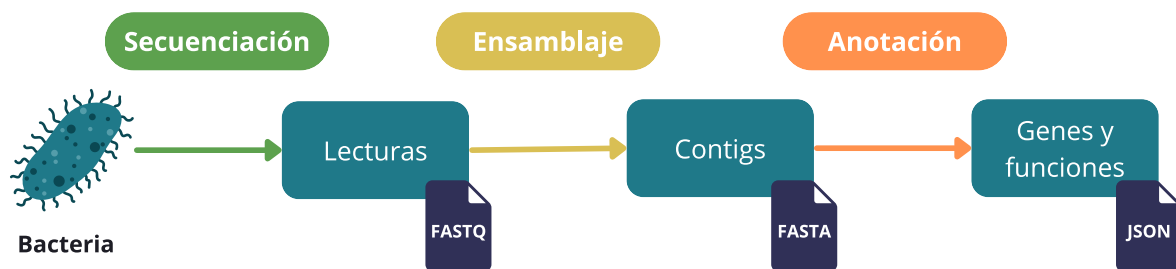


Figura 3.1: Flujo de trabajo típico en el análisis genómico

Estos procesos son complejos y requieren herramientas especializadas, así como un elevado coste computacional. En las siguientes secciones se describen las etapas principales del análisis genómico: secuenciación, ensamblaje y anotación del genoma.

3.2.1. Secuenciación

La secuenciación del ADN es el proceso que consiste en determinar el orden de los nucleótidos (A, T, C y G) en una molécula de ADN, **Figura 3.2**. En el caso de los microorganismos, esto implica la fragmentación del material genético en pequeñas secuencias denominadas *lecturas*, que representan observaciones parciales del genoma original.

La secuenciación se realiza introduciendo una muestra del microorganismo en máquinas especializadas que generan las lecturas. Existen distintas tecnologías de secuenciación, como Illumina u Oxford Nanopore Technologies (ONT), que se diferencian principalmente en la longitud de sus lecturas, la precisión y el coste. La elección de la tecnología depende de los objetivos del estudio, el presupuesto disponible y las características del microorganismo a analizar.

El framework abordado en este proyecto ha sido diseñado para ser compatible con dos de las tecnologías de secuenciado más extendidas, así como para la posible inclusión futura de forma sencilla de nuevas herramientas. Las principales características de las dos tecnologías abordadas en este proyecto son:

- **Illumina:** genera lecturas cortas con alta precisión. Habitualmente produce lecturas pareadas (paired-end), lo que mejora la calidad del ensamblaje.
- **ONT:** produce lecturas largas que facilitan la reconstrucción de regiones complejas del genoma, pero con una mayor tasa de error.

Los datos generados se almacenan habitualmente en formato FASTQ, el cual incluye tanto la secuencia de nucleótidos como una puntuación de calidad para cada base, que indica la confianza en la precisión de la lectura.

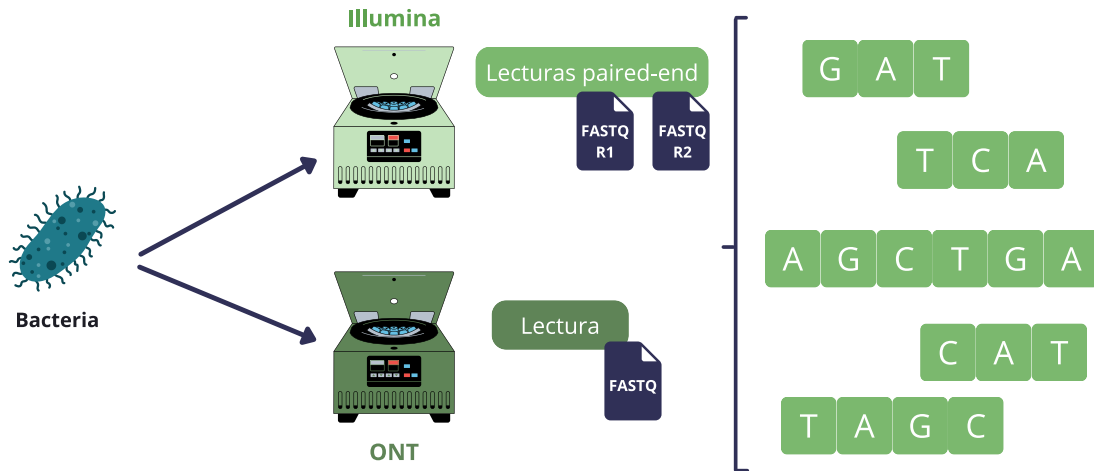


Figura 3.2: Resumen de flujo de secuenciación

3.2.2. Ensamblaje

El ensamblaje genómico, *assembly* en inglés, es el proceso mediante el cual se reconstruye la secuencia completa del genoma a partir de las lecturas obtenidas en la secuenciación (Figura 3.3). Este proceso es especialmente complejo debido a la naturaleza fragmentada de los datos y la presencia de errores en las lecturas.

Para llevar a cabo el ensamblaje, se utilizan algoritmos que buscan solapamientos entre las lecturas para construir secuencias más largas llamadas *contigs*. Los *contigs* representan fragmentos continuos del genoma y, en condiciones ideales, permiten reconstruirlo de forma completa.

Dependiendo de la tecnología de secuenciación utilizada, se emplean distintas herramientas de ensamblaje:

- Para lecturas cortas (como las generadas por Illumina), se emplean herramientas como *SPAdes*, optimizadas para manejar múltiples lecturas de alta precisión.
- Para lecturas largas (como las de ONT), se utilizan herramientas como *Raven* o *Flye*, diseñadas para manejar secuencias más largas con mayores tasas de error.

Los resultados de este proceso se almacenan en formato **FASTA**, que contiene las secuencias ensambladas del genoma.

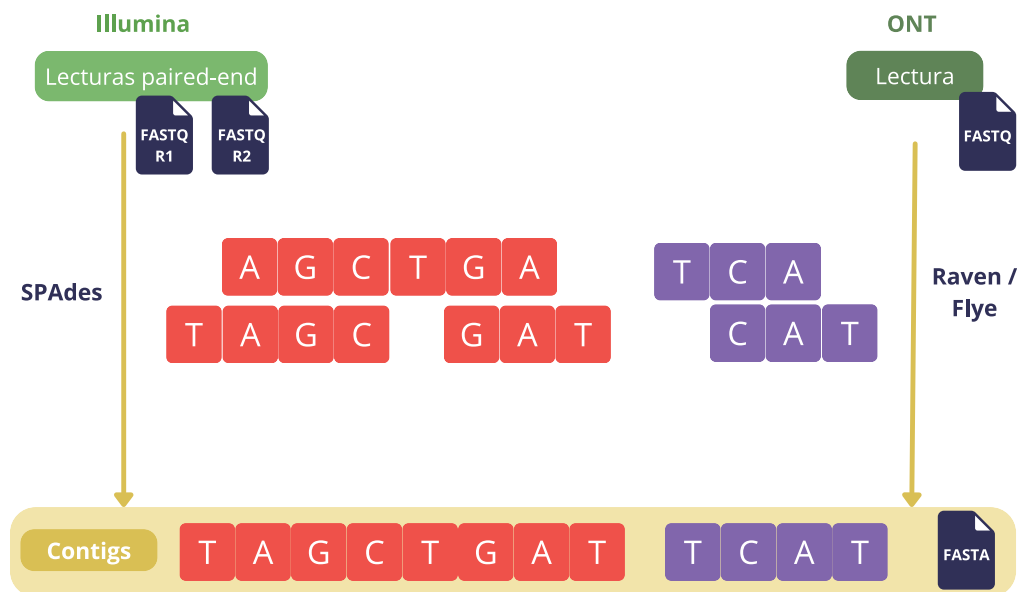


Figura 3.3: Resumen de flujo de ensamblaje

3.2.3. Anotación

Una vez obtenido el genoma ensamblado, se inicia el proceso de anotación, o *annotation* en inglés, cuyo objetivo es identificar elementos biológicos relevantes como genes, regiones codificantes y proteínas asociadas, **Figura 3.4**.

En este trabajo se utiliza la herramienta *Bakta*, ampliamente empleada en el ámbito de la genómica bacteriana. *Bakta* realiza la identificación de genes a partir de secuencias genómicas y asigna funciones biológicas mediante la comparación con bases de datos de referencia y modelos de anotación.

El resultado de este proceso es un conjunto de anotaciones estructuradas que describen la localización y función de los genes identificados dentro del genoma ensamblado.

En este proyecto, dicha información se exporta en formato JSON, lo que permite su procesamiento automatizado e integración con el sistema desarrollado, facilitando su uso en modelos predictivos.

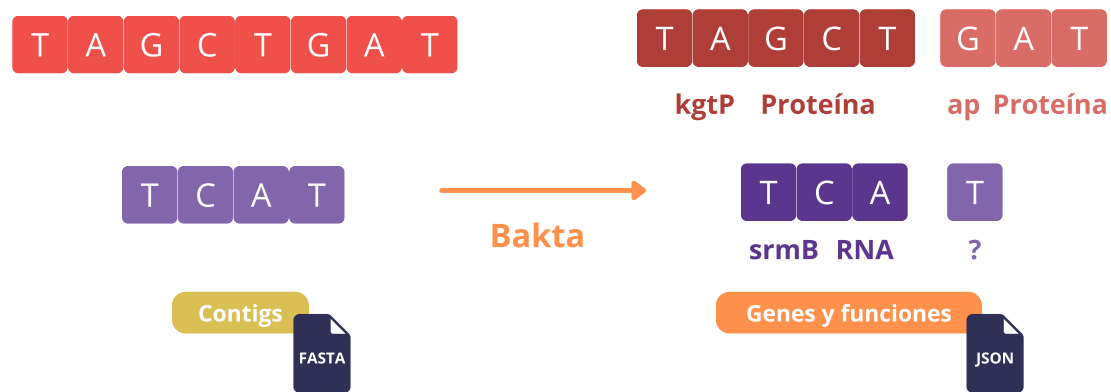


Figura 3.4: Resumen de flujo de anotación

3.3. Predicción de resistencia a antibióticos

El segundo pilar del proyecto es la aplicación de modelos de predicción en el ámbito de la resistencia a antibióticos. Este es un problema de gran relevancia en la bioinformática, donde se busca inferir el fenotipo de resistencia a partir de características derivadas del genoma.

Estos modelos operan sobre representaciones obtenidas del análisis genómico, como la presencia de genes de resistencia, anotaciones funcionales u otras características derivadas del ensamblaje.

Sin embargo, el flujo completo de análisis genómico es largo y complejo, y los datos generados requieren un preprocesamiento y estructuración adecuados para ser utilizados como entrada en modelos predictivos. Esto dificulta su uso por parte de usuarios sin experiencia en bioinformática o programación, lo que limita su aplicabilidad en entornos clínicos o de investigación.

3.4. Retos y necesidades

Actualmente, no existe una solución única que integre de forma unificada el flujo completo de procesamiento de datos genómicos, desde el ensamblaje hasta la anotación, de forma accesible para el usuario sin necesidad de utilizar la terminal o

escribir código. Esto hace que el análisis genómico sea un proceso complejo y tedioso para usuarios sin experiencia en bioinformática.

Este proyecto aborda esta necesidad mediante el desarrollo de una herramienta que integra todo el flujo de análisis genómico, permitiendo a los usuarios realizar estas tareas de manera sencilla a través de una interfaz intuitiva.

Por otro lado, aunque existen modelos para la predicción de resistencia a antibióticos, no existe una forma sencilla de ofrecerlos a los usuarios ni integrarlos con el flujo de análisis genómico, limitando su aplicabilidad.

Este proyecto, propone una solución que integra el proceso completo de ensamblaje, anotación, preparación de datos y predicción de resistencia a antibióticos, proporcionando una única herramienta unificada y extensible. Esto se logra a través de un mecanismo sencillo para integrar modelos de predicción y una interfaz intuitiva que facilita su uso por parte de usuarios sin experiencia en programación, permitiendo así su aplicación en entornos clínicos o de investigación.

3.5. Conclusiones

En este capítulo se ha presentado el contexto biológico necesario para comprender el problema abordado en este proyecto, describiendo el flujo de procesamiento de datos genómicos y las bases de los modelos de predicción aplicados al ámbito de la resistencia a antibióticos. Este contexto sirve como fundamento para el desarrollo del sistema propuesto, descrito en los siguientes capítulos.

Análisis del problema

4.1. Introducción

En este capítulo se realiza un análisis detallado del problema a abordar, identificando las limitaciones del enfoque actual en el análisis genómico y la predicción de resistencia a antibióticos, así como las necesidades de los usuarios finales. A partir de este análisis, se extraen los requisitos generales del sistema y se presentan las historias de usuario que guiarán el diseño y desarrollo de la solución propuesta. Tras esto, se presenta un diseño conceptual de la solución que responde a las necesidades identificadas y se establece un catálogo de requisitos que servirá como base para la implementación del sistema.

4.2. Situación actual

4.2.1. Flujo actual de análisis genómico

En la práctica, el análisis genómico sigue un flujo de trabajo secuencial que transforma los datos obtenidos mediante secuenciación en información biológica interpretable. Desde una perspectiva aplicada, podemos entender este flujo como una pipeline de procesamiento de datos compuesto por herramientas independientes, cada una encargada de una etapa del análisis, que se encadenan para realizar el flujo completo.

Este proceso consta de tres etapas principales: ensamblaje del genoma, anotación y análisis de los resultados. Dependiendo del caso de uso, pueden incluirse etapas adicionales como el control de calidad de las lecturas o el alineamiento contra un genoma de referencia.

En la práctica, los usuarios parten de archivos de lecturas en formato FASTQ, aplican herramientas de ensamblaje para generar contigs en formato FASTA y, posteriormente, utilizan herramientas de anotación para identificar genes y asignar funciones biológicas. Cada fase opera sobre formatos de datos específicos, lo que requiere una gestión de los datos intermedios y la correcta aplicación de cada herramienta en el orden adecuado.

Estas herramientas se utilizan habitualmente en entornos Linux mediante línea de comandos, lo que proporciona flexibilidad pero introduce una barrera para usuarios sin experiencia en informática, como es el caso en muchos contextos clínicos. Algunos ejemplos de herramientas comunmente utilizadas son:

- Para el ensamblaje: *SPAdes*, *Velvet*, *Raven* o *Flye*.
- Para la anotación: *Bakta*, *Prokka* o *RAST*.

Aunque estas herramientas son ampliamente utilizadas en la comunidad científica, su uso requiere conocimientos técnicos tanto para su instalación y configuración como para su ejecución. Además, la gestión manual de los datos intermedios y la necesidad de asegurar la compatibilidad entre las herramientas utilizadas incrementan el riesgo de errores y llevan a frustración, dificultando su adopción generalizada.

 **TODO:** Incluir un ejemplo de comando para cada etapa, o una captura de pantalla de la terminal con el proceso

Con el objetivo de reducir esta barrera de entrada, han surgido plataformas que ofrecen interfaces gráficas más accesibles para la ejecución de estos flujos de trabajo. Entre ellas destacan *Galaxy* y *Geneious*.

Galaxy es una plataforma de código abierto basada en un entorno web que permite ejecutar herramientas de análisis bioinformático, gestionar datos y construir flujos de trabajo reproducibles. Su principal ventaja es la centralización de herramientas heterogéneas bajo una única interfaz, ofreciendo gran variedad de herramientas con usos distintos para cada fase. Sin embargo, el usuario debe seleccionar manualmente cada herramienta, su orden de ejecución y configurar sus parámetros, lo que puede resultar complejo para usuarios sin experiencia técnica. Además, la amplia variedad de herramientas disponibles dificulta la selección de las más adecuadas para cada uso y la búsqueda de herramientas específicas puede resultar complicada.

Por otro lado, Geneious es una plataforma comercial de pago con interfaz gráfica que integra diversas herramientas de análisis de secuencias en un entorno unificado. Su principal ventaja es la simplificación del flujo de trabajo, eliminando la necesidad de gestionar herramientas independientes y sus datos intermedios. Esto reduce la curva de aprendizaje, no obstante, implica limitaciones en términos de flexibilidad y personalización, ya que los usuarios no pueden elegir libremente las herramientas a utilizar ni configurar sus parámetros de forma detallada. Adicionalmente, su enfoque más orientado al diseño molecular y análisis de secuencias limita su aplicabilidad en análisis genómico completo, lo que puede ser una barrera para usuarios con necesidades más específicas o avanzadas.

En conclusión, las soluciones actuales proporcionan distintos niveles de abstracción sobre el flujo del análisis genómico. Mientras que plataformas como Galaxy ofrecen una amplia variedad de herramientas con gran flexibilidad, su complejidad técnica puede dificultar su adopción por parte de usuarios sin experiencia. Por otro lado, soluciones como Geneious simplifican el proceso pero a costa de limitar la personalización y la aplicabilidad en análisis genómico completo. Estas limitaciones resaltan que la brecha entre las herramientas de análisis genómico y los usuarios finales, especialmente en contextos clínicos, sigue siendo un desafío importante a abordar.

 **TODO:** Añadir capturas

4.2.2. Predicción de resistencia a antibióticos

Existen diversos enfoques computacionales para la predicción de resistencia a antibióticos a partir de datos genómicos, que incluyen tanto modelos basados en aprendizaje automático, como métodos basados en alineamiento contra bases de datos de genes de resistencia o modelos basados en las características de una familia de bacterias concreta. Entre los primeros se encuentran herramientas como *DeepARG*, basadas en redes neuronales, mientras que otras herramientas como *ResFinder* se apoyan en estrategias de búsqueda por homología frente a bases de datos curadas de datos de resistencia.

 **TODO:** Incluir un ejemplo de modelo de ML para la predicción de resistencia a antibióticos para una familia/bacteria concreta

⚠ TODO: Añadir citas a algunos papers con ejemplos de modelos de ML para la predicción de resistencia a antibióticos, y breve descripción de los mismos (A lo mejor en el estado actual)

Estos enfoques han demostrado ser efectivos para inferir la resistencia a antibióticos a partir de representaciones derivadas del análisis genómico, como la presencia o ausencia de genes específicos o funciones biológicas obtenidas tras la anotación de la secuencia. En este sentido, podemos considerarlas como parte del análisis final dentro del flujo de análisis genómico, que utiliza los resultados generados en las etapas anteriores para producir predicciones.

Sin embargo, a pesar de su potencial utilidad, en la práctica su uso es limitado. Una de las principales causas es su falta de integración con los flujos de trabajo estándar de análisis genómico, así como la falta de interfaces unificadas y accesibles para usuarios sin experiencia técnica.

La ejecución de estos modelos suele requerir pasos adicionales de preprocesamiento y transformación de los datos de entrada, que no son parte de los pipelines estándar. Además, la preparación de los datos de entrada suele implicar conversión entre distintos formatos, así como la selección de características relevantes, lo que frecuentemente requiere un esfuerzo manual significativo o el desarrollo de scripts personalizados. A esto se suma la complejidad asociada a la instalación y configuración de los modelos, incluyendo la gestión de dependencias, la configuración de entornos de ejecución y la comprensión de los parámetros de entrada.

Como resultado, estos enfoques se utilizan principalmente en entornos de investigación, especialmente en equipos con experiencia en bioinformática o aprendizaje automático. En cambio, su adopción en entornos generales es todavía limitada. Este escenario resalta la necesidad de soluciones que permitan una integración más directa y fluida de modelos predictivos en el flujo de análisis genómico, reduciendo la fricción entre el procesamiento de datos y su análisis y explotación.

En este contexto, diferentes estudios [1] señalan que, a pesar del creciente interés en sistemas de inteligencia artificial aplicados a la salud, muchos modelos permanecen en la fase de prototipo y no llegan a integrarse en entornos reales. La brecha entre el desarrollo de modelos y su despliegue sigue siendo uno de los principales retos en este ámbito.

La disponibilidad de un framework como el propuesto en este trabajo puede contribuir a reducir esta brecha, al proporcionar un entorno que simplifica el despliegue, la integración y el uso de modelos predictivos dentro de flujos de análisis

genómico completos, favoreciendo así tanto su adopción como la experimentación, validación y mejora de los mismos.

4.2.3. Limitaciones del enfoque actual

Como se ha expuesto, el enfoque actual tanto del flujo de análisis genómico como de los modelos de predicción de resistencia a antibióticos presenta diversas limitaciones que afectan a la eficiencia, la escalabilidad y la adopción de estas herramientas en entornos reales.

En primer lugar, en el caso del análisis genómico, se identifican las siguientes limitaciones principales:

DES-001 - Fragmentación de herramientas Actualmente, el flujo de análisis genómico está compuesto por múltiples herramientas independientes que deben ejecutarse de forma secuencial, sin una integración unificada. Esta fragmentación obliga a gestionar manualmente la transferencia de datos entre etapas y dificulta la automatización del proceso.

DES-002 - Complejidad técnica y barrera de entrada La mayoría de herramientas se ejecutan en entornos Linux mediante línea de comandos, lo que requiere conocimientos técnicos en sistemas operativos, gestión de dependencias y configuración. Esto supone una barrera de entrada para usuarios no especializados, como los profesionales sanitarios, limitando su adopción.

DES-003 - Dependencia de configuración manual y baja automatización La construcción de pipelines de análisis genómico requiere la configuración manual de cada herramienta, la gestión de los datos intermedios y la verificación de compatibilidad entre ellas. Aunque existen plataformas que ofrecen interfaces gráficas, estas no eliminan completamente la complejidad ni la necesidad de tomar decisiones técnicas por parte del usuario.

En segundo lugar, en el caso de los modelos de predicción de resistencia a antibióticos, se identifican las siguientes limitaciones principales:

DES-004 - Falta de integración con flujo de análisis genómico Los modelos de predicción de resistencia a antibióticos suelen desarrollarse como herramientas independientes, especialmente para su uso en entornos de investigación, sin una integración directa con los flujos de análisis genómico estándar. Esto obliga a realizar pasos adicionales de preprocesamiento y adaptación de datos.

DES-005 - Complejidad de configuración y ejecución Muchos modelos requieren la instalación de entornos específicos, la gestión de dependencias, la descargas de bases de datos adicionales y la configuración de parámetros. Esto incrementa la complejidad operativa, especialmente para usuarios no especializados.

DES-006 - Requisitos de preprocesamiento y transformación de datos La ejecución de estos modelos requiere la transformación de datos genómicos a representaciones específicas, como matrices de presencia/ausencia de genes. Este proceso no está automatizado para la mayoría de flujos estándar, lo que implica el desarrollo de scripts personalizados desde los resultados de etapas previas.

DES-007 - Limitaciones de usabilidad e interfaces de usuario La mayoría de modelos no disponen de interfaces gráficas accesibles ni de una documentación orientada a usuarios sin experiencia técnica. Esto limita su uso a perfiles con conocimientos de bioinformática o aprendizaje automático, dificultando su adopción en entornos clínicos o por parte de profesionales sanitarios.

En conjunto, estas limitaciones reflejan una brecha entre la complejidad técnica bioinformática y la necesidad de los usuarios finales, donde la accesibilidad y la integración de herramientas y modelos son aspectos clave a abordar para facilitar su adopción y uso efectivo.

4.3. Usuarios y necesidades

4.3.1. Usuarios

Los principales usuarios a los que está destinado este proyecto son:

Investigadores y profesionales de la salud Usuarios finales de la aplicación, realizan análisis genómicos para obtener información biológica a partir de datos de secuenciación, así como para predecir la resistencia a antibióticos. Su objetivo es obtener resultados de forma eficiente y accesible, sin necesidad de conocimientos técnicos avanzados en bioinformática o aprendizaje automático.

Desarrolladores / Investigadores de modelos predictivos Usuarios especializados en el desarrollo de modelos predictivos para la resistencia a antibióticos. Su interés principal es poder integrar sus modelos en el sistema de forma sencilla, utilizándola como una plataforma o marco de trabajo que facilite su validación y aplicación en entornos reales. Para ello, requieren un mecanismo que permita incorporar nuevos modelos sin necesidad de modificar significativamente la infraestructura del sistema, favoreciendo su utilización y adopción.

4.3.2. Requisitos generales

A partir de las limitaciones del enfoque actual y las necesidades de los distintos perfiles de usuario, se extraen los siguientes requisitos generales del sistema:

RG-001 El sistema deberá proporcionar mecanismos para automatizar el flujo de análisis genómico, incluyendo ensamblaje y anotación en un único proceso

integrado y abstrayendo la complejidad asociada a la ejecución de múltiples herramientas independientes.

RG-002 El sistema deberá permitir el acceso al flujo de análisis tanto de forma completa como a cada una de sus etapas. Se debe poder ejecutar cada etapa de forma independiente si así lo desea el usuario.

RG-003 La arquitectura del sistema deberá facilitar la integración de modelos de predicción de resistencia a antibióticos en el flujo de análisis genómico, permitiendo su ejecución directa a partir de los resultados generados durante las etapas previas.

RG-004 El sistema deberá proporcionar una interfaz gráfica accesible para usuarios sin experiencia técnica, que permita la ejecución de flujos de análisis genómico y modelos de predicción de resistencia a antibióticos de forma sencilla.

RG-005 El sistema deberá minimizar la complejidad asociada a la gestión de herramientas, entornos de ejecución y dependencias, reduciendo la carga cognitiva del usuario.

RG-006 El sistema deberá proporcionar mecanismos que permitan realizar un seguimiento de los procesos de análisis genómico, permitiendo a los usuarios monitorizar el progreso, revisar resultados intermedios y gestionar los datos generados durante el proceso.

4.3.3. Historias de usuario

A continuación, se exponen las historias de usuario derivadas de los requisitos generales, que describen las necesidades y objetivos de los distintos perfiles de usuario:

HU-001:

Como investigador o profesional de la salud

Quiero ejecutar el ensamblaje y anotación de datos genómicos en un único proceso a partir de archivos de secuenciación (FASTQ)

Para analizar muestras genómicas de forma eficiente.

HU-002:

Como investigador o profesional de la salud

Quiero ejecutar etapas individuales del análisis genómico

Para adaptar el flujo a mis necesidades específicas.

HU-003:

Como investigador o profesional de la salud

Quiero utilizar los resultados de una etapa previa del análisis como entrada de etapas posteriores

Para evitar repetir procesamiento innecesarios.

HU-004:

Como investigador o profesional de la salud

Quiero ejecutar modelos de predicción de resistencia a antibióticos a partir de los resultados generados en la anotación de datos genómicos

Para obtener predicciones de resistencia que apoyen la interpretación de la muestra.

HU-005:

Como investigador o profesional de la salud

Quiero visualizar las predicciones de resistencia de forma clara e interpretable

Para utilizarlas como apoyo en la toma de decisiones clínicas o en investigación.

HU-006:

Como investigador o profesional de la salud

Quiero conocer el estado de los procesos de análisis genómico ejecutados

Para monitorizar su progreso y acceder a los resultados.

HU-007:

Como investigador o profesional de la salud

Quiero revisar y descargar los resultados de los procesos de análisis genómico

Para almacenarlos, compartirlos o analizarlos posteriormente.

HU-008:

Como investigador o profesional de la salud

Quiero descargar los resultados de las predicciones de resistencia a antibióticos

Para almacenarlos, compartirlos o analizarlos posteriormente.

HU-009:

Como investigador o profesional de la salud

Quiero recibir una notificación cuando se completen los procesos de análisis genómico que he ejecutado

Para evitar comprobaciones manuales y acceder a los resultados en cuanto estén disponibles.

HU-010:

Como desarrollador o investigador de modelos predictivos

Quiero integrar modelos de predicción en el sistema de forma sencilla

Para facilitar su aplicación en análisis genómicos reales.

HU-011:

Como desarrollador o investigador de modelos predictivos

Quiero usar los resultados generados durante etapas previas del análisis genómico como entrada para modelos de predicción

Para evitar transformaciones manuales de los datos.

4.4. Diseño conceptual de la solución

4.4.1. Enfoque del sistema

Dado el análisis de necesidades realizado, se propone el desarrollo de una plataforma orientada a la integración del flujo de análisis genómico y la predicción de resistencia a antibióticos, como una solución única incorporándolo todo en un único proceso. Se busca un enfoque dual: por un lado, una **interfaz gráfica accesible** e intuitiva que permite ir desde la secuenciación a la predicción de forma sencilla; y por otro, un **framework extensible** que facilita la incorporación de nuevos modelos y componentes por parte de los desarrolladores.

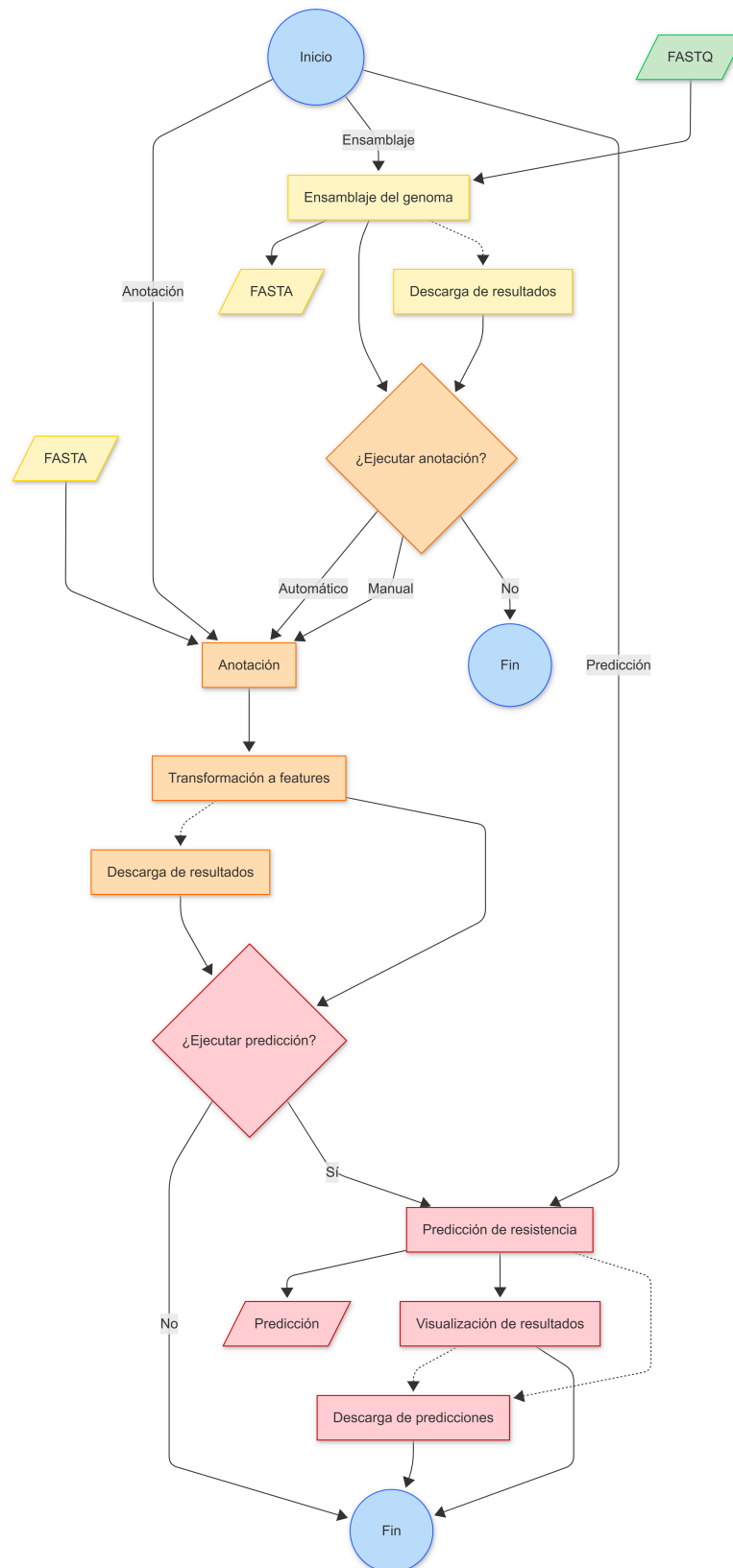
Se identifican tres principios clave:

1. **Abstracción del flujo de análisis genómico:** El sistema incluye la ejecución de todas las etapas del proceso, desde la secuenciación hasta la obtención de los resultados de la predicción, proporcionando una experiencia de uso unificada que elimina la necesidad de gestionar herramientas independientes.
2. **Accesibilidad mediante interfaz gráfica intuitiva:** Se ofrece una interfaz gráfica orientada a usuarios sin experiencia en bioinformática, especialmente profesionales sanitarios o personal investigador, que permite ejecutar el flujo complejo de resistencia sin poseer conocimientos técnicos.
3. **Extensibilidad mediante un enfoque de framework:** El sistema está diseñado como una plataforma extensible que permite a desarrollador e investigadores integrar nuevos modelos predictivos de forma sencilla, favoreciendo su uso en entornos reales.

Con estos principios, se busca dar soporte a los perfiles definidos en la sección de análisis de usuarios, proporcionando una solución para las necesidades de ambos.

4.4.2. Flujo de análisis genómico integrado

El sistema implementa un flujo de procesamiento de datos genómicos en el que cada etapa transforma la información de entrada, generando resultados que pueden ser utilizados en etapas posteriores. Este flujo parte de los datos de secuenciación en formato FASTQ y finaliza con la generación de predicciones de resistencia a antibióticos. Se sigue un flujo modular y flexible, que permite la ejecución completa del pipeline o la ejecución de sus distintas etapas de forma independiente, según las necesidades del usuario. El flujo general se muestra en la [Figura 4.1](#).



■ = secuenciación ■ = ensamblaje ■ = anotación ■ = predicción

Figura 4.1: Flujo de análisis genómico integrado del sistema.

El proceso puede iniciarse desde distintos puntos del sistema. En el caso del flujo completo, los datos de secuenciación en formato FASTQ se utilizan como entrada para iniciar el proceso de **ensamblaje** del genoma, obteniéndose como resultado un conjunto de contigs en formato FASTA.

A partir de estos resultados, el sistema permite la ejecución de la etapa de **anotación**. Esta puede activarse de forma automática al finalizar el ensamblaje, si así se indica al iniciar el proceso, o manual. El sistema toma como entrada los contigs generados en la etapa anterior o el archivo FASTA proporcionado por el usuario, produciendo una representación funcional del genoma.

Posteriormente, esta representación es transformada de forma automática en un conjunto de características (features) compatibles con los modelos de predicción. Este proceso de adaptación de datos se realiza de forma automática dentro del sistema, permitiendo la interoperabilidad entre las etapas del análisis genómico y los modelos predictivos sin intervención manual.

A partir de estas características, el usuario puede ejecutar el proceso de **predicción** de resistencia a antibióticos. En caso de activación, el sistema genera resultados interpretables que muestra de forma visual.

Además, el sistema permite la descarga de resultados en cada una de las etapas principales del flujo (ensamblaje, anotación y predicción). Esto permite adaptar el sistema a distintos usos y necesidades de los usuarios.

En conclusión, el flujo desacopla las distintas etapas del análisis genómico mediante un diseño modular, permitiendo su ejecución de forma encadenada o independiente, proporcionando así flexibilidad para su aplicación.

4.4.3. Alcance del sistema

El sistema propuesto se centra en la integración y orquestación del flujo de análisis genómico y de modelos predictivos ya existentes, actuando como una capa de abstracción sobre las herramientas consolidadas en la bioinformática.

En este sentido, el sistema no pretende reemplazar las herramientas existentes ni desarrollar nuevos modelos de predicción, sino facilitar su uso mediante una plataforma unificada que mejore su usabilidad, accesibilidad e integración.

El alcance del sistema incluye la integración de herramientas en un flujo unificado, la automatización de las etapas del flujo de análisis genómico, la gestión de datos intermedios, la integración de modelos predictivos y la presentación de resultados de forma clara e interpretable para el usuario. Adicionalmente, se incluye la implementación de un mecanismo que permita la incorporación de nuevos modelos y procesamiento de datos desde el anotado de forma sencilla.

Por el contrario, quedan fuera del alcance aspectos como el desarrollo de nuevas herramientas de ensamblaje o anotación, la mejora de herramientas ya existentes o la investigación de nuevos modelos de predicción.

En conjunto, el sistema se orienta a reducir la barrera técnica existente en el proceso de análisis genómico y el uso de modelos de predicción de resistencia a antibióticos, proporcionando una interfaz unificada que abstrae dicha complejidad.

 **TODO:** Mencionar algo del desarrollo del servicio de conversiones??

4.5. Catálogo de requisitos

Se presenta a continuación un catálogo de requisitos que se deriva del análisis realizado en las secciones anteriores, estructurado en casos de uso, requisitos de información, requisitos funcionales y requisitos no funcionales, que servirán como base para el diseño detallado e implementación del sistema.

4.5.1. Casos de uso

En esta sección se presentan los casos de uso derivados de las historias de usuario. Cada caso de uso describe el comportamiento del sistema, incluyendo el flujo principal, los resultados esperados y las posibles excepciones o errores que pueden ocurrir durante su ejecución.

Actores

Los actores principales que interactúan en los casos de uso son:

ACT-001 - Usuario investigador o profesional de la salud utiliza el sistema para realizar análisis genómicos y ejecutar modelos de predicción de resistencia a antibióticos, con el objetivo de obtener información biológica a partir de datos de secuenciación y apoyar la interpretación de muestras genómicas.

ACT-002 - Sistema de análisis genómico y predicción de resistencia a antibióticos Sistema propuesto que integra el flujo completo de análisis genómico y modelos predictivos, proporcionando una interfaz accesible para el procesamiento de datos biológicos.

4.5.2. Diagramas y definición de casos de uso

Todos los casos de uso del sistema requieren que el usuario (ACT-001), investigador o profesional de la salud, esté autenticado en el sistema mediante una cuenta de usuario válida.

Asimismo, el sistema (ACT-002) debe garantizar la seguridad y privacidad de los datos del usuario, implementando mecanismos de control de propiedad, garantizando que únicamente se puedan consultar o modificar entidades pertenecientes al usuario autenticado, como procesos de análisis genómico, resultados o notificaciones asociadas a su cuenta.

Pipeline de análisis genómico

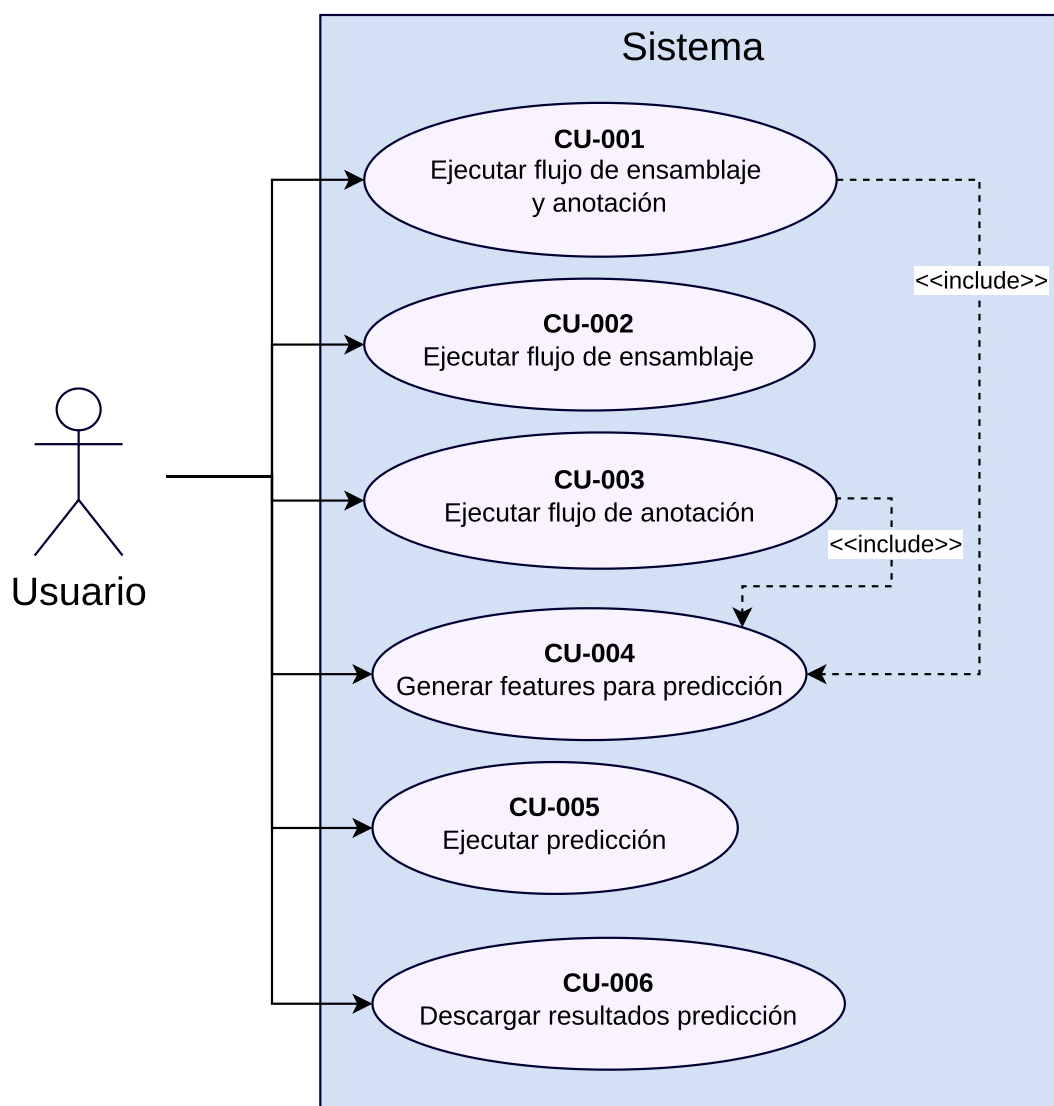


Figura 4.2: Diagrama de casos de uso para el flujo de análisis genómico integrado

CU-001	Ejecutar flujo de ensamblaje y anotación
Precondición	El usuario (ACT-001) dispone de datos de secuenciación en formato FASTQ.
Descripción	Permite ejecutar el flujo completo de análisis genómico, desde el ensamblaje hasta la anotación, incluyendo la transformación automática de resultados a características (features) compatibles con modelos de predicción.
Flujo principal	1. El usuario (ACT-001) accede a la sección Ensamblaje del sistema. 2. El usuario (ACT-001) selecciona el tipo de datos de entrada. 3. El actor (ACT-001) proporciona los datos de secuenciación en formato FASTQ. 4. El actor (ACT-001) selecciona la opción de ejecutar el flujo completo de análisis genómico e inicia el proceso. 5. El sistema (ACT-002) inicia el ensamblaje utilizando los datos proporcionados. 6. El sistema (ACT-002) genera contigs en formato FASTA. 7. El sistema (ACT-002) ejecuta la anotación utilizando los contigs generados en la etapa anterior. 8. El sistema (ACT-002) genera una representación funcional del genoma como resultado de la anotación. 9. El sistema (ACT-002) transforma automáticamente la representación funcional en features.
Postcondición	Se generan resultados de ensamblaje y anotación en el sistema, que pueden ser descargados o utilizados para predicciones. Las features se encuentran calculadas y almacenadas en el sistema, listas para su uso en modelos de predicción.
Excepciones	• Datos inválidos: se muestra un mensaje explicativo al usuario. • Error en el proceso de ensamblaje o anotación: se marca el proceso como fallido y se informa al usuario.

CU-002	Ejecutar flujo de ensamblaje
Precondición	El usuario (ACT-001) dispone de datos de secuenciación en formato FASTQ.
Descripción	Permite ejecutar la etapa de ensamblaje a partir de datos de secuenciación.

CU-002	Ejecutar flujo de ensamblaje
Flujo principal	1. El usuario (ACT-001) accede a la sección Ensamblaje del sistema. 2. El usuario (ACT-001) selecciona el tipo de datos de entrada. 3. El actor (ACT-001) proporciona los datos de secuenciación en formato FASTQ. 4. El sistema (ACT-002) inicia el proceso de ensamblaje utilizando los datos proporcionados. 5. El sistema (ACT-002) genera contigs en formato FASTA como resultado del ensamblaje.
Postcondición	Se genera un archivo FASTA con los contigs ensamblados, que puede ser descargado o utilizado para etapas posteriores.
Excepciones	• Datos inválidos: se muestra un mensaje explicativo al usuario. • Error en el proceso de ensamblaje: se marca el proceso como fallido y se informa al usuario.

CU-003	Ejecutar flujo de anotación
Precondición	El usuario (ACT-001) dispone de datos lecturas en formato FASTA o un proceso de ensamblaje anterior que no ha sido anotado todavía.
Descripción	Permite ejecutar la etapa de anotación.
Flujo principal	1. El usuario (ACT-001) accede a la sección Anotación del sistema. 2. El usuario (ACT-001) selecciona el archivo de entrada para la anotación, que puede ser un archivo FASTA con contigs ensamblados o un proceso de ensamblaje previo sin anotación asociada. 3. El sistema (ACT-002) inicia el proceso de anotación utilizando los datos proporcionados. 4. El sistema (ACT-002) genera una representación funcional del genoma como resultado de la anotación. 5. El sistema (ACT-002) transforma automáticamente la representación funcional en un conjunto de características compatibles con modelos de predicción.
Postcondición	Se permite al usuario acceder a los datos de la anotación, que puede ser descargado o utilizado para etapas posteriores y

CU-003	Ejecutar flujo de anotación
	las features se encuentran calculadas y almacenadas en el sistema, disponibles para su uso en modelos de predicción.
Excepciones	• Datos inválidos: se muestra un mensaje explicativo al usuario. • Error en el proceso de anotación: se marca el proceso como fallido y se informa al usuario.

CU-004	Generar features para predicción
Precondición	El usuario (ACT-001) dispone de resultados de anotación válidos.
Descripción	Permite cargar resultados de anotación externos y generar features compatibles con los modelos de predicción.
Flujo principal	1. El usuario (ACT-001) entra en la sección Anotación del sistema. 2. El usuario (ACT-001) selecciona el archivo de entrada. 3. El sistema (ACT-002) inicia el proceso de generación de features con los datos proporcionados. 4. El sistema (ACT-002) transforma automáticamente la representación funcional en un conjunto de características compatibles con modelos de predicción.
Postcondición	Las features se encuentran calculadas y almacenadas en el sistema, disponibles para su uso en modelos de predicción.
Excepciones	• Datos inválidos: se muestra un mensaje explicativo al usuario. • Error en el proceso de generación de features: se marca el proceso como fallido y se informa al usuario.

CU-005	Ejecutar predicción de resistencia a antibióticos
Precondición	El usuario (ACT-001) cuenta con características generadas.
Descripción	Permite ejecutar la predicción de resistencia a antibióticos.
Flujo principal	1. El usuario (ACT-001) entra en la sección Predicción del sistema. 2. El usuario (ACT-001) selecciona los modelos de predicción a ejecutar, los antibióticos y las características a utilizar como entrada.

CU-005	Ejecutar predicción de resistencia a antibióticos
	3. El sistema (ACT-002) ejecuta los modelos de predicción para los antibióticos y las características seleccionadas. 4. El sistema (ACT-002) genera resultados de predicción de resistencia a antibióticos. 5. El sistema (ACT-002) muestra los resultados de forma visual e interpretable.
Postcondición	Se generan resultados de predicción y se muestran al usuario de forma visual e interpretable.
Excepciones	• Combinación de modelos y antibióticos inválida: se muestra un mensaje explicativo al usuario. • Error en el modelo: se muestra un mensaje explicativo.

CU-006	Descargar resultados de predicción de resistencia a antibióticos
Precondición	El usuario (ACT-001) cuenta con procesos de anotación o generación de features finalizados con éxito.
Descripción	Permite descargar los resultados de predicción de resistencia a antibióticos.
Flujo principal	1. El usuario (ACT-001) entra en la sección Predicción del sistema. 2. El usuario (ACT-001) selecciona los modelos de predicción a ejecutar, los antibióticos y las características a utilizar como entrada. 3. Si no se ha generado ya la predicción, el sistema (ACT-002) ejecuta los modelos de predicción para los antibióticos. 4. El sistema (ACT-002) descarga los resultados de predicción de resistencia a antibióticos.
Postcondición	El archivo es descargado por el usuario.
Excepciones	• Combinación de modelos y antibióticos inválida: se muestra un mensaje explicativo al usuario. • Error en el modelo: se muestra un mensaje explicativo. • Error en la descarga: se muestra un mensaje explicativo al usuario.

Notificaciones

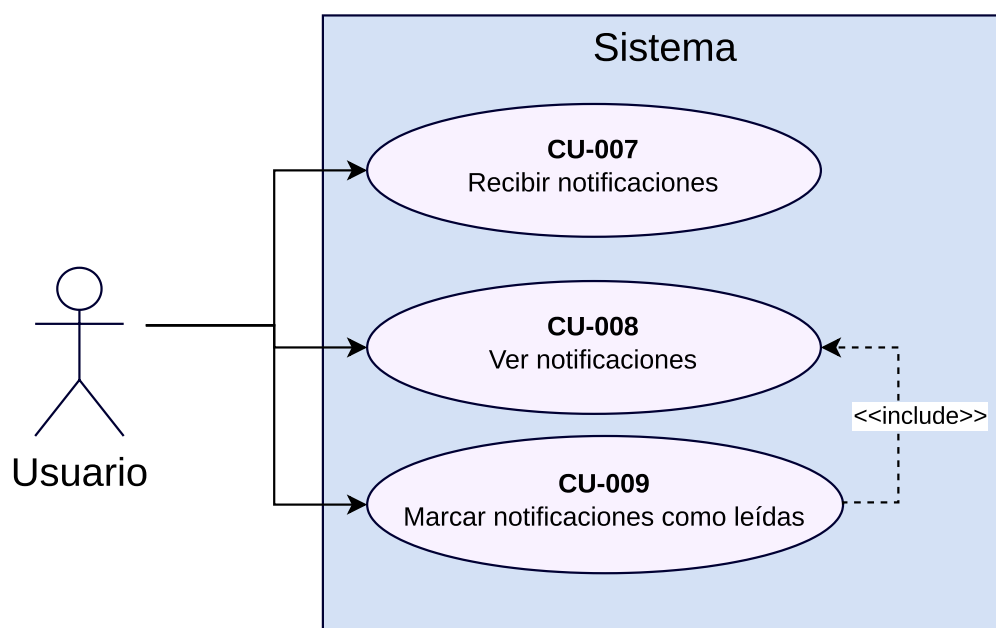


Figura 4.3: Diagrama de casos de uso para el flujo de notificaciones del sistema

CU-007	Recibir notificaciones
Precondición	El usuario (ACT-001) ha iniciado procesos de análisis genómico.
Descripción	Permite recibir notificaciones sobre el estado de los procesos (inicio, finalización correcta y error en la ejecución).
Flujo principal	1. El sistema (ACT-002) detecta un evento (inicio, finalización correcta o error) en un proceso de análisis genómico iniciado por el usuario (ACT-001). 2. El sistema (ACT-002) envía una notificación al usuario (ACT-001) informando sobre el evento ocurrido.
Postcondición	Se ha enviado una notificación al usuario (ACT-001) informando sobre el evento ocurrido.
Excepciones	Ninguna.

CU-008	Ver notificaciones
Precondición	El usuario (ACT-001) ha recibido alguna notificación.
Descripción	Permite visualizar el histórico de notificaciones generadas por el sistema para el usuario (ACT-001).
Flujo principal	1. El usuario (ACT-001) accede al panel de notificaciones. 2. El sistema (ACT-002) muestra el histórico de notificaciones para el usuario (ACT-001).

CU-008	Ver notificaciones
Postcondición	El usuario (ACT-001) visualiza el histórico de notificaciones generadas por el sistema.
Excepciones	No existen notificaciones: se informa al usuario.

CU-009	Marcar notificaciones como leídas
Precondición	El usuario (ACT-001) cuenta con notificaciones no leídas.
Descripción	Permite marcar una o todas las notificaciones como leídas para gestionar el histórico de notificaciones.
Flujo principal	- El usuario (ACT-001) accede al panel de notificaciones. - El usuario (ACT-001) selecciona la opción de marcar como leída una notificación o todas. - El sistema (ACT-002) actualiza el estado de las notificaciones seleccionadas a leídas.
Postcondición	Las notificaciones quedan registradas como leídas.
Excepciones	No existen las notificaciones seleccionadas: se informa al usuario.

Gestión de procesos

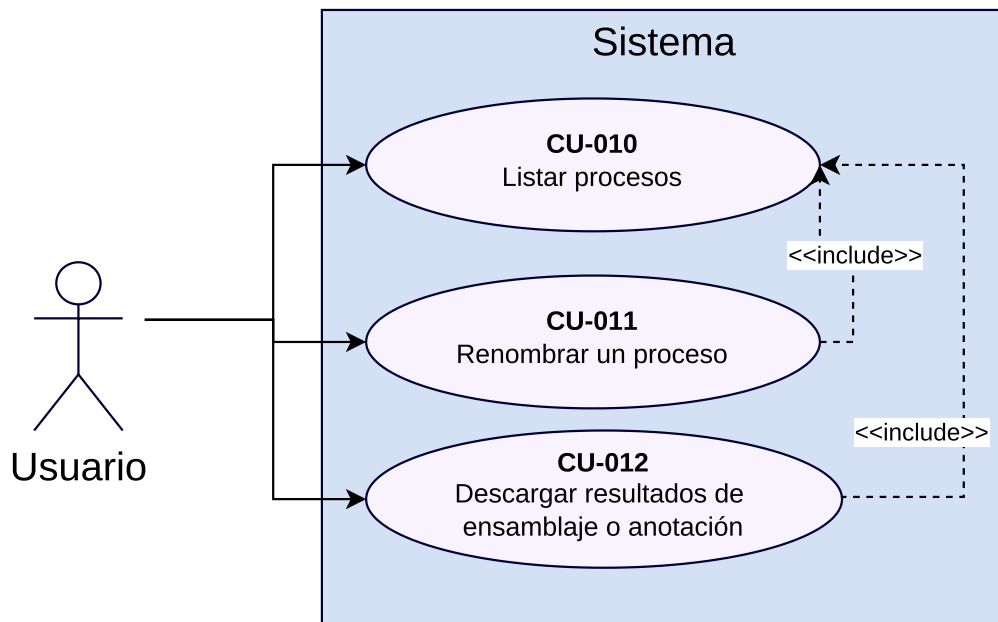


Figura 4.4: Diagrama de casos de uso para el flujo de gestión de procesos

CU-010	Listar procesos
Precondición	El usuario (ACT-001) ha ejecutado algún proceso de ensamblaje o anotación.
Descripción	Permite visualizar la lista de procesos ejecutados por el usuario (ACT-001) con su estado (en ejecución, finalizado correctamente o error).
Flujo principal	1. El usuario (ACT-001) accede a la sección Procesos del sistema. 2. El sistema (ACT-002) muestra la lista de procesos ejecutados por el usuario (ACT-001) con su estado.
Postcondición	El usuario (ACT-001) visualiza la lista de procesos ejecutados con su estado.
Excepciones	• No existen procesos para el usuario: se informa al usuario.

CU-011	Renombrar un proceso
Precondición	El usuario (ACT-001) ha ejecutado algún proceso de ensamblaje o anotación.
Descripción	Permite cambiar el nombre de un proceso ejecutado por el usuario (ACT-001) para facilitar su identificación.
Flujo principal	1. El usuario (ACT-001) accede a la sección Procesos del sistema. 2. El sistema (ACT-002) muestra la lista de procesos ejecutados por el usuario (ACT-001) con su estado. 3. El usuario (ACT-001) selecciona un proceso de la lista. 4. El usuario (ACT-001) selecciona la opción de renombrar el proceso. 5. El usuario (ACT-001) introduce el nuevo nombre para el proceso. 6. El sistema (ACT-002) actualiza el nombre del proceso seleccionado.
Postcondición	El proceso queda actualizado con el nuevo nombre.
Excepciones	• Nombre inválido: se muestra un mensaje explicativo al usuario. • No existe el proceso seleccionado: se muestra un mensaje explicativo al usuario.

CU-012	Descargar resultados de ensamblaje o anotación
Precondición	El usuario (ACT-001) cuenta con procesos de ensamblaje o anotación finalizados con éxito.
Descripción	Permite descargar los resultados de ensamblaje o anotación.
Flujo principal	1. El usuario (ACT-001) entra en la sección Procesos del sistema. 2. El usuario (ACT-001) selecciona un proceso de ensamblaje o anotación finalizado. 3. El usuario (ACT-001) selecciona la opción de descargar los resultados. 4. El sistema (ACT-002) inicia la descarga de los resultados.
Postcondición	El archivo es descargado por el usuario.
Excepciones	• Los resultados no están disponibles para descarga: se muestra un mensaje explicativo al usuario.

4.5.3. Requisitos de información

En esta sección se describen los requisitos de información del sistema, derivados de los casos de uso con el objetivo de soportar el flujo completo de análisis genómico, desde la carga de datos hasta la generación de predicciones y notificaciones.

RI-001: Usuario

El sistema deberá gestionar los usuarios autenticados, que actuarán como propietarios de los datos generados en el sistema

RI-002: Archivos subidos

El sistema deberá almacenar los archivos subidos por los usuarios, incluyendo:

- Ruta del archivo.
- Fecha de subida.
- Tipo de archivo (FASTA, JSON...).
- Usuario propietario.

RI-003: Genes

El sistema deberá almacenar información de genes para su uso como features de predicción, incluyendo:

- Identificadores del gen.
- Relación con los archivos subidos.

RI-004: Información sobre la presencia de un gen en un archivo

El sistema deberá almacenar información sobre la presencia de un gen en un archivo, incluyendo:

- Gen asociado.
- Archivo de origen.
- Información del experto (expert).
- Posición del gen en la secuencia (start, stop).
- Secuencias asociadas al gen (nucleotídica, nt; y proteica, aa).

RI-005: Proceso de análisis

El sistema deberá almacenar procesos de análisis genómico y transformación ejecutados, incluyendo:

- Tipo de proceso (ensamblaje, anotación, generación de features).
- Estado (pendiente, en ejecución, completado, fallido).
- Nombre asignado por el usuario.
- Ruta del archivo de entrada.
- Usuario propietario.
- Relación con procesos previos (por ejemplo, un proceso de anotación puede estar relacionado con un proceso de ensamblaje previo).

RI-006: Tipos de procesos

El sistema deberá soportar distintos tipos de tareas:

- Ensamblaje ONT.
- Ensamblaje Illumina.
- Ensamblaje con anotación.
- Anotación.
- Generación de features desde JSON de anotación.

RI-007: Notificaciones de procesos

El sistema deberá almacenar notificaciones asociadas a procesos, incluyendo:

- Usuario propietario.
- Proceso asociado.
- Tipo de evento (inicio, completado, alerta, fallo).
- Mensaje.
- Estado de lectura (leída/no leída).
- Canales de notificación (email o notificación en la aplicación).

RI-008: Preferencias de notificación

El sistema deberá almacenar las configuraciones de notificación por usuario, incluyendo:

- Usuario.
- Si desea recibir notificaciones por email.

A partir de los requisitos de información definidos, se deriva el Diagrama de Entidad-Relación (E-R), **Figura 4.5**, recogiendo las principales entidades del sistema y sus relaciones.

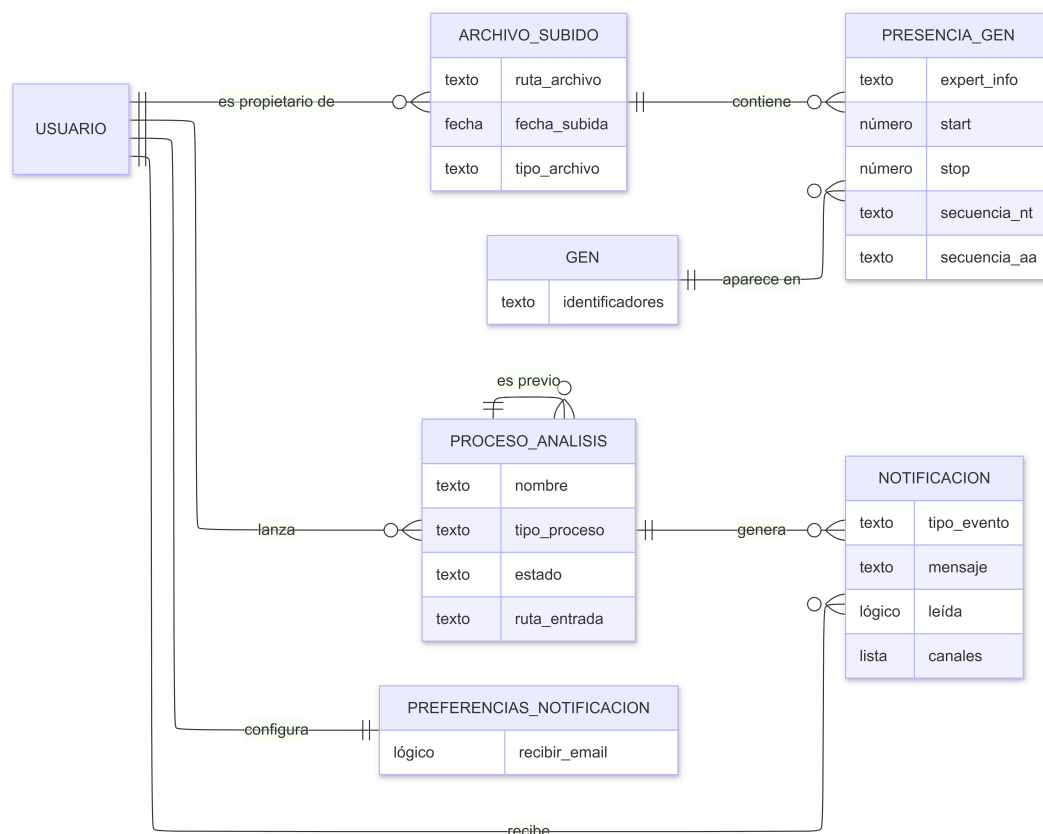


Figura 4.5: Diagrama Entidad-Relación

El modelo se estructura alrededor del usuario como elemento central, que actúa como propietario de los datos generados y como ejecutor de los procesos de análisis genómico. Los archivos subidos constituyen el punto de entrada de los datos genómicos, que son transformados a través de los distintos procesos de análisis genómico. A partir de los resultados de los procesos de anotación se generan las features, los genes y la información asociada a su presencia en cada muestra.

Los procesos de análisis pueden encadenarse, permitiendo modelar las dependencias entre las distintas etapas del análisis. Por ejemplo, un proceso de anotación puede depender de un proceso de ensamblaje previo, indicando el origen de los datos utilizados.

Las notificaciones actúan como mecanismo de comunicación, informando al usuario sobre el estado de sus procesos, ya que estos pueden requerir tiempos de ejecución prolongados. Finalmente, las preferencias de usuario permiten personalizar la recepción de estas notificaciones.

4.5.4. Requisitos funcionales

De los casos de uso y los requisitos de información se derivan los siguientes requisitos funcionales, que describen las funcionalidades específicas que el sistema debe

implementar para cumplir con las necesidades de los usuarios y soportar el flujo completo de análisis genómico y predicción de resistencia a antibióticos.

RF-001: Ensamblaje de datos genómicos

El sistema deberá permitir crear tareas de ensamblaje a partir de archivos FASTQ.

RF-002: Selección del tipo de ensamblaje

El sistema deberá permitir seleccionar el tipo de datos de entrada (originados por secuenciación Illumina o por secuenciación de ONT) para el ensamblaje, aplicando el proceso de ensamblaje adecuado para cada tipo de datos.

RF-003: Anotación de datos genómicos

El sistema deberá permitir ejecutar tareas de anotación a partir de archivos FASTA o procesos previos de ensamblaje sin anotación asociada, generando automáticamente features para la aplicación de modelos predictivos a partir de los resultados de anotación.

RF-004: Ejecución de flujo genómico completo

El sistema deberá permitir ejecutar tareas de anotación de forma automática tras el ensamblaje, generando automáticamente features para la aplicación de modelos predictivos a partir de los resultados de anotación.

RF-005: Generación de features

El sistema deberá permitir generar features a partir de los resultados de anotación en formatos soportados.

RF-006: Registro de tareas

El sistema deberá registrar los procesos de análisis genómico ejecutados por el usuario.

RF-007: Estado de procesos

El sistema deberá actualizar automáticamente el estado de los procesos durante su ejecución.

RF-008: Nombrado de procesos

El sistema deberá permitir asignar y modificar el nombre de los procesos para facilitar su identificación.

RF-009: Subida de archivos

El sistema deberá permitir subir archivos asociados a un usuario.

RF-010: Descarga de archivos

El sistema deberá permitir la descarga de los archivos subidos por el usuario o generados en las distintas etapas del análisis.

RF-011: Predicciones de resistencia a antibióticos

El sistema deberá permitir ejecutar modelos de predicción de resistencia a antibióticos a partir de las características generadas.

RF-012: Visualización de predicciones

El sistema deberá permitir visualizar los resultados de las predicciones de resistencia a antibióticos de forma clara e interpretable para el usuario.

RF-013: Integración de modelos predictivos

El sistema deberá permitir la integración de modelos de predicción que utilicen las características generadas.

RF-014: Generación de notificaciones

El sistema deberá generar notificaciones cuando un proceso de análisis genómico (ensamblaje o anotación) se inicie, se complete correctamente o falle.

RF-015: Consulta de notificaciones

El sistema deberá permitir consultar las notificaciones generadas para el usuario.

RF-016: Marcar notificaciones como leídas

El sistema deberá permitir marcar notificaciones como leídas.

RF-017: Notificaciones por email

El sistema deberá permitir enviar notificaciones por email al usuario si este activa esta opción en sus preferencias de notificación.

RF-018: Autenticación

El sistema deberá requerir autenticación para acceder a sus funcionalidades.

RF-019: Gestión de cuentas

El sistema deberá permitir a los usuarios registrarse y autenticarse mediante credenciales.

RF-020: Control de acceso

El sistema deberá restringir el acceso a recursos únicamente a su usuario propietario.

4.5.5. Requisitos no funcionales

De las necesidades de los usuarios y del análisis del problema y la solución propuesta, se derivan los siguientes requisitos no funcionales.

Rendimiento y eficiencia

RNF-001: Ejecución asíncrona

El sistema deberá permitir la ejecución de tareas de análisis genómico de forma no bloqueante, de manera que el usuario pueda continuar interactuando con la interfaz durante su procesamiento.

 **TODO:** A lo mejor hay que dar un número específico y hacer los requisitos no funcionales menos generales

RNF-002: Concurrencia

El sistema deberá permitir la ejecución concurrente de múltiples procesos de análisis genómico sin que esto afecte negativamente al rendimiento o la estabilidad del sistema.

RNF-003: Gestión de almacenamiento

El sistema deberá gestionar los archivos de entrada y salida de forma eficiente, eliminando aquellos que no sean necesarios tras su procesamiento.

Escalabilidad y extensibilidad

RNF-004: Extensibilidad de modelos predictivos

El sistema deberá permitir la incorporación de nuevos modelos de predicción de resistencia a antibióticos sin necesidad de modificar la infraestructura o el flujo base del sistema.

RNF-005: Extensibilidad de formatos de entrada

El sistema deberá permitir la incorporación de nuevos formatos de entrada para la generación de características mediante mecanismos desacoplados como parsers o adaptadores.

Mantenibilidad y modularidad

RNF-006: Arquitectura desacoplada

El sistema deberá estar estructurado en componentes desacoplados que separen responsabilidades.

RNF-007: Modularidad

El sistema deberá estar diseñado de forma modular, permitiendo la evolución independiente de sus componentes sin afectar al resto del sistema.

Seguridad

RNF-008: Integridad de datos

El sistema deberá garantizar la integridad de los datos mediante validaciones en las entidades y restricciones.

RNF-009: Control de acceso

El sistema deberá garantizar que los usuarios solo puedan acceder a los recursos de su propiedad.

Trazabilidad

RNF-010: Trazabilidad del flujo

El sistema deberá permitir reconstruir el flujo de análisis genómico completo a partir de las relaciones entre tareas y sus resultados.

Usabilidad y accesibilidad

RNF-011: Facilidad de uso

El sistema deberá proporcionar una interfaz gráfica que abstraiga la complejidad técnica del pipeline de análisis genómico.

Portabilidad e interoperabilidad

RNF-012: Despliegue

El sistema deberá poder desplegarse en entornos Linux utilizando tecnologías estándar.

RNF-013: Interoperabilidad

El sistema deberá permitir la importación y exportación de datos en formatos estándar (FASTA, JSON, CSV...) para su uso en otras herramientas o plataformas.

4.6. Conclusiones

En este capítulo se ha descrito el análisis del problema y la solución propuesta para el desarrollo de un sistema de análisis genómico integrado con modelos de predicción de resistencia a antibióticos. Adicionalmente, se han definido los casos de uso, los requisitos de información, los requisitos funcionales y no funcionales que guiarán el diseño e implementación del sistema. Estos requisitos reflejan la necesidad de un sistema no solo funcional, sino también robusto, extensible y adecuado para entornos reales de análisis bioinformático.

Diseño de la solución

5.1. Introducción

En este capítulo se explica el diseño de la solución propuesta para abordar el problema de la predicción de resistencia a antibióticos a partir de datos genómicos. Se describe la arquitectura del sistema, los módulos que lo componen, los flujos principales de ejecución y las decisiones de diseño, tanto patrones como decisiones técnicas. Además, se presenta un mockup de la interfaz de visualización de resultados de predicción, uno de los aspectos clave del sistema, diseñado para facilitar la interpretación de los resultados por parte de los usuarios.

5.2. Visión general de la arquitectura

5.2.1. Diagrama de arquitectura

El sistema se ha diseñado siguiendo una estructura distribuida compuesta por tres bloques principales, **Figura 5.1**: la capa de usuario, el sistema web y el sistema bioinformático. Esta separación permite aislar responsabilidades, mejorar la escalabilidad y facilitar la evolución independiente de cada componente.

En la **capa de usuario**, el usuario interactúa con el sistema a través de una interfaz web que actúa como punto de entrada a la funcionalidad del sistema. Esta

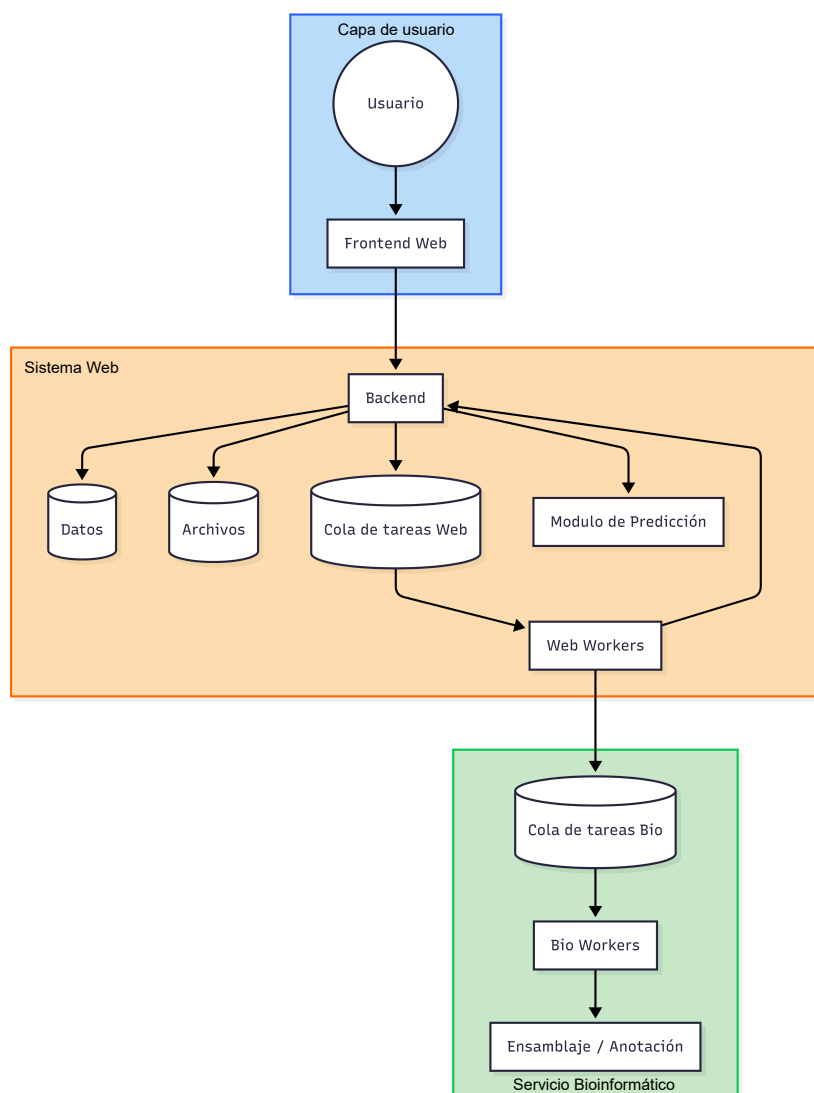


Figura 5.1: Diagrama de arquitectura del sistema a alto nivel

interfaz permite la ejecución de procesos de análisis genómico y predicciones, la gestión de tareas y la consulta de resultados. Se ha diseñado con el objetivo de abstraer la complejidad técnica, para que sea intuitiva y accesible, facilitando la interacción con el sistema incluso para usuarios sin experiencia técnica.

El **sistema web** es el núcleo de la aplicación, el encargado de gestionar la lógica de negocio. Este bloque incluye un backend que expone una API, encargada de procesar las peticiones del usuario, gestionando el persistencia de datos y ficheros y coordinando la ejecución de tareas. Para ello, el sistema dispone de:

- Un sistema de persistencia para almacenar datos estructurados (procesos, notificaciones, usuarios, etc.).
- Un sistema de almacenamiento de archivos para gestionar los datos de entrada y salida (FASTA, JSON, entre otros).

- Un mecanismo de ejecución asíncrona que permite delegar procesos de larga duración sin bloquear la interacción del usuario, para así cumplir con los requisitos de rendimiento y eficiencia definidos.
- Un módulo de predicción, encargado de ejecutar modelos de resistencia a antibióticos a partir de las características generadas en el pipeline.

Adicionalmente, el sistema web cuenta con un sistema de notificaciones para informar a los usuarios sobre el estado de sus tareas, así como mecanismos de autenticación y control de acceso para garantizar la seguridad de la información.

El **sistema bioinformático** es el encargado de la ejecución de los procesos de análisis genómico, como el ensamblaje y la anotación. Este sistema se encuentra desacoplado del sistema web, disponiendo de su propia API y mecanismo de gestión de tareas asíncronas. La interacción entre ambos sistemas se realiza mediante el envío de tareas desde el sistema web hacia el sistema bioinformático, que procesa dichas tareas y gestiona su ejecución de forma independiente. Esta separación permite mejorar la escalabilidad y facilita la integración de nuevas herramientas sin afectar al resto del sistema.

El flujo general de ejecución comienza cuando el usuario inicia un proceso desde la interfaz web. El backend registra la tarea y la delega para su procesamiento mediante un mecanismo de workers asíncronos. Estos workers se encargan de coordinar la ejecución del proceso en el sistema bioinformático, que realiza el análisis correspondiente. Una vez finalizado, los resultados son recuperados por el sistema web, que actualiza el estado de la tarea y los pone a disposición del usuario para su consulta o para su uso en etapas posteriores, como la predicción de resistencia a antibióticos.

Este enfoque presenta características propias de una arquitectura distribuida basada en servicios, adoptando principios asociados a arquitecturas de microservicios, como el desacoplamiento entre componentes y la comunicación asíncrona entre subsistemas. No obstante, no se trata de una arquitectura de microservicios en sentido estricto, ya que el sistema web mantiene una estructura unificada. Esta decisión permite equilibrar simplicidad y mantenibilidad dentro del alcance del proyecto. Así, se busca cumplir con requisitos no funcionales como la escalabilidad, la extensibilidad y la trazabilidad del flujo de procesamiento, facilitando la integración de nuevas herramientas bioinformáticas y modelos predictivos.

5.2.2. Diagrama de despliegue

Para el despliegue del sistema se ha optado por una arquitectura basada en contenedores, utilizando Docker para la gestión de los mismos. Esta elección permite una mayor flexibilidad y portabilidad, facilitando la implementación en diferentes

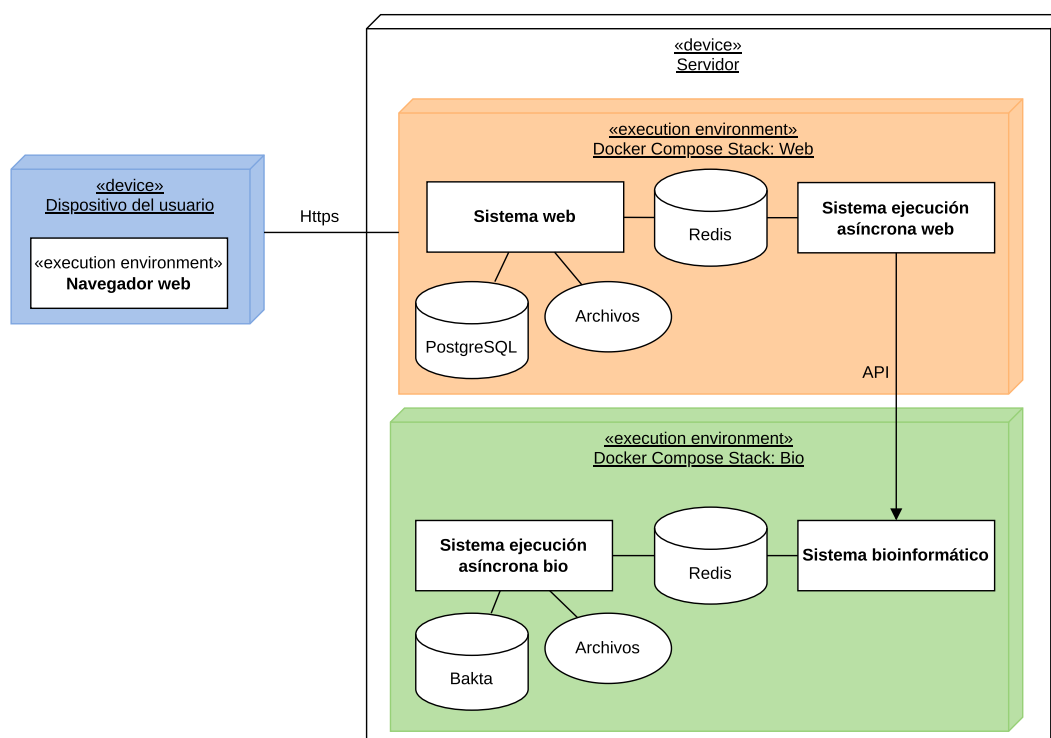


Figura 5.2: Diagrama de despliegue del sistema

entornos y la escalabilidad del sistema. El diagrama de despliegue se muestra en la **Figura 5.2**.

Para el despliegue del sistema se ha optado por una arquitectura basada en contenedores, utilizando Docker para la gestión de los mismos. Esta elección permite una mayor flexibilidad y portabilidad, facilitando la implementación en diferentes entornos, muy importante para el objetivo del proyecto, y la escalabilidad del sistema. El diagrama de despliegue se muestra en la **Figura 5.2**.

Se han definido dos subsistemas principales, el **sistema web** y el **sistema bioinformático**, desplegado en entornos independientes. El sistema web gestiona la lógica de negocio, los usuarios, el almacenamiento de datos, la obtención de features y la ejecución de modelos predictivos. Por otro lado, el sistema bioinformático se concentra en la ejecución de los procesos de análisis genómico, ensamblaje y anotación.

Cada subsistema se compone de múltiples servicios desplegados en contenedores, incluyendo bases de datos, volúmenes y componentes de ejecución asíncrona basados en colas de tareas y workers, lo que permite gestionar procesos de larga duración sin bloquear la interacción con el usuario. Esta separación facilita el escalado independiente de los distintos componentes en función de su carga de trabajo.

La comunicación entre ambos sistemas se realiza a través de interfaces bien definidas basadas en APIs, permitiendo un acoplamiento débil entre ellos. Esto

facilita la sustitución o evolución independiente de cada subsistema sin afectar al resto del sistema.

El usuario accede al sistema a través de un cliente web, típicamente el navegador de su dispositivo, que se comunica con el sistema web a través de peticiones HTTP. El sistema web puede desplegarse en diferentes entornos (locales o en la nube), mientras que el sistema bioinformático puede ubicarse en infraestructuras optimizadas para el procesamiento intensivo de datos, lo que permite adaptar el despliegue a las necesidades de rendimiento del sistema.

5.3. Descomposición en módulos

Para la comprensión de las responsabilidades y la organización del sistema, se ha realizado una descomposición en módulos de cada uno de los bloques principales. Esta descomposición se ha llevado a cabo siguiendo principios de diseño modular y separación de responsabilidades, con el objetivo de facilitar la mantenibilidad, la escalabilidad y la evolución del sistema. La descomposición en módulos presentada es una organización coceptual del sistema, basada en responsabilidades, no necesariamente una correspondencia directa con la estructura final del código o con las aplicaciones del framework utilizado.

5.3.1. Módulos del sistema web

Los módulos que componen el sistema web, mostrados en la figura **Figura 5.3**, son los siguientes:

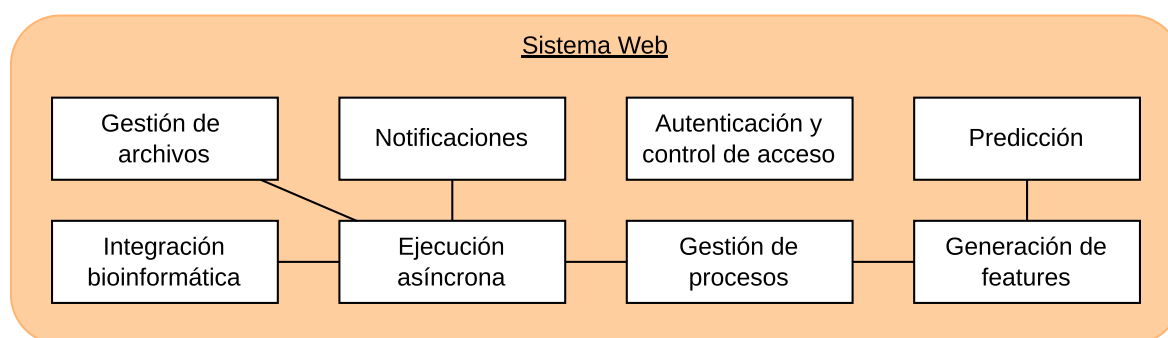


Figura 5.3: Diagrama de módulos del sistema web

- **Módulo de autenticación y control de acceso:** responsable de la gestión de usuarios y de garantizar que el acceso a los recursos esté restringido a su propietario.
- **Módulo de gestión de procesos:** encargado de la creación, seguimiento y gestión del ciclo de vida de los procesos de análisis genómico, incluyendo su estado y relaciones entre tareas. Este módulo se encarga de registrar las tareas

iniciadas por los usuarios, gestionar su estado a lo largo del tiempo y coordinar la ejecución de las mismas.

- **Módulo de gestión de archivos:** responsable del almacenamiento y organización de los archivos de entrada y salida del sistema, como datos genómicos y resultados intermedios o finales. Adicionalmente, se encarga de la gestión de datos persistentes y datos temporales, incluyendo su eliminación cuando ya no son necesarios para mejorar la eficiencia.
- **Módulo de ejecución asíncrona:** responsable de la gestión de tareas de larga duración mediante mecanismos desacoplados, permitiendo la ejecución no bloqueante de los procesos de análisis genómico.
- **Módulo de integración bioinformática:** encargado de la comunicación con el sistema bioinformático externo, gestionando el envío de tareas, la monitorización de su estado y la recuperación de resultados mediante su API.
- **Módulo de generación de features:** encargado de la extracción y procesamiento de características a partir de los resultados de anotación, generando la información necesaria para alimentar los modelos de predicción de resistencia a antibióticos. Se trata de un módulo extensible que permite incorporar nuevos parsers o transformaciones de forma sencilla, facilitando la evolución y mejora continua del sistema.
- **Módulo de predicción:** responsable de la ejecución de modelos de resistencia a antibióticos a partir de las características generadas. Se trata de un módulo diseñado para ser extensible, permitiendo la incorporación de nuevos modelos mediante mecanismos desacoplados, como los adaptadores.
- **Módulo de notificaciones:** encargado de la generación y gestión de notificaciones asociadas al estado de los procesos y de su entrega a los usuarios. Este módulo se encarga de informar a los usuarios sobre el inicio, fallo o finalización de sus procesos.

5.3.2. Módulos del sistema bioinformático

Por otro lado, los módulos que componen el sistema bioinformático, mostrados en la figura [Figura 5.4](#), son los siguientes:

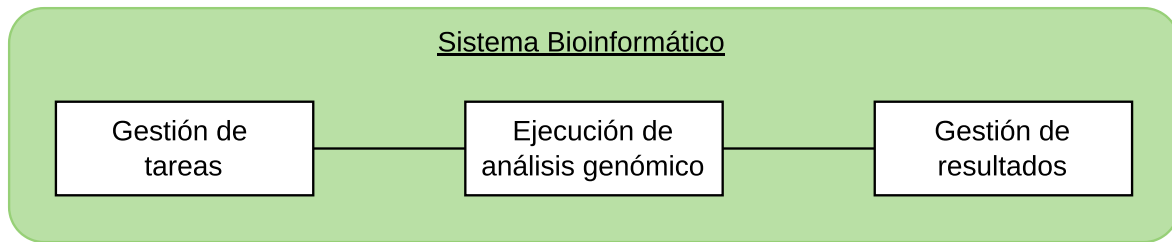


Figura 5.4: Diagrama de módulos del sistema bioinformático

- **Módulo de gestión de tareas bioinformáticas:** encargado de recibir y gestionar las tareas de análisis genómico enviadas desde el sistema web, coordinando su ejecución de forma asíncrona.
- **Módulo de ejecución del pipeline:** responsable de la ejecución de las herramientas bioinformáticas necesarias para el ensamblaje y la anotación, gestionando la ejecución de las distintas etapas del proceso.
- **Módulo de gestión de resultados:** encargado de procesar los resultados generados y facilitar su recuperación por parte del sistema web.

5.4. Modelo de datos y clases

⚠ **TODO:** Quiero cambiar una cosa del modelo de datos así que no quiero trabajar en esto todavía, me lo voy a saltar por ahora. Pero aquí se explicaría el modelo de datos, las clases principales, sus responsabilidades, etc. También se podrían incluir algunos diagramas de clases para ilustrar la estructura del sistema y la relación entre los distintos componentes.

- Diagrama de clases
- Diagrama de estado para los procesos!

5.5. Flujos principales del sistema

Aunque el sistema permite la ejecución completa del flujo de análisis genómico, desde la carga de datos hasta la predicción de resistencia a antibióticos, los diagramas de secuencia han sido separados para facilitar la comprensión de cada etapa del proceso. A continuación, se presentan los diagramas de secuencia para cada una de las etapas principales del flujo de análisis genómico.

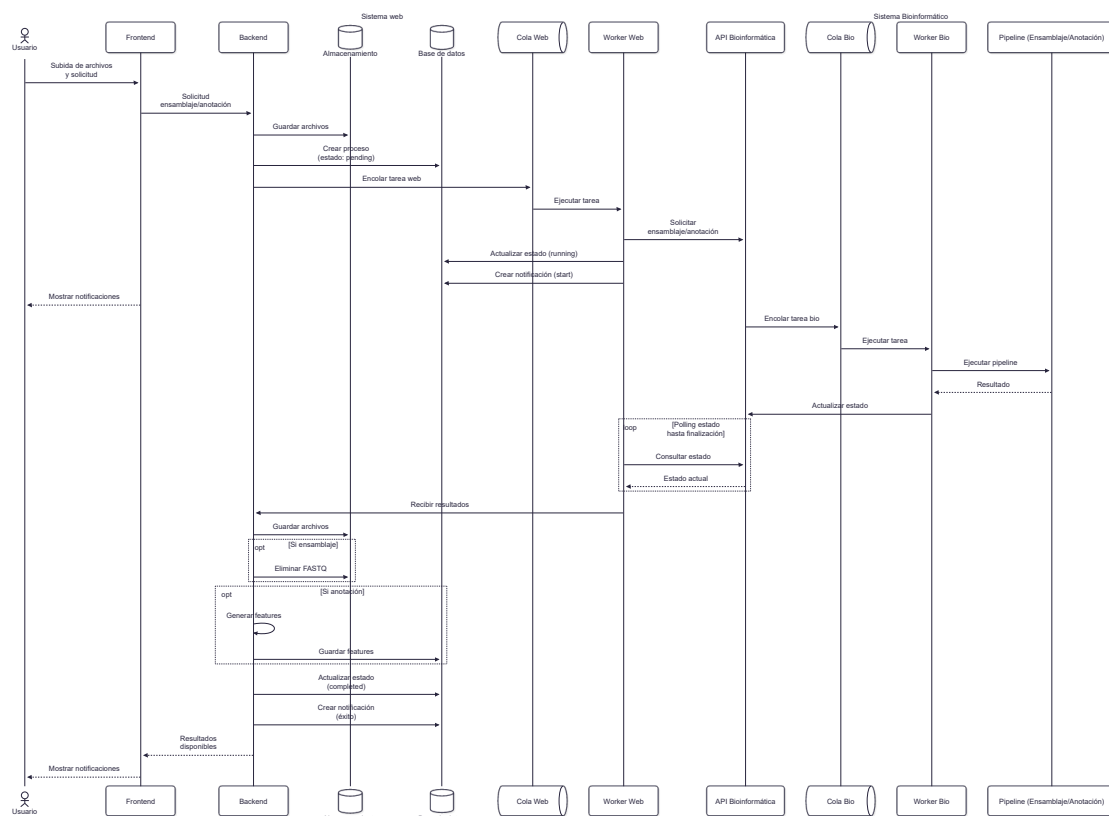


Figura 5.5: Diagrama de secuencia del flujo de ensamblaje o anotación

5.5.1. Flujo de ensamblaje y anotación

El diagrama de secuencia **Figura 5.5** describe el flujo de ejecución de los procesos de ensamblaje y anotación, desde la carga de los datos por parte del usuario hasta la disponibilidad de los resultados en el sistema.

El proceso se inicia cuando el **usuario** sube los archivos de entrada (datos FASTQ en el caso del ensamblaje o un archivo FASTA en el caso de la anotación) a través de la interfaz web y solicita la ejecución de un proceso de ensamblaje o anotación. El **frontend** envía esta información al **backend**, que se encarga de almacenar los archivos en el sistema de almacenamiento y registrar un nuevo proceso en la base de datos con estado inicial *pending*.

A continuación, el backend delega la ejecución del proceso a un **worker**, insertando la tarea en la **cola de procesamiento del sistema web**. Cuando se recoge la tarea, el worker se encarga de coordinar la ejecución del proceso, actualizando su estado a *running* y generando una notificación de inicio para el usuario, que se muestra en el frontend.

El worker realiza entonces una llamada a la **API del sistema bioinformático**, solicitando la ejecución del proceso de ensamblaje o anotación. Este sistema se encarga de gestionar la ejecución de forma independiente, encolando la tarea en su

propia cola de procesamiento, donde es recogida por un **worker** bioinformático que ejecuta el **pipeline** correspondiente. Dicho pipeline realiza las etapas necesarias (ensamblaje/anotación) y genera los resultados asociados.

Una vez iniciado el proceso en el sistema bioinformático, el worker del sistema web monitoriza su estado mediante un mecanismo de polling, consultando periódicamente a la API bioinformática por el estado del proceso hasta la finalización del mismo. Cuando el proceso finaliza, el sistema bioinformático actualiza su estado y el worker del sistema web recupera los resultados a través de la API. Estos resultados son almacenados en el sistema de archivos del sistema web para su posterior uso o descarga por parte del usuario.

Dependiendo del tipo de proceso ejecutado, se realizan algunas opciones adicionales. En el caso del ensamblaje, se eliminan los archivos FASTQ que proporcionó el usuario como entrada, ya que estos se tratan como datos temporales. En el caso de la anotación, el sistema genera automáticamente las características (features) a partir de los resultados obtenidos y estas son almacenadas en la base de datos para su uso en modelos de predicción.

Finalmente, el backend actualiza el estado del proceso a *completed* y genera una notificación de finalización, que es presentada al usuario a través de la interfaz web junto con la disponibilidad de los resultados.

Este flujo refleja la naturaleza distribuida y asíncrona del sistema, donde la orquestación se realiza desde el sistema web mientras que la ejecución de los procesos bioinformáticos se delega a un subsistema especializado. Esta separación permite gestionar eficientemente tareas de larga duración y facilita la escalabilidad del sistema.

5.5.2. Flujo de predicción de resistencia a antibióticos

El diagrama de la **Figura 5.6** muestra el **flujo de predicción de resistencia a antibióticos** dentro del sistema web, desde la interacción inicial del usuario hasta la obtención de resultados.

El flujo comienza cuando el **usuario** inicia un proceso de predicción a través de la interfaz web, el **frontend**. Para esto debe seleccionar la muestra sobre la que quiere hacer la predicción, así como los modelos y antibióticos de interés. Esta información se envía al **backend** mediante una solicitud de predicción, que incluye los datos seleccionados por el usuario.

Una vez recibida la petición, el backend consulta la **base de datos** para recuperar las características asociadas a la muestra seleccionada, que han sido previamente generadas a partir de los resultados de anotación. Estas características constituyen la información de entrada necesaria para alimentar los modelos de predicción.

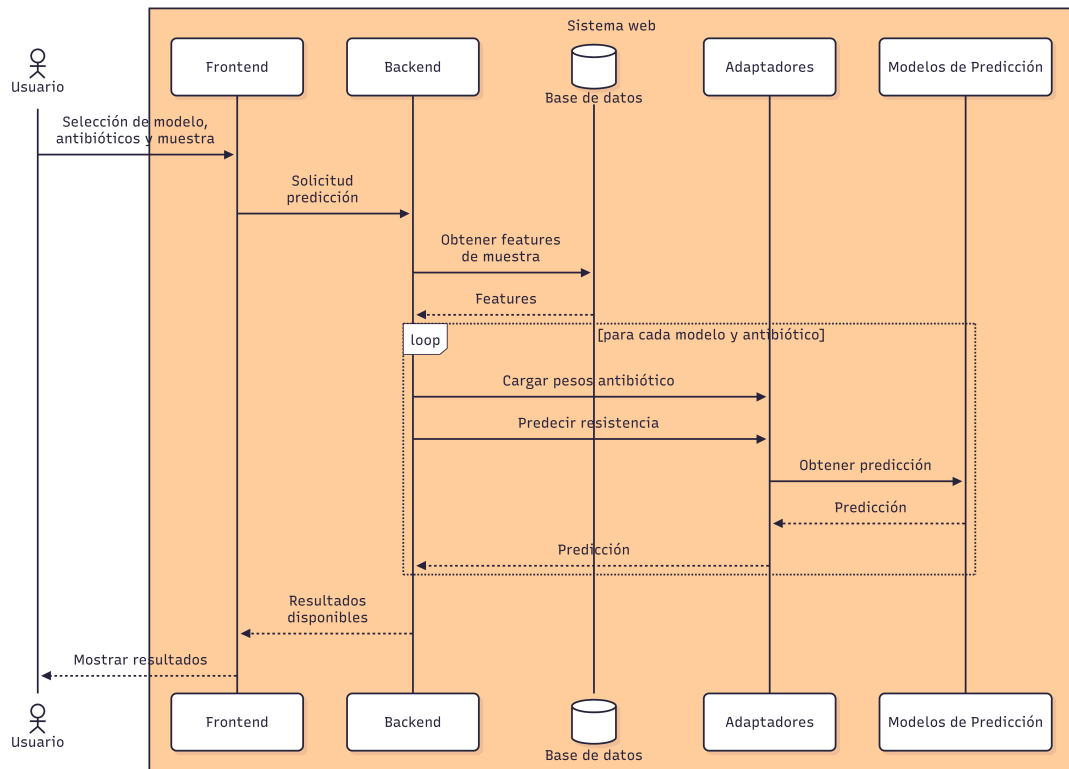


Figura 5.6: Diagrama de secuencia del flujo de predicción de resistencia

A continuación, el sistema itera sobre cada combinación de modelo y antibiótico seleccionados, ejecutando el modelo correspondiente con las características de la muestra. Para cada caso, el backend llama a los **adaptadores** necesarios, que actúan como capa de abstracción entre la lógica del sistema y los **modelos de predicción**. Estos se encargan de preparar la entrada en el formato requerido a partir de las características y gestionar la carga de parámetros del modelo, como los pesos asociados a cada antibiótico.

El adaptador invoca entonces al modelo de predicción correspondiente, que procesa las características de entrada y devuelve la predicción de resistencia. Esta respuesta se devuelve al adaptador que la envía de vuelta al backend, que se encarga de agregar los resultados de las ejecuciones y prepararlos para su visualización.

Finalmente, el backend envía la respuesta al frontend, que se encarga de mostrar los resultados de la predicción al usuario de forma clara e intuitiva, permitiéndole interpretar la información y tomar decisiones informadas sobre el tratamiento a seguir.

Este diseño introduce una clara separación de responsabilidades entre la orquestación del proceso, realizado por el backend, y la ejecución de los modelos de predicción, facilitando la extensibilidad del sistema. En particular, la incorporación de nuevos modelos o la adaptación a diferentes formatos de entrada se realiza

mediante la implementación de nuevos adaptadores, sin necesidad de modificar el flujo principal de la aplicación.

5.6. Decisiones de diseño

En esta sección se explican las decisiones de diseño tomadas durante el desarrollo del sistema, incluyendo los patrones de diseño aplicados y otras decisiones técnicas relevantes. Estas decisiones se han tomado con el objetivo de cumplir con los requisitos definidos para el sistema, especialmente en lo que respecta a la escalabilidad, la extensibilidad y la mantenibilidad.

5.6.1. Patrones de diseño

Para el diseño del sistema se han aplicado diversos patrones de diseño de software, algunos derivados de las tecnologías empleadas (framework y sistema de ejecución asíncrona), y otros aplicados de forma consciente para atender a los requisitos específicos definidos para el sistema, especialmente en lo que respecta a la extensibilidad.

Patrones derivados del framework y la infraestructura

Estos patrones se han aplicado de forma implícita en el sistema, ya que vienen dados por las herramientas utilizadas.

Modelo-Vista-Template (MVT) El sistema web se apoya en el patrón MVT proporcionado por *Django*, que organiza la aplicación entorno a modelos de datos, vistas que gestionan la lógica de negocio y templates para la presentación de la información al usuario. Este patrón facilita la separación de responsabilidades y mejora la mantenibilidad del código.

Command El sistema de ejecución asíncrona basado en *Celery* sigue el patrón Command, donde cada tarea se define como un comando que encapsula una acción (por ejemplo, lanzar un ensamblaje o consultar el estado de un proceso) y puede ser ejecutada por los workers de forma desacoplada del resto del sistema.

Singleton Componentes como la configuración de *Django* y la instancia de *Celery* actúan como singletons, ya que se inician una sola vez y se reutilizan en todo el sistema.

Patrones aplicados en el diseño del sistema

Adapter - Adaptadores para modelos predictivos Uno de los principales retos del sistema es la integración de modelos de predicción heterogéneos, con distintos formatos de entrada, configuraciones y salidas. Para abordar esto, se ha adoptado el patrón *Adapter*, creando una capa de adaptadores que actúan como intermediarios entre la lógica del sistema y los modelos de predicción. Cada adaptador se encarga de transformar las características generadas a partir de los resultados de anotación en el formato requerido por el modelo, así como de gestionar la carga de parámetros y la ejecución del modelo. Esto permite:

- Incorporar nuevos modelos sin modificar la lógica principal del sistema, mediante la implementación de nuevos adaptadores.
- Aislar dependencias externas, ya que los adaptadores pueden gestionar las particularidades de cada modelo sin afectar al resto del sistema.
- Facilitar la experimentación con distintos enfoques de predicción.

Registry - Registro dinámico de modelos y parsers El sistema implementa mecanismos de registro dinámico, basado en decoradores, que permite descubrir automáticamente modelos predictivos y parsers disponibles en el sistema. Este enfoque sigue un patrón de tipo *Registry*, donde nuevas clases de modelos o parsers pueden añadirse al sistema simplemente registrando nuevas clases, sin necesidad de modificar el código existente. Esto facilita:

- Cumplir los requisitos de extensibilidad.
- Reducir el acoplamiento.
- Facilitar la evolución del sistema.

Strategy - Selección dinámica de comportamiento De forma complementaria al patrón de registro, los distintos modelos no solo deben estar en el sistema sino que deben poder ser utilizados por el usuario a petición. Usando el patrón *Strategy*, el sistema puede seleccionar de forma dinámica qué modelo de predicción utilizar. Múltiples algoritmos, en este caso modelos de predicción, comparten una interfaz común y pueden intercambiarse en tiempo de ejecución.

Fachada - Encapsulación de complejidad La interacción con subsistemas complejos, como el sistema bioinformático o la gestión de procesos, se encapsulan servicios que actúan como fachadas, proporcionando una interfaz sencilla y unificada al resto del sistema. Esto:

- Oculta la complejidad de la interacción con otros sistemas.
- Simplifica el uso del sistema.
- Reduce el acoplamiento entre componentes.
- Centra la lógica de interacción con otros sistemas en las fachadas.

En conjunto, estos patrones permiten estructurar el sistema como una plataforma extensible y desacoplada, capaz de integrar nuevos modelos de predicción y herramientas bioinformáticas de forma sencilla, cumpliendo con los requisitos definidos y facilitando la evolución continua del sistema.

5.6.2. Decisiones técnicas

Por otro lado, en el diseño del sistema se han tomado diversas decisiones técnicas orientadas a satisfacer los requisitos no funcionales definidos, especialmente en términos de rendimiento, escalabilidad, mantenibilidad y extensibilidad. A continuación, se describen las más relevantes.

Ejecución asíncrona de tareas se ha optado por el uso de un sistema de workers asíncronos para la ejecución de tareas de larga duración, como los procesos de ensamblaje y anotación. Esta decisión permite evitar el bloqueo de la interfaz de usuario y mejorar la experiencia de uso, permitiendo que el usuario continúe interactuando con el sistema mientras se ejecutan dichas tareas. Además, facilita la ejecución concurrente de múltiples procesos, contribuyendo al cumplimiento de los requisitos de rendimiento y eficiencia.

Arquitectura distribuida basada en servicios el sistema se ha diseñado siguiendo una arquitectura distribuida basada en servicios, separando la aplicación web del sistema bioinformático. Esta separación permite desacoplar la evolución independiente de ambos subsistemas, facilitando la integración de nuevas herramientas bioinformáticas sin afectar a la lógica de negocio del sistema web. Además, posibilita escalar cada componente de forma independiente según sus necesidades específicas, mejorando la eficiencia en el uso de recursos.

Despliegue a través de contenedores se ha optado por el uso de contenedores para el despliegue del sistema, lo que facilita la portabilidad entre entornos y simplifica la gestión de dependencias. Dado que el sistema está compuesto por dos subsistemas independientes, el uso de contenedores permite aislar de forma eficiente las configuraciones y dependencias específicas de cada uno de ellos.

Diseño de la interfaz de usuario para la implementación de la interfaz de usuario se han seguido principios de diseño orientados a la simplicidad, la flexibilidad y la accesibilidad, buscando crear una experiencia de usuario intuitiva y eficiente, evitando complejidad innecesaria.

5.7. Diseño de la interfaz de usuario

5.7.1. Principios de diseño

El diseño de la interfaz se ha realizado en coherencia con los objetivos definidos en la [Sección 2.2](#), buscando crear una experiencia de usuario intuitiva y accesible, que permita a los usuarios interactuar con el sistema de forma eficiente y sin necesidad de conocimientos técnicos avanzados. Para ello, se han seguido los siguientes principios de diseño:

- **Simplificación del flujo de trabajo:** se abstrae la complejidad del pipeline de análisis genómico, permitiendo su ejecución mediante interacciones sencillas como la subida de un archivo o la selección de opciones en la interfaz.
- **Flexibilidad en la ejecución:** se permite ejecutar el flujo completo o por etapas, dando a los usuarios la posibilidad de gestionar sus procesos de forma personalizada según sus necesidades.
- **Minimización de la barrera técnica:** se evita exponer detalles internos del procesamiento, facilitando el uso del sistema por parte de usuarios sin experiencia técnica.
- **Feedback continuo:** se proporciona a los usuarios información clara sobre el estado de cada proceso, especialmente en las tareas de larga duración, mediante indicadores visuales y notificaciones.
- **Consistencia visual:** se mantiene un diseño uniforme y coherente a lo largo de toda la interfaz, mejorando la usabilidad y reduciendo la curva de aprendizaje.
- **Diseño limpio y orientado al dominio:** se prioriza la claridad visual y el orden en la presentación de la información, alineada con el contexto científico del sistema.

5.7.2. Estructura general de la interfaz

La interfaz web es ofrecida por completo por el sistema web, que actúa como punto de entrada a toda la funcionalidad del sistema. Su estructura se ha diseñado para facilitar la navegación entre las distintas funcionalidades, permitiendo tanto la ejecución completa del flujo de análisis genómico como su uso por etapas, así como el seguimiento de los procesos en ejecución.

La navegación principal se organiza en torno a un conjunto reducido de secciones que reflejan las distintas etapas del flujo de análisis genómico y las funcionalidades de gestión del sistema, tal y como se muestra en la [Figura 5.7](#).

Se distinguen dos grandes bloques: los módulos de análisis, que incluyen las funcionalidades de ensamblaje, anotación y predicción, y las secciones de gestión

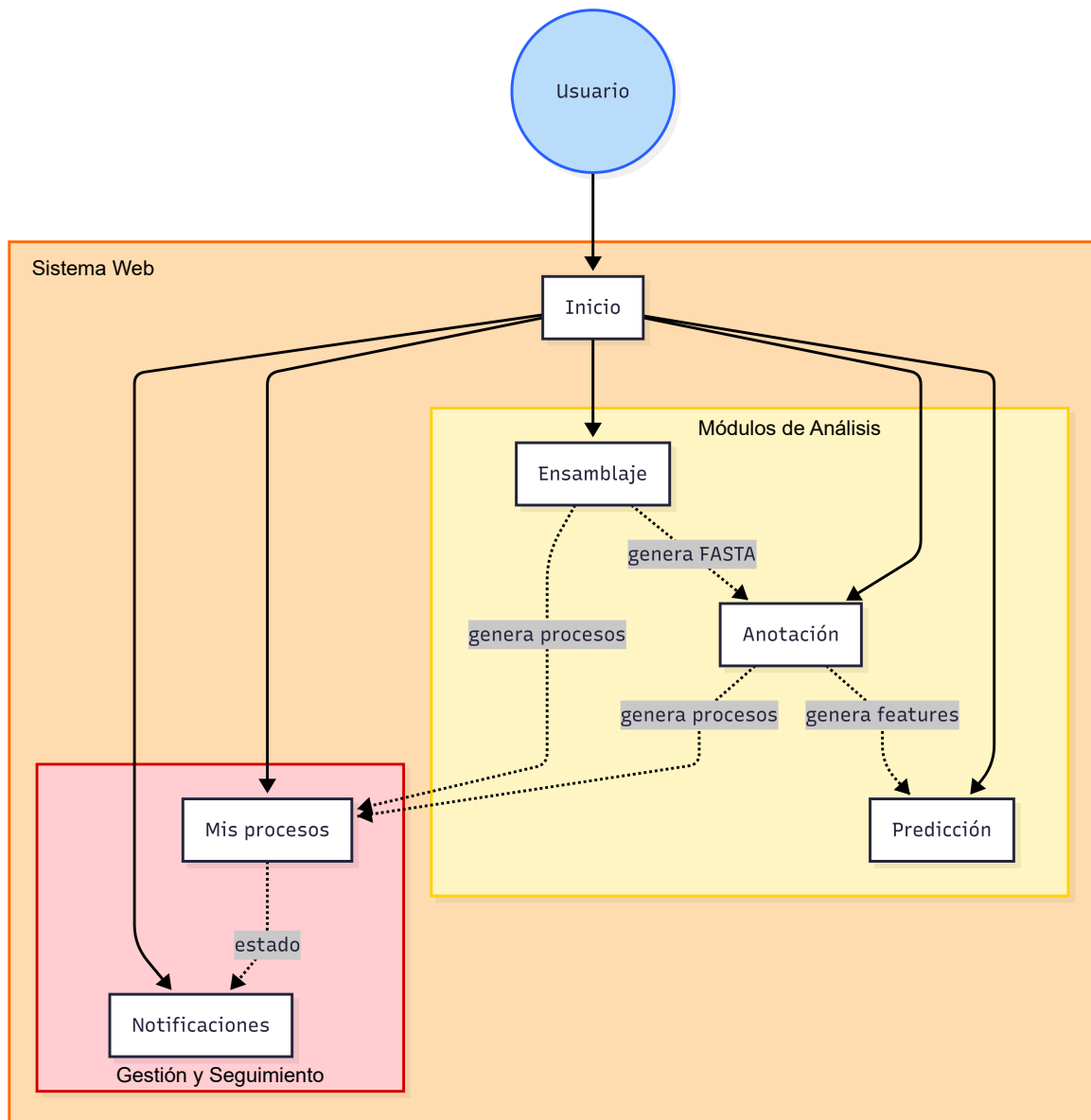


Figura 5.7: Diagrama de navegación de la interfaz

y seguimiento, centradas en la consulta de procesos y notificaciones. Asimismo, el diagrama refleja las relaciones entre estas secciones. Destaca el papel de la gestión de procesos como punto central del seguimiento de las ejecuciones. Adicionalmente, se observa la dependencia entre etapas del flujo, donde la anotación puede partir de resultados de ensamblaje y la predicción utiliza las características generadas durante la anotación.

A continuación, se describe brevemente cada una de las secciones principales de la interfaz:

- **Inicio:** sección de entrada al sistema que proporciona una visión general y acceso a las funcionalidades principales.

- **Ensamblaje:** permite la subida de datos FASTQ y la ejecución del proceso de ensamblaje, ofreciendo además la opción de encadenar automáticamente la anotación.
- **Anotación:** permite seleccionar datos FASTA y ejecutar procesos de anotación, así como generar características a partir de archivos de anotación externos.
- **Predicción:** sección dedicada a la ejecución de modelos de predicción de resistencia a antibióticos, donde los usuarios pueden seleccionar las muestras, modelos y antibióticos de interés para obtener predicciones.
- **Procesos:** sección central para la gestión de los procesos de análisis, incluyendo el seguimiento de su estado y la consulta de resultados.
- **Notificaciones:** permite a los usuarios consultar las notificaciones generadas por el sistema relacionadas con el estado de sus procesos y otras alertas relevantes.

5.7.3. Visualización e interpretación de resultados

La visualización de los resultados de predicción de forma sencilla e interpretable es uno de los requisitos fundamentales del sistema. Por este motivo, el diseño de la interfaz se ha orientado a representar de forma clara la información generada durante la ejecución de los modelos predictivos.

En los resultados de predicción (**Figura 5.8**) se muestran los antibióticos evaluados frente a los modelos seleccionados, indicando la predicción de resistencia generada para cada combinación, indicando con colores si la muestra presenta resistencia o sensibilidad frente a cada antibiótico. Esta visualización permite a los usuarios interpretar rápidamente los resultados y tomar decisiones informadas sobre el tratamiento a seguir.

Además de las predicciones individuales generadas por cada modelo, el sistema representa una predicción agregada, a través de la media de los resultados. Esta aproximación proporciona una visión global, facilitando la interpretación en escenarios en los que intervienen múltiples modelos.

La interfaz también incorpora mecanismos de representación del estado de los procesos, permitiendo al usuario identificar de forma sencilla si una tarea se encuentra en ejecución, finalizada correctamente o ha producido algún error. Para ello, se utilizan colores, marcadores de estado e indicadores de progreso. Esto resulta especialmente relevante debido al elevado tiempo de ejecución asociado a determinadas tareas bioinformáticas.

Resultados:

	Bakta_90	Bakta_50	Media
Amikacin	0,9 	0,85 	0,88 
Doripenem	0,65 	0,6 	0,63 
Oxacilin	0,45 	0,2 	0,33 

Figura 5.8: Mockup de la interfaz de visualización de predicciones.

5.8. Conclusiones

En conclusión, el diseño del sistema se ha orientado a cumplir con los requisitos definidos para el proyecto, especialmente en términos de extensibilidad, mantenibilidad y facilidad de uso. La arquitectura distribuida basada en servicios, junto con la aplicación de patrones de diseño como Adapter, Registry y Strategy, permite integrar nuevos modelos de predicción y herramientas bioinformáticas de forma sencilla, sin necesidad de modificar la lógica principal del sistema.

Implementación

6.1. Introducción

En este capítulo se aborda el proceso de implementación del sistema desarrollado en este trabajo, organizado de forma incremental mediante distintos sprints. A lo largo de estos sprints se abordaron tanto la definición de la arquitectura y la infraestructura base del sistema como la implementación progresiva de las funcionalidades relacionadas con el análisis genómico y la predicción de resistencia a antibióticos.

Para cada sprint se describen los objetivos planteados, las principales decisiones técnicas adoptadas, los problemas encontrados durante el desarrollo y los resultados obtenidos.

6.2. Tecnologías y herramientas utilizadas

 **TODO:** Maybe añadir un diagrama a modo resumen de las tecnologías?

En esta subsección se describen las principales tecnologías y herramientas utilizadas en el desarrollo del sistema, explicando las razones detrás de su elección. La selección

de las mismas se ha realizado teniendo en cuenta los requisitos definidos para el proyecto, especialmente aquellos relacionados con la mantenibilidad, extensibilidad, interoperabilidad y facilidad de despliegue.

6.2.1. Frontend

La interfaz web se ha desarrollado utilizando HTML, CSS y JavaScript, junto con Tailwind CSS y DaisyUI para el diseño visual de la aplicación.

Se ha optado por no utilizar frameworks frontend complejos como React o Angular, ya que la interfaz del sistema se centra principalmente en la gestión de formularios, la visualización de resultados y el seguimiento de procesos asíncronos, sin requerir una lógica de interacción especialmente compleja en el lado del cliente. El uso de este tipo de frameworks habría introducido una complejidad adicional tanto en el desarrollo como en el despliegue del sistema, sin aportar beneficios significativos para los objetivos del proyecto. En su lugar se ha priorizado una arquitectura frontend ligera y sencilla de mantener que permite iterar rápidamente sobre el diseño de la interfaz y adaptarla a las necesidades de los usuarios, sin la sobrecarga que implicaría un framework más pesado.

Para el diseño visual se ha utilizado *DaisyUI*, basada en *Tailwind CSS*. Tailwind permite construir interfaces de forma flexible mediante clases, mientras que DaisyUI proporciona componentes reutilizables que permiten ofrecer una apariencia consistente. Esta decisión permite acelerar el desarrollo de la interfaz, evitando complejidad innecesaria garantizando al mismo tiempo una apariencia consistente y profesional.

6.2.2. Backend web

El backend del sistema web se ha desarrollado utilizando *Django* como framework principal y *PostgreSQL* como sistema de gestión de bases de datos.

Debido a la integración con modelos de predicción, se ha decidido utilizar *Python* como lenguaje principal del proyecto, ya que se trata del lenguaje predominante tanto en el ámbito de la ciencia de datos y el aprendizaje automático. Esto facilita la integración con modelos predictivos y herramientas externas en el mismo ecosistema tecnológico.

Dentro de los frameworks de desarrollo web disponibles para Python, se ha optado por *Django* debido a su amplia gama de funcionalidades integradas, como su sistema de gestión de usuarios y autenticación, el ORM para la persistencia de datos y el panel de administración. Estas características permiten acelerar el desarrollo de funcionalidades comunes y permiten centrarse en la lógica específica del proyecto. Además, gran parte del sistema consiste en la gestión de entidades

persistentes relacionadas entre sí como procesos, archivos o notificaciones, lo que se adapta bien al modelo relacional y el ORM de Django.

Como sistema de persistencia se ha utilizado *PostgreSQL* debido a su fiabilidad, rendimiento y compatibilidad con Django.

6.2.3. Sistema bioinformático

El subsistema bioinformático se ha desarrollado también principalmente en Python, debido a la amplia disponibilidad de herramientas y bibliotecas bioinformáticas, haciendo que sea el lenguaje más adecuado para la integración de las herramientas de ensamblaje y anotación, así como para la gestión de pipelines de procesamiento.

Para la implementación de la API del sistema bioinformático se ha utilizado *FastAPI*. A diferencia del sistema web principal, este subsistema no requiere funcionalidades avanzadas de gestión de usuarios o renderizado de vistas, sino una interfaz ligera y eficiente orientada a la comunicación entre servicios. FastAPI permite implementar esta API de forma sencilla, con buena integración con Python.

El sistema bioinformático integra herramientas especializadas para distintas etapas del pipeline genómico. Por un lado se hace uso de herramientas de ensamblaje como *SPAdes*, *Raven* or *Flye*, permitiendo adaptar el procesamiento a distintos tipos de datos de secuenciación, incluyendo tecnologías Illumina y ONT.

Por otro lado, para la anotación del genoma se utiliza *Bakta*, una herramienta orientada a la anotación de secuencias de ADN especialmente diseñada para muestras de bacterias. Bakta ofrece una anotación rápida y estandarizada, lo que facilita la generación de resultados consistentes y de alta calidad, además de ser compatible con el formato de salida JSON, lo que permite su integración directa con el sistema web para la generación de características y la ejecución de modelos predictivos.

6.2.4. Gestión de tareas asíncronas

Para la gestión de las tareas asíncronas de ensamblaje y anotación se ha utilizado *Celery* junto con *Redis* como sistema de cola de mensajes.

Celery permite definir tareas desacopladas que son ejecutadas por workers independientes del servidor web principal. Esto permite delegar la ejecución de procesos de larga duración sin bloquear la interacción del usuario con la aplicación y facilita la ejecución concurrente de múltiples tareas.

Por otro lado, Redis se emplea como intermediario para la gestión de colas gracias a su sencilla interacción con Celery y el hecho de que es un sistema de almacenamiento en memoria que ofrece un alto rendimiento en operaciones de lectura y escritura.

Además, este enfoque simplifica la coordinación entre el sistema web y el sistema bioinformático, permitiendo gestionar el envío de tareas, la monitorización de su estado y la recuperación de resultados de forma desacoplada.

6.2.5. Despliegue y contenedorización

Para el despliegue del sistema se ha optado por una arquitectura basada en contenedores utilizando *Docker*. Se ha optado por esta tecnología ya que se trata del sistema más extendido y conocido de contenedorización. Además, cuenta con muy buen soporte y documentación online, lo que facilita su uso tanto durante el desarrollo como en la fase de despliegue y producción.

Para la orquestación de los contenedores se ha decidido usar *Docker Compose*, lo que permite definir y gestionar la infraestructura del sistema de forma sencilla a través de archivos de configuración. Esto simplifica tanto el despliegue como la puesta en producción del sistema.

6.3. Sprint 1 - Estudio previo y definición del sistema

6.3.1. Objetivos

Este sprint se desarrolló entre el 25 de noviembre y el 2 de diciembre de 2025, con el objetivo principal de establecer las bases conceptuales y técnicas del sistema. Durante esta fase, se realizó un estudio preliminar del dominio bioinformático para comprender el contexto biológico relacionado con la resistencia a antibióticos, así como las técnicas de análisis genómico y predicción basadas en técnicas de aprendizaje automático. Además, se definieron las funcionalidades principales de la aplicación y se elaboró un plan de trabajo detallado para guiar el desarrollo del proyecto en los siguientes sprints.

Los objetivos principales de este sprint fueron:

- Analizar el flujo de procesamiento necesario para el análisis genómico y la predicción de resistencia a antibióticos.
- Identificar herramientas bioinformáticas relevantes para el proyecto.
- Estudiar modelos de machine learning aplicados a la predicción de resistencia a antibióticos.
- Definir el alcance funcional del sistema.
- Establecer una planificación inicial del desarrollo.

6.3.2. Detalles de implementación

Durante este sprint se realizó un análisis técnico y conceptual del dominio bioinformático, incluyendo los distintos pasos necesarios para construir un pipeline completo y útil de análisis genómico. Se estudiaron las etapas de secuenciación,

ensamblaje, anotación y transformación de datos genómicos en características utilizables por modelos de predicción. Además, se realizó una investigación de las técnicas de machine learning más relevantes para la predicción de resistencia a antibióticos.

Se realizaron pruebas preliminares con distintas herramientas bioinformáticas para evaluar sus requisitos de entrada, formatos de salida y complejidad de integración dentro del flujo de procesamiento planteado. A partir de este análisis se definió el flujo principal soportado por el sistema, compuesto por las siguientes etapas:

1. **Ensamblaje** de datos genómicos a partir de secuencias crudas utilizando herramientas como *SPAdes*, *Flye* o *Raven*.
2. **Anotación** de datos genómicos utilizando *Bakta*.
3. **Generación de features** a partir de los datos anotados.
4. **Predicción** de resistencia a antibióticos utilizando modelos de machine learning.

También se identificó la necesidad de desacoplar la ejecución de tareas bioinformáticas del sistema web principal debido al elevado tiempo de ejecución y consumo de recursos de las herramientas de ensamblaje y anotación. Se decidió diseñar en el sprint siguiente un sistema que aislaría las tareas largas y computacionalmente intensivas del backend principal, evitando que las tareas de larga duración afectasen a la capacidad de respuesta del sistema web.

Durante esta fase se definió además una primera aproximación de los requisitos funcionales y no funcionales del sistema, así como una planificación inicial del desarrollo organizada en distintos sprints.

6.3.3. Resultados

Durante este sprint se sentaron las bases para el desarrollo del sistema, se adquirió un entendimiento clave del contexto biológico y se definieron los objetivos y el alcance del proyecto. Esto permitió establecer una dirección clara para el desarrollo del sistema en los siguientes sprints.

6.4. Sprint 2 - Diseño de arquitectura y definición del framework

6.4.1. Objetivos

Este sprint se desarrolló entre el 2 de diciembre y el 16 de diciembre de 2025 y tuvo como objetivo diseñar la arquitectura general del sistema y seleccionar las tecnologías principales utilizadas durante el desarrollo.

Los objetivos principales fueron:

- Definir la arquitectura general del sistema.
- Diseñar el modelo inicial de datos.
- Seleccionar frameworks y tecnologías base.
- Definir la comunicación entre subsistemas.
- Definir patrones de diseño a utilizar.

6.4.2. Detalles de implementación

Durante este sprint se diseñó la arquitectura del sistema, teniendo en cuenta las necesidades y requisitos definidos en el sprint anterior. Debido a la necesidad de aislar las tareas largas y computacionalmente intensivas del backend principal, se optó por una arquitectura distribuida que separa el sistema en dos subsistemas principales:

- Un sistema web encargado de la gestión de usuarios, procesos, almacenamiento y predicciones.
- Un sistema bioinformático encargado de ejecutar las tareas de ensamblaje y anotación.

También se definió un mecanismo de comunicación basado en APIs HTTP y ejecución asíncrona mediante colas de tareas y workers desacoplados, permitiendo separar la gestión de procesos de su ejecución real. Se tomó la decisión de implementar instancias de la cola de tareas y workers específicos para cada subsistema, para así mantener una separación clara entre las tareas relacionadas con el sistema web y las tareas relacionadas con el sistema bioinformático y además poder escalar, desplegar y monitorear cada subsistema de forma independiente según las necesidades del proyecto.

En esta fase se seleccionaron las tecnologías principales utilizadas durante el desarrollo:

- *Django* para el sistema web.
- *FastAPI* para el sistema bioinformático.
- *PostgreSQL* como sistema de persistencia.
- *Celery* y *Redis* para la gestión de tareas asíncronas.
- *Docker* para el despliegue y aislamiento de dependencias.

Adicionalmente, se diseñó el modelo inicial de datos, definiendo entidades relacionadas con usuarios, procesos, archivos y notificaciones, así como las relaciones entre ellas. Finalmente, se evaluaron distintos enfoques de diseño, investigando patrones comunes en sistemas similares, con el objetivo de facilitar la extensibilidad del sistema. Se puso especial énfasis en lo relativo a la integración de nuevos modelos predictivos y mecanismos de generación de características.

6.4.3. Resultados

Este sprint fue fundamental para establecer una arquitectura sólida y escalable que permitiera abordar las necesidades específicas del proyecto, especialmente en relación con la ejecución de tareas bioinformáticas. La separación en subsistemas permitió diseñar un sistema más modular y mantenible, facilitando el desarrollo progresivo de funcionalidades en los siguientes sprints.

6.5. Sprint 3 - Desarrollo de la infraestructura base

6.5.1. Objetivos

Este sprint se desarrolló entre el 16 de diciembre de 2025 y el 13 de enero de 2026, con el objetivo principal de implementar la infraestructura base del sistema, incluyendo la configuración de la base de datos, la implementación de mecanismos de autenticación y autorización, y la creación de contenedores Docker para facilitar el desarrollo y despliegue del sistema. Los objetivos específicos de este sprint fueron:

- Establecer las bases técnicas para el desarrollo del sistema.
- Configurar la base de datos.
- Implementar mecanismos de autenticación y autorización.
- Configurar contenedores Docker para el desarrollo y despliegue.

6.5.2. Detalles de implementación

La principal decisión técnica que ha condicionado el desarrollo de este sprint es la separación entre el sistema web y el sistema bioinformático, definida previamente durante el diseño del sistema. Esta separación condicionó tanto la organización del código como el despliegue de servicios y el mecanismo de ejecución de tareas asíncronas.

Organización del proyecto Django

Se inicializó el sistema web utilizando *Django*, organizando la funcionalidad en distintas aplicaciones siguiendo una separación basada en responsabilidades. La división en apps real no coincide exactamente con la descomposición conceptual presentada en la [Sección 5.3.1](#), ya que se ha adaptado a la estructura de *Django* buscando un equilibrio entre la separación de responsabilidades y la coherencia con el framework utilizado. Las aplicaciones principales definidas fueron:

- **home**: gestión de autenticación, navegación general y página principal.
- **conversion**: gestión de procesos bioinformáticos, comunicación con el sistema bioinformático, almacenamiento de archivos y generación de features.
- **prediction**: ejecución de modelos de predicción y gestión de resultados.

- **notifications:** gestión de notificaciones asociadas al estado de los procesos.

Esta organización permite mantener una separación clara entre las distintas áreas del sistema, encapsulando la lógica específica de cada dominio dentro de su propia aplicación y reduciendo el acoplamiento entre componentes.

Configuración de persistencia y almacenamiento

Aunque inicialmente se consideró el uso de *SQLite* como sistema de persistencia en las primeras fases del desarrollo por su simplicidad, finalmente se optó por hacer uso de *PostgreSQL* desde el inicio debido a que el acceso a la base de datos de distintos workers podía dar problemas de concurrencia. Así mismo, se optó por esta opción para evitar problemas de migración y compatibilidad que podrían surgir al cambiar de *SQLite* a *PostgreSQL* en fases posteriores.

Se definió el modelo de datos inicial utilizando los modelos de *Django*, incluyendo entidades relacionadas con usuarios, procesos, archivos y notificaciones y sus relaciones, especificadas en [Sección 5.4](#).

 **TODO:** Añadir cosas interesantes del modelo de datos tras el cambio

Por otro lado, debido al gran tamaño de los archivos relacionados con el pipeline genómico (FASTQ, FASTA y JSON) y a las diferencias en su ciclo de vida, se decidió diferenciar entre almacenamiento temporal y persistente, diferenciando:

- **Archivos Temporales:** Los archivos FASTQ utilizados como entrada durante los procesos de ensamblaje se almacenan temporalmente y se eliminan automáticamente tras finalizar el procesamiento para reducir el consumo de espacio. Estos se encuentran almacenados en su propia carpeta temporal `uploads/temp/` durante el procesamiento.
- **Archivos Persistentes:** Archivos FASTA o JSON generados en los procesos de ensamblaje o anotación. Se almacenan en `uploads/persistent/user_{user_id}/{file_type}/`, según el usuario y el tipo de archivo, facilitando búsquedas y gestión.

 **TODO:** Meter una captura ejemplo de la estructura de carpetas de almacenamiento de archivos, diferenciando entre temporales y persistentes

Esta separación permitió optimizar el uso de almacenamiento y simplificar la gestión de resultados a largo plazo.

Automatización del entorno de desarrollo

Con el objetivo de simplificar la puesta en marcha del sistema web en desarrollo y reducir la complejidad de la configuración inicial, se desarrolló un script de automatización encargado de instalar dependencias, construir recursos frontend y preparar el entorno de ejecución. Este script, `setup.py`, se encuentra en la raíz del subsistema web y se puede ejecutar con un único comando para configurar el entorno de desarrollo de forma rápida y sencilla.

Este proceso:

- Instala dependencias de Python utilizando `pip` y el archivo `requirements.txt` ([Código 6.1-10](#)).
- Instala dependencias de frontend utilizando `npm` y el archivo `package.json` ([Código 6.1-16](#)).
- Genera los recursos CSS necesarios ([Código 6.1-22](#)).

Esto permite realizar la configuración inicial del proyecto de forma rápida con un único comando.

Configuración de Docker y despliegue

Por otro lado, para facilitar el despliegue del sistema al completo se utilizó *Docker* para crear contenedores independientes para cada servicio, permitiendo aislar dependencias y facilitar el despliegue en diferentes entornos. Se definieron entornos independientes para el sistema web y el sistema bioinformático, cada uno con sus propios servicios y dependencias.

- **Sistema Web** (`tfg/docker/docker-compose.yml`):
 - *PostgreSQL*: Base de datos del sistema.
 - *Redis*: Broker de mensajes para Celery.
 - *Web*: Servidor Django.
 - *Celery Worker*: Procesa tareas de conversión y predicción de forma asíncrona.
- **Sistema Bioinformático** (`docker-servicios-bio/docker-compose.yml`):
Orquesta los servicios del subsistema bioinformático:
 - *PostgreSQL*: Base de datos de metadatos del servicio.
 - *Redis*: Broker para colas de tareas.
 - *FastAPI*: Servidor API del servicio bioinformático.
 - *Celery Worker Bioinformático*: Procesan tareas de assembly y anotación.
 - *Celery Worker de mantenimiento*: Procesa tareas de limpieza y mantenimiento del servicio.

```

1  def main():
2      """Main setup routine."""
3      project_root = Path(__file__).parent
4      app_dir = project_root / "app"
5
6      print("Cocos Setup")
7      print("=" * 50)
8
9      # Install Python dependencies
10     print("\nInstalling Python dependencies...")
11     if run_command([sys.executable, "-m", "pip", "install", "-r", "requirements.txt"],
12                    cwd=project_root):
13         print("Failed to install Python dependencies")
14         return 1
15
16     # Install frontend dependencies
17     print("\nInstalling frontend dependencies (Tailwind + daisyUI)...")
18     if run_command(["npm", "install"], cwd=app_dir):
19         print("Failed to install npm dependencies")
20         return 1
21
22     # Build CSS
23     print("\nBuilding frontend CSS bundle...")
24     if run_command(["npm", "run", "build:css"], cwd=app_dir):
25         print("Failed to build CSS")
26         return 1
27
28     print("\n" + "=" * 50)
29     print("Setup completed successfully!")
30
31     return 0

```

Código 6.1: Segmento del script de setup.py utilizado para configurar el proyecto

- *Celery Beat*: Programador de tareas periódicas.
- Opcionalmente *Flower* para monitorización de tareas Celery.

Para la comunicación entre los contenedores, se definieron redes Docker personalizadas, permitiendo que los servicios se comuniquen entre sí utilizando nombres de host definidos en el `docker-compose.yml`.

Por otro lado, se definieron volúmenes Docker para persistir datos importantes, como los archivos de resultados y los datos de la base de datos, asegurando que estos datos no se pierdan al detener o reiniciar los contenedores.

Finalmente, para realizar de forma sencilla el despliegue de ambos sistemas en un entorno de producción, se desarrolló un script de despliegue (`deploy.sh`) que automatiza la construcción de las imágenes Docker y el despliegue de los contenedores.

⚠ TODO: Mejorar párrafo final y meter code snippet del script de despliegue cuando esté en Python y no en PowerShell

Ejecución asíncrona

Para coordinar la ejecución de tareas de larga duración se configuró *Celery* junto con *Redis*. Los workers definidos en cada subsistema permiten ejecutar procesos de forma asíncrona y desacoplada del flujo principal de la aplicación.

En el sistema web los workers monitorizarán el envío de tareas al sistema bioinformático, la monitorización de su estado, la recuperación de resultados y la ejecución de predicciones.

Por otro lado, en el sistema bioinformático, los workers se encargan de ejecutar las herramientas de ensamblaje y anotación, además de tareas auxiliares de mantenimiento y limpieza.

La separación entre los workers del sistema web y del sistema bioinformático permite escalar ambos subsistemas de forma independiente según la carga de cada uno y permite mantener los dos sistemas totalmente desacoplados.

Implementación de autenticación y control de acceso

Aunque la gestión de usuarios no es un punto clave del sistema, se consideró necesario para asociar procesos y resultados a usuarios concretos y permitir un seguimiento individualizado de su progreso. Para ello se decidió utilizar el sistema de autenticación integrado de *Django*, aprovechando sus mecanismos de gestión de usuarios, sesiones y permisos.

Se implementaron funcionalidades de:

- **Registro:** Creación de nuevas cuentas de usuario.
- **Login:** Autenticación de usuarios con email/usuario y contraseña.
- **Logout:** Cierre de sesión.

Esto permitió restringir el acceso a procesos y resultados únicamente a sus propietarios.

6.5.3. Resultados

En este sprint se estableció la infraestructura técnica fundamental del sistema, permitiendo el desarrollo de funcionalidades posteriores. La separación en subsistemas independientes mediante Docker y la comunicación HTTP REST proporciona escalabilidad y mantenibilidad. La utilización de Celery con Redis permite procesar tareas largas sin bloquear el sistema web, mientras que la arquitectura modular de Django facilita la extensión futura del sistema con nuevas aplicaciones y funcionalidades.

6.6. Sprint 4 - Servicio de conversión de datos genómicos

6.6.1. Objetivos

Este sprint se desarrolló entre el 13 y el 27 de enero de 2026 y tuvo como objetivo implementar el subsistema bioinformático encargado de ejecutar los procesos de ensamblaje y anotación, así como la integración inicial entre dicho subsistema y el sistema web. Los objetivos principales son:

- Implementar el sistema bioinformático desacoplado del sistema web.
- Integrar herramientas externas de ensamblaje y anotación.
- Diseñar la API de comunicación entre subsistemas.
- Implementar la ejecución asíncrona de los procesos bioinformáticos.
- Gestionar el almacenamiento y la recuperación de resultados.

6.6.2. Detalles de implementación

Implementación del subsistema bioinformático

El sistema bioinformático se desarrolló como un servicio independiente usando *FastAPI* y *Celery*. Este subsistema actúa como responsable de la ejecución de herramientas bioinformáticas externas, desacoplando las tareas de análisis genómico del backend principal del sistema web.

La separación de ambos subsistemas permitió aislar las dependencias específicas de las herramientas bioinformáticas, los elevados requisitos computacionales y los largos tiempos de ejecución asociados a los procesos de ensamblaje y anotación.

Además, este enfoque facilita la escalabilidad del sistema y recude el impacto de posibles errores producidos durante la ejecución de herramientas externas.

Internamente, el sistema bioinformático se estructura en tres componentes principales:

- Una API HTTP encargada de recibir solicitudes de ejecución desde el sistema web (`app.py`).
- Un sistema de workers asíncronos responsable de ejecutar los procesos bioinformáticos, integrados con las herramientas externas (`task.py`).
- Un módulo de utilidades para la gestión de los procesos (`utils.py`).

Las herramientas bioinformáticas se integraron mediante la ejecución de procesos externos desde Python utilizando el módulo `subprocess`. Este enfoque permite encapsular la lógica de invocación de cada herramienta y controlar parámetros de ejecución, directorios de salida y gestión de errores de forma homogénea.

6.6.2.1.1. Integración de herramientas de ensamblaje

El sistema se diseñó para soportar distintos tipos de tecnologías de secuenciación, integrando diferentes herramientas de ensamblaje según el tipo de lecturas utilizadas. Para secuenciación de lecturas cortas (*Illumina*) se integró *SPAdes*, mientras que para secuenciación de lecturas largas (*ONT*) se integraron *Flye* y *Raven*. En todos los casos, la integración sigue una estructura similar:

1. Asegurar que la herramienta se encuentra instalada.
2. Validar los parámetros de entrada.
3. Generar el comando de ejecución con los parámetros adecuados.
4. Ejecutar el comando utilizando `subprocess` y monitorizar su progreso.
5. Almacenar los resultados generados y los logs de ejecución.

Un ejemplo simplificado de la integración de *SPAdes* se muestra en [Código 6.2](#).

De forma similar, se implementó la integración de las herramientas *Flye* y *Raven*, adaptando los parámetros de ejecución a las características específicas de cada ensamblador. Un ejemplo simplificado de la integración de *Flye* se muestra en [Código 6.3](#), mientras que un ejemplo de la integración de *Raven* se muestra en [Código 6.4](#).

6.6.2.1.2. Integración de herramientas de anotación

Respecto a la etapa de anotación, se integró la herramienta *Bakta*, especializada en anotación automática de genomas bacterianos. Esta herramienta requiere una base de datos externa de gran tamaño, necesaria para realizar búsquedas y anotaciones genómicas. Para evitar incluir dicha base de datos dentro de la imagen *Docker* del

```

1  def perform_illumina_assembly(r1_path: str, r2_path:
    str, output_dir: Path,
2                                     threads: int, memory: int,
    assembler: str = None, job_id:
    str = None):
3
4      spades = assembler if assembler else which_any(["spades.py",
    "spades"])
5      if not spades:
6          raise RuntimeError("SPAdes not found in PATH.")
7
8      output_dir.mkdir(parents=True, exist_ok=True)
9      cmd = [spades, "--threads", str(threads), "--memory",
    str(memory), "-o", str(output_dir)]
10
11     if r2_path:
12         cmd += ["-1", r1_path, "-2", r2_path]
13     elif r1_path:
14         cmd += ["-s", r1_path]
15     else:
16         raise ValueError("For Illumina specify --r1 (and
    optionally --r2).")
17
18     run_cmd(cmd, job_id=job_id)
19
20     contigs = output_dir / "contigs.fasta"
21     if not contigs.exists():
22         raise RuntimeError("SPAdes did not generate
    contigs.fasta")
23
24     final = output_dir / "contigs.filtered.fasta"
25     append_job_log(job_id, f"Illumina assembly completed:
    {final}")
26     return final

```

Código 6.2: Segmento del código de integración de la herramienta de ensamblaje SPAdes dentro del sistema bioinformático

servicio, se decidió desacoplarla mediante volúmenes montados durante el despliegue. Esta decisión reduce significativamente el tamaño de las imágenes, evita descargas


```

1     if mode == "flye":
2         cmd = ["flye", "--nano-raw", reads_path, "--out-dir",
3               str(output_dir), "--threads", str(threads)]
4         run_cmd(cmd, job_id=job_id)
5         contigs = output_dir / "assembly.fasta"

```

Python

Código 6.3: Segmento del código de integración de la herramienta de ensamblaje Flye dentro del sistema bioinformático

```

1     contigs = output_dir / "contigs.fasta"
2     cmd = ["raven", "-t", str(threads), reads_path]
3     try:
4         with open(contigs, "wb") as w:
5             result = subprocess.run(cmd, check=True,
6                                     stdout=w, stderr=subprocess.PIPE, text=True)
7             if result.stderr:
8                 append_job_log(job_id, f"[RAVEN STDERR]
9                                     {result.stderr[:500]}")
10            except subprocess.CalledProcessError as e:
11                error_msg = f"Raven failed with code
12                           {e.returncode}\nSTDERR: {e.stderr or 'No stderr'}"
13                raise RuntimeError(error_msg) from e

```

Python

Código 6.4: Segmento del código de integración de la herramienta de ensamblaje Raven dentro del sistema bioinformático

repetidas y facilita tanto la actualización de la base de datos como la utilización de variantes ligeras como *bakta-light*.

Un ejemplo simplificado del comando utilizado para la ejecución de Bakta se muestra en [Código 6.5](#).

6.6.2.1.3. Gestión de archivos y directorios

Finalmente, se implementó un sistema de gestión de directorios temporales y persistentes para aislar la ejecución de cada proceso bioinformático. Cada tarea dispone de un directorio de trabajo independiente donde se almacenan archivos intermedios, logs y resultados generados durante la ejecución. Esto facilita tanto la recuperación de errores como la limpieza automática de datos temporales una vez finalizado el procesamiento.

```

1     bakta = which_any(["bakta"])
2     if not bakta:
3         raise RuntimeError("Bakta not found in PATH.")
4
5     output_dir.mkdir(parents=True, exist_ok=True)
6     cmd = [
7         bakta,
8         "--db", "/data/bakta_db",
9         "--threads", str(threads),
10        "--output", str(output_dir),
11        "--force",
12        contigs_path
13    ]
14
15    run_cmd(cmd, job_id=job_id)
16
17    base = Path(contigs_path).stem
18    preferred_gff3 = output_dir / f"{base}.gff3"
19    if preferred_gff3.exists():
20        gff_file = preferred_gff3
21    else:
22        gff_candidates = list(output_dir.glob("*.gff3"))
23        if not gff_candidates:
24            files = [p.name for p in output_dir.glob("*")]
25            raise RuntimeError(f"Bakta did not generate a .gff3 file. Files: {files}")
26        gff_file = gff_candidates[0]
27
28    return gff_file

```

Código 6.5: Segmento del código de integración de la herramienta de anotado Bakta dentro del sistema bioinformático

Diseño de la API de comunicación entre subsistemas

La comunicación entre el sistema web y el sistema bioinformático se implementó mediante una API REST sobre HTTP. Esta API permite desacoplar completamente ambos subsistemas y facilita la evolución independiente de cada uno. El sistema bioinformático expone los endpoints mostrados en la [Tabla 6.1](#), que permiten iniciar

procesos de ensamblaje y anotación, consultar su estado y recuperar resultados. Cada endpoint se diseñó siguiendo principios RESTful, utilizando métodos HTTP adecuados para cada operación y estructurando las URLs de forma clara y consistente.

Cada proceso de ensamblaje o anotación se identifica mediante un UUID único que permite sincronizar el estado entre ambos subsistemas. Un ejemplo simplificado de endpoint utilizado para iniciar procesos de ensamblaje se muestra en [Código 6.6](#).

Integración con el sistema web

La integración del sistema web con el sistema bioinformático se implementó mediante una capa de servicios encargada de encapsular las llamadas HTTP. La comunicación con el sistema bioinformático se encapsuló en una capa de servicios implementada en el archivo `bio_api_client.py`, encargada de abstraer las llamadas HTTP y unificar la interacción con la API externa, siguiendo un patrón fachada. Se puede ver un ejemplo en [Código 6.7](#).

Las tareas asíncronas del sistema web, definidas en `tasks.py`, utilizan este cliente para iniciar procesos de ensamblaje o anotación en el subsistema bioinformático, consultar el estado de los procesos y recuperar resultados.

La ejecución de procesos se realiza mediante tareas *Celery*, permitiendo que el usuario continúe interactuando con la aplicación mientras los procesos bioinformáticos se ejecutan en segundo plano.

Monitorización y flujo de procesos

Uno de los principales retos de la integración es la sincronización del estado de procesos de larga duración entre ambos subsistemas. Debido a que las tareas de ensamblaje y anotación pueden prolongarse varios minutos o incluso horas, no es adecuado mantener conexiones HTTP persistentes entre ambos sistemas. Para abordar este problema se implementó un mecanismo de monitorización basado en polling periódico desde los workers del sistema web.

Una vez dada la orden de iniciar una tarea bioinformática, se almacena el identificador del proceso remoto en la base datos del sistema web. A partir de ese momento, un worker consulta periódicamente el estado del proceso mediante llamadas HTTP a la API bioinformática. Un ejemplo simplificado del mecanismo de polling implementado se muestra en [Código 6.8](#).

Cuando el estado del proceso cambia, el worker actualiza la información asociada en la base de datos del sistema web y coordina la recuperación de resultados, la generación de notificaciones y la gestión del ciclo de vida de los archivos temporales

Endpoint	Método	Descripción
/assembly/illumina	POST	Iniciar ensamblaje de secuencias Illumina con archivos FASTQ R1 y R2 con SPAdes.
/assembly/ont	POST	Iniciar ensamblaje de secuencias ONT con Raven o Flye a partir de un archivo FASTQ.
/annotation/bakta/upload	POST	Ejecutar anotación Bakta directamente desde un archivo FASTA subido.
/annotation/bakta/existing/{job_id}	POST	Ejecutar anotación Bakta sobre un ensamblaje completado previamente.
/jobs/{job_id}	GET	Consultar estado actual de un proceso específico indicado por {job_id}.
/jobs	GET	Listar todos los procesos con paginación.
/jobs/{job_id}/logs	GET	Obtener logs detallados de ejecución de un proceso.
/jobs/{job_id}/retry	POST	Reintentar un proceso fallido sin volver a subir los datos.
/jobs/{job_id}/cancel	POST	Cancelar un proceso en estado pending o running.
/assembly/{job_id}/download	GET	Descargar resultados del ensamblaje completado en formato FASTA.
/annotation/{job_id}/download	GET	Descargar resultados de anotación en múltiples formatos o como ZIP o JSON.
/jobs/{job_id}	DELETE	Eliminar un proceso específico y sus datos asociados.
/jobs/old/{days}	DELETE	Eliminar procesos más antiguos que los especificados en el endpoint raíz.
/jobs/old/{days}	GET	Obtener información sobre los procesos más antiguos que los especificados en el endpoint raíz.

Tabla 6.1: Endpoints expuestos por la API del sistema de bioinformática para la comunicación con el sistema web

```

1 @app.get("/jobs/{job_id}")
2 async def get_job_status(job_id: str):
3     """Check job status"""
4     job = get_job(job_id)
5     if not job:
6         raise HTTPException(status_code=404, detail="Job not
7         found")
8     return job

```

Código 6.6: Ejemplo del código de la API del sistema bioinformático

```

1 def get_job_status(job_id):
2     response = requests.get(f"{BASE}/jobs/{job_id}")
3     response.raise_for_status()
4     status = response.json()["status"]
5     return status, response.status_code

```

Código 6.7: Ejemplo del código de la capa de servicios encargada de la comunicación con el sistema bioinformático

y persistentes. Este enfoque permite mantener sincronizados ambos subsistemas sin bloquear la aplicación web ni depender de conexiones persistentes de larga duración.

6.6.3. Resultados

Durante este sprint se implementó la funcionalidad clave del pipeline bioinformático del sistema, incluyendo tanto el sistema bioinformático como su integración con el sistema web. La independencia de ambos subsistemas permitió ejecutar procesos de ensamblaje y anotación de forma desacoplada y asíncrona, sentando las bases para la integración posterior de generación de features y la integración de modelos de predicción.

6.7. Sprint 5 - Integración y generación de features

6.7.1. Objetivos

Este sprint se desarrolló entre el 27 de enero y el 10 de febrero de 2026, con el objetivo principal de implementar el procesamiento de los resultados de anotación y su transformación en características estructuradas utilizables por el sistema de predicción. Adicionalmente, se buscaba implementar el diseño de la arquitectura desacoplada orientada a facilitar la incorporación futura de nuevos formatos y herramientas bioinformáticas. Los objetivos específicos de este sprint fueron:

```

1  @shared_task(bind=True, autoretry_for=(Exception,),
    retry_backoff=10, max_retries=1000)
2  def poll_conversion_status(self, task_id):
3      task = Task.objects.get(id=task_id)
4      status, code = get_job_status(task.external_job_id)
5
6      if code == 404:
7          task.status = "failed"
8          task.save()
9          notify_user_conversion_failed(task.user, task)
10         return
11
12     if status != task.status:
13         if status == "annotated" or status == "assembled":
14             status = "completed"
15             task.status = status
16             task.save()
17
18     if status == "completed":
19         _persist_output(task)
20         _cleanup_temp_fastq_inputs(task)
21         notify_user_conversion_complete(task.user, task)
22         return
23
24     if status == "failed":
25         notify_user_conversion_failed(task.user, task)
26         return
27
28     try:
29         self.retry(countdown=60) # Retry after 60 seconds
30
31     except MaxRetriesExceededError: # When retries are exhausted
32         task.status = "failed"
33         task.save()
34         notify_user_conversion_failed(task.user, task)
35         return

```

Código 6.8: Mecanismo de polling implementado para monitorizar el estado de los procesos bioinformáticos desde el sistema web

- Procesar los resultados generados durante la anotación genómica.
- Diseñar un sistema de generación de features reutilizable y extensible.
- Persistir la información genómica relevante en la base de datos.
- Integrar la generación de features dentro del flujo asíncrono del sistema.
- Facilitar la incorporación futura de nuevos parsers y formatos de anotación.

6.7.2. Detalles de implementación

Procesamiento y persistencia de resultados anotados

Una vez finalizado el proceso de anotación, el sistema bioinformático genera distintos archivos que contienen información sobre genes, proteínas y otros elementos genómicos identificados durante el análisis. Para que esta información pudiera ser utilizada posteriormente por el sistema de predicción, se decidió transformarla en información estructurada persistente dentro de la base de datos del sistema web.

Inicialmente, se optó por trabajar con las salidas JSON generadas por *Bakta*, debido a su estructura jerárquica y facilidad de procesamiento desde Python. Un ejemplo simplificado del parser implementado se muestra en [Código 6.9](#).

A partir de los resultados se decidió extraer las siguientes características relevantes para la posterior generación de features y predicción de resistencia. Entre ellas destacan:

- **Identificadores asociados al gen** (identifiers), incluyendo:
 - El nombre asociado al gen (gene, [Código 6.9-5](#)).
 - Referencias externas (db_xrefs, [Código 6.9-6](#)).
 - El nombre del producto asociado (product, [Código 6.9-7](#)).
- **Información del sistema experto de anotación:** incluyendo:
 - El campo del experto (expert_field, [Código 6.9-25](#)).
 - El tipo del experto (expert_type, [Código 6.9-26](#)).

Además, el sistema permite opcionalmente realizar una extracción más exhaustiva de las **características de la secuencia genética**:

- Posición inicial (start, [Código 6.9-31](#)).
- Posición final (end, [Código 6.9-32](#)).
- Secuencia nucleotídica (nt, [Código 6.9-33](#)).
- Secuencia aminoacídica (aa, [Código 6.9-34](#)).

Dichas características, se almacenan asociadas al proceso de anotación correspondiente a través de los modelos Gene y FileGene. El modelo Gene representa la información global asociada a un gen identificado, mientras que FileGene modela la relación entre un gen concreto y un archivo o proceso de anotación específico.

```

1  def parse(self, data, file, user, complete=False):
2      file_upload = FileUpload.objects.create(file=file, user=user)
3      features = data.get('features', [])
4      for gene in features:
5          gene_name = gene.get('gene')
6          gene_db_xrefs = gene.get('db_xrefs')
7          gene_product = gene.get('product')
8          identifiers = []
9          if gene_db_xrefs:
10             for xref in gene_db_xrefs:
11                 identifiers.append(xref)
12             if gene_product:
13                 identifiers.append(gene_product)
14             if gene_name:
15                 identifiers.append(gene_name)
16
17             identifiers = {ident.strip() for ident in identifiers}
18             gene_obj = None
19             if identifiers:
20                 gene_obj =
21                     Gene.objects.search_identifiers(identifiers).first()
22                 if not gene_obj:
23                     gene_obj = Gene.objects.create()
24                     gene_obj.add_identifiers(identifiers)
25
26             expert_field = gene.get('expert')[0] if gene.get('expert')
27             else None
28             expert_type = expert_field.get('type') if expert_field else
29             'unknown'
30             if gene_obj:
31                 file_gene = FileGene.objects.create(
32                     file_upload=file_upload, gene=gene_obj,
33                     expert=expert_type)
34                 if complete:
35                     file_gene.start = gene.get('start')
36                     file_gene.stop = gene.get('stop')
37                     file_gene.nt = gene.get('nt')
38                     file_gene.aa = gene.get('aa')
39                     file_gene.save(update_fields=['start', 'stop', 'nt',
40                     'aa'])
41                 file_upload.genes.add(gene_obj)

```

Código 6.9: Ejemplo simplificado de parser de JSON generado por Bakta


```

1 class GeneQuerySet(models.QuerySet):
2     def search_identifiers(self, identifiers):
3         """Search for genes containing any of the identifiers (case
4             insensitive)"""
5         queries = models.Q()
6         for identifier in identifiers:
7             esc = re.escape(identifier.strip())
8             pattern = rf'(^|,\s*){esc}($|,)'
9             queries |= models.Q(identifiers__iregex=pattern)
10        return self.filter(queries)

```

Python

Código 6.10: Queryset personalizado para el modelo Gene

Adicionalmente, para el modelo Gene se implementó un queryset personalizado que permite recuperar genes de forma sencilla utilizando cualquiera de sus identificadores asociados. Esto facilita la búsqueda de información relacionada con un gen independientemente del nombre concreto utilizado por distintas herramientas bioinformáticas o modelos de predicción. Un extracto de código de dicho queryset se encuentra en [Código 6.10](#).

Arquitectura modular de parsers

Como punto de extensión adicional del sistema y con el objetivo de facilitar la incorporación futura de nuevas herramientas bioinformáticas y formatos de salida, se decidió desacoplar la lógica de extracción de features mediante un mecanismo basado en parsers. Este enfoque permite definir un parser para cada tipo de salida bioinformática, evitando acoplar el pipeline de generación de features a una herramienta bioinformática concreta y permitiendo extender el sistema sin modificar la lógica principal de procesamiento.

Se definió la clase base `BaseParser` que establece la interfaz común para todos los parsers ([Código 6.11](#)). Esta clase define el método `parse`, que debe ser implementado por cada parser concreto para transformar los resultados de la anotación en estructuras procesables por el sistema.

para facilitar la extensibilidad del sistema, se implementó un mecanismo de registro automático basado en el patrón *Registry* ([Código 6.12](#)). Este mecanismo permite registrar automáticamente nuevos parsers mediante decoradores, evitando configuraciones manuales.

```

1 class BaseParser():
2     def parse(self, data, file, user=None, options=None):
3         """Parse data and persist results.
4         `options` is a dict for parser-specific flags (e.g.
5         {'complete_version': True}).
6         Must return `FileUpload`.
7         """
8         raise NotImplementedError

```

Código 6.11: Clase base para los parsers

```

1 PARSERS = {}
2
3 def register_parser(name):
4     def decorator(cls):
5         PARSERS[name] = cls()
6         return cls
7     return decorator
8
9 def get_parser(name):
10    return PARSERS.get(name)

```

Código 6.12: Registro de parsers

Cada parser se registra utilizando un identificador único que posteriormente permite su resolución dinámica dentro del flujo de generación de features. Un ejemplo simplificado de implementación de parser se muestra en [Código 6.13](#).

El sistema de parsers combina el mecanismo de registro automático con un enfoque inspirado en el patrón *Strategy* ([Código 6.14](#)). Cada parser implementa una interfaz común de procesamiento (`parse_file`), permitiendo intercambiar

```

1 @register_parser('bakta_json')
2 class BaktaJsonParser(BaseParser):
3
4     def parse(self, data, file, user=None, options=None):
5         # Lógica de parsing específica para el formato JSON generado
6         # por Bakta
7         return file_upload

```

Código 6.13: Ejemplo de implementación de un parser concreto utilizando el sistema de registro mediante decoradores

```
1 def parse_file(parser, data, file, user=None, options=None):
2     parser = get_parser(parser)
3     if not parser:
4         raise RuntimeError(f'No parser registered for parser:
5                             {parser}')
6     return parser.parse(data, file, user=user, options=options)
```

 Python**Código 6.14:** Interfaz Strategy para los parsers

dinámicamente distintos algoritmos de extracción de features (parsers) según el tipo de resultado bioinformático recibido. La selección del parser concreto se realiza mediante resolución dinámica basada en identificadores registrados, desacoplando completamente la lógica de selección de la implementación concreta del parser.

Integración con el flujo de procesamiento

Finalmente, la generación de features se integró dentro del flujo asíncrono de anotado del sistema web. Una vez finalizado el proceso de anotación en el sistema bioinformático, el worker encargado de monitorizar el estado del proceso detecta la finalización de la tarea y recupera automáticamente los resultados generados. A continuación, se inicia el proceso de generación de features utilizando el sistema de parsers definido previamente. Esto permite automatizar completamente la transición entre la etapa de anotación y la posterior fase de predicción.

Adicionalmente, se incorporó la posibilidad de realizar una extracción de las características desde archivos externos de anotación previamente generados. De este modo, el usuario puede cargar un archivo JSON generado por *Bakta* y reutilizar resultados de anotación existentes sin necesidad de ejecutar nuevamente el pipeline bioinformático completo.

6.7.3. Resultados

A lo largo de este sprint se implementó la generación automática de features a partir de los resultados de anotación genómica, así como una arquitectura modular y extensible de parsers. La integración de esta funcionalidad dentro del flujo de procesamiento del sistema web permitió automatizar completamente la transición entre la etapa de anotación y la posterior fase de predicción. Además, el soporte para archivos externos de anotación permite al usuario final reutilizar resultados previamente generados sin necesidad de ejecutar nuevamente las herramientas bioinformáticas, aumentando la flexibilidad del sistema.

 **TODO:** El S6 y el S7 he de leerlos y reescribirlos, están en versión draft

6.8. Sprint 6 - Módulo de predicción de resistencia

6.8.1. Objetivos

Este sprint se desarrolló entre el 10 de febrero y el 3 de marzo de 2026 y tuvo como objetivo implementar el módulo encargado de realizar predicciones de resistencia a antibióticos a partir de las características almacenadas en base de datos durante las etapas anteriores del pipeline. Los objetivos principales de este sprint fueron:

- Integrar modelos de machine learning dentro del sistema web.
- Ofrecer utilidades para la transformación de features al formato de entrada de los modelos.

6.8.2. Detalles de implementación

Implementación de modelos de predicción

Para la versión propia del sistema y a modo de ejemplo, se decidió implementar el flujo básico de predicción para dos modelos proporcionados por el equipo de investigación, el modelo `base_bakta_50` y el modelo `base_bakta_90`. Se trata de dos modelos similares por lo que comentaremos la implementación del primero, definido en el archivo `model_classes.py` ([Código 6.15](#)).

El modelo `base_bakta_50` es un modelo de machine learning específico, diseñado para realizar predicciones de resistencia a antibióticos a partir de una lista de presencia/ausencia de genes relevantes para la resistencia.

Para obtener las features se definió el método `features()` ([Código 6.16](#)). El modelo `bakta_50` se entrenó para recibir como inputs genes específicos que vienen dados en un archivo de pickle. Por lo tanto, sacamos los nombres de los genes del archivo y los procesamos para añadir los identificadores de la base de datos correspondiente para poder identificarlos correctamente en nuestro sistema ([Código 6.16-10](#)).

Por otro lado, para la carga del modelo se definió el método `load()` ([Código 6.17](#)), encargado de cargar los pesos del modelo para el antibiótico específico desde los archivos de pesos y cambiar el modelo a modo evaluación. En este método se llama al método `features` para cargar los nombres de los genes en las variables internas del modelo.

Finalmente, se definió un método `predict()` ([Código 6.18](#)) encargado de realizar la predicción a partir de las características obtenidas, devolviendo un valor numérico

```

1  class BaselineMLP(nn.Module):
2      def __init__(self, input_dim: int, hidden_dims: List[int],
3          dropout: float):
4          super().__init__()
5          layers: List[nn.Module] = []
6          last_dim = input_dim
7          for h in hidden_dims:
8              layers.append(nn.Linear(last_dim, h))
9              layers.append(nn.ReLU(inplace=True))
10             if dropout and dropout > 0:
11                 layers.append(nn.Dropout(dropout))
12             last_dim = h
13             layers.append(nn.Linear(last_dim, 1))
14         self.net = nn.Sequential(*layers)
15
16     def forward(self, x: torch.Tensor) -> torch.Tensor:
17         return self.net(x).squeeze(-1)

```

Código 6.15: Modelo de predicción base_bakta_50

```

1  def features(self) -> list[str]:
2      if self._columns is None:
3          self._columns = get_columns_from_pickle(self.model_name,
4              self.column_file_name)
5          self._columns =
6              self._process_column_names(self._columns)
7
8      if self._input_dim is None:
9          self._input_dim = len(self._columns)
10         return self._columns
11
12     def _process_column_names(self, names: Iterable[str]) ->
13         list[str]:
14         names = [n for n in names if not str(n).startswith("a_")]
15         out = [n if str(n).startswith("UniRef:UniRef50_") else
16             f"UniRef:UniRef50_{n}" for n in names]
17         return out

```

Código 6.16: Definición de las features utilizadas por el modelo base_bakta_50

```

1  def load(self) -> None:
2      self.features()
3      model = BaselineMLP(input_dim=self._input_dim,
4                          hidden_dims=self.hidden_dims, dropout=self.dropout)
5      resolved_path = get_model_weights_path(self.antibiotic,
6      self.model_name)
7      checkpoint = torch.load(resolved_path, map_location='cpu')
8      if isinstance(checkpoint, dict) and 'state_dict' in
9      checkpoint:
10         state_dict = checkpoint['state_dict']
11     else:
12         state_dict = checkpoint
13     model.load_state_dict(state_dict)
14     model.eval()
15     self.model = model

```

Código 6.17: Definición del método de carga para el modelo base_bakta_50

que representa la probabilidad de resistencia a un antibiótico específico. Dentro de este método se calculan las features de presencia y ausencia haciendo uso de las funciones auxiliares explicadas en la próxima sección.

Utilidades de transformación de features

Para facilitar la transformación de las características almacenadas en la base de datos al formato requerido por los distintos modelos de predicción, se implementó un conjunto de utilidades de transformación en el archivo `input_utils.py`.

```

1  def predict(self, file_upload) -> float:
2      presence = presence_from_list(self._columns, file_upload)
3
4      tensor = torch.FloatTensor([list(presence)])
5      with torch.no_grad():
6          raw = self.model(tensor)
7          prob = torch.sigmoid(raw).squeeze().item()
8      return float(prob)

```

Código 6.18: Definición del método de predicción para el modelo base_bakta_50

El objetivo es que en la definición de cada modelo se puedan utilizar las utilidades de transformación de forma automática, ocultando la complejidad de algunas de las funciones más comunes.

Para la transformación, muchos de los modelos estudiados requiere como input la ausencia o presencia de ciertos genes en la muestra. Por esto se definió la función `presence_from_list(model_feature, file_upload)` (Código 6.19), encargada de calcular la presencia o ausencia de genes a partir de los resultados almacenados en la base de datos, devolviendo un vector binario que indica la presencia o ausencia de cada gen relevante para el modelo normalizado. El método devuelve una lista de valores binarios con un elemento por cada gen en `model_features`, indicando el 1 presencia y el 0 ausencia del mismo en la muestra.

Por otro lado, se definió la función `get_columns_from_pickle`. Pickle es un formato muy usado para guardar listas de cosas YAY especialmente en esto (supongo???). Esta función abstrae la lógica de extracción de los nombres de las columnas de un archivo pickle por ejemplo para la extracción de la lista de genes de input de un modelo.

Finalmente, se implementó la función `get_model_weights_path(antibiotic, model_name)` encargada de resolver la ruta al archivo de pesos del modelo específico para un antibiótico concreto. Esta función permite abstraer la lógica de localización de los archivos de pesos, facilitando la organización de los mismos y evitando hardcodear rutas dentro de los modelos. Además de abstraer la lógica de carga de los mismos.

6.8.3. Resultados

En este sprint se implementó el módulo de predicción de resistencia a antibióticos, incluyendo la integración de modelos de machine learning dentro del sistema web y la definición de utilidades para la transformación de features al formato requerido por los modelos. La implementación de este módulo sentó las bases para la posterior integración de una arquitectura modular orientada a facilitar la incorporación futura de nuevos modelos de predicción.

6.9. Sprint 7 - Arquitectura modular para modelos de predicción

6.9.1. Objetivos

Este sprint se desarrolló entre el 3 y el 17 de marzo de 2026 y tuvo como objetivo refinar la arquitectura de modelos de predicción, optimizar el rendimiento del sistema y preparar la integración con la interfaz de usuario. Los objetivos principales fueron:

```

1  def presence_from_list(model_features, file_upload):
2      # If file_upload is None or doesn't have genes, return all
      zeros
3      present_features = set()
4      genes = getattr(file_upload, 'genes', None)
5      if genes is None:
6          return [0] * len(list(model_features))
7
8      # Iterate genes and gather identifier lists
9      gene_iter = genes.all()
10
11     for gene in gene_iter:
12         ids = gene.identifiers_list() or []
13
14         for gid in ids:
15             present_features.add(str(gid).strip().lower())
16
17     # Normalize
18     model_features_norm = [str(f).strip().lower() for f in
19                             model_features]
20
21     presence_vector = [1 if feat in present_features else 0 for
22                         feat in model_features_norm]
23
24     return presence_vector

```

Código 6.19: Función de cálculo de presencia o ausencia de genes a partir de los resultados almacenados en la base de datos

- Implementar un mecanismo de registro automático de modelos mediante decoradores.
- Definición de adaptadores para los modelos.
- Validar la integridad de los adaptadores de modelos durante el registro.
- Validar la compatibilidad de los modelos con los distintos antibióticos.


```

1 def get_columns_from_pickle(model_name: str,
   column_file_name: str) -> list:
2     base_dir = Path(__file__).resolve().parents[1]
3     pkl_path = base_dir / 'ai_models' / model_name /
       column_file_name
4     with open(pkl_path, 'rb') as f:
5         columns = pickle.load(f)
6     return columns

```

Python

Código 6.20: Función de carga de archivos de pickle con información relevante para la generación de features

```

1 def get_model_weights_path(antibiotic: str, model_name:
   str) -> str:
2     base_dir = Path(__file__).resolve().parents[1]
3     pesos_dir = base_dir / 'ai_models' / model_name / 'pesos'
4     candidate = pesos_dir / f"{antibiotic}.pt"
5     if not candidate.exists():
6         raise FileNotFoundError(f"Weights file not found:
       {candidate}")
7     return str(candidate)

```

Python

Código 6.21: Función de carga de archivos de pesos para modelos de torch

6.9.2. Detalles de implementación

Arquitectura modular de modelos de predicción

Con el objetivo de cumplir con los requisitos de extensibilidad y facilitar la incorporación futura de nuevos modelos de predicción, se implementó la arquitectura modular definida para los modelos de predicción, siguiendo un enfoque basado en los patrones *Registry* y *Strategy*. El mecanismo final es similar al implementado para los parsers, permitiendo registrar nuevos modelos de forma automática mediante decoradores y seleccionar dinámicamente el modelo concreto a utilizar en función de la configuración del proceso de predicción.

Primero, se definió una clase base *ModelInterface* que establece la interfaz común para los adaptadores de todos los modelos de predicción, definiendo los métodos que deben ser implementados por cada modelo concreto para realizar predicciones a partir de las características almacenadas en la base de datos (Código 6.22). Patrón *Adapter* (decir algo más).

```

1  class ModelInterface:
2
3      def features(self, file_upload) -> list:
4          raise NotImplementedError()
5
6      def load(self) -> None:
7          raise NotImplementedError()
8
9      def predict(self, file_upload) -> float:
10         raise NotImplementedError()

```

Código 6.22: Clase base para los modelos de predicción

En la interfaz se definen tres métodos:

- `features(file_upload)` - (**Código 6.22-3**): Dado un archivo de subida persistido en base de datos (`FileUpload`), devuelve las características necesarias para realizar la predicción (por ejemplo, una lista de genes presentes o ausentes).
- `load()` - (**Código 6.22-6**): Carga los pesos del modelo y realiza cualquier preparación necesaria para la ejecución de predicciones.
- `predict(file_upload)` - (**Código 6.22-9**): Realiza la predicción de resistencia a antibióticos a partir de las características asociadas al archivo de subida proporcionado, devolviendo un valor numérico que representa la probabilidad de resistencia.

A continuación, se implementó un mecanismo de registro automático para los modelos de predicción, siguiendo el patrón *Registry* de forma similar al utilizado para los parsers. Este mecanismo permite registrar nuevos modelos mediante decoradores, evitando configuraciones manuales y facilitando la extensión del sistema con nuevos modelos de forma sencilla (**Código 6.23**).

Se definieron dos diccionarios con los modelos y sus clases y los antibioticos a los que dan soporte, que se populan al iniciar la aplicación de forma automática gracias al decorador definido en **Código 6.23-22**. Al decorador se le puede pasar un identificador que mostrará a los usuarios en el sistema final así como se usará de forma interna para identificar al modelo. Adicionalmente, en el registro se computan los antibióticos que están disponibles para cada modelo a partir de los archivos de pesos asociados a cada modelo, lo que permite mostrar esta información de forma dinámica en la interfaz de usuario y evitar errores de configuración.

```

1  MODEL_REGISTRY = {
2      # 'model_name': ModelAdapterClass,
3  }
4
5  MODEL_ANTIOTICS = {
6      # 'model_name': ['antibiotic1', 'antibiotic2', ...],
7  }
8
9  def _validate_adapter_init(cls) -> None:
10     try:
11         sig = inspect.signature(cls.__init__)
12     except (TypeError, ValueError):
13         raise TypeError(f"Cannot inspect __init__ of adapter {cls!r}")
14
15     params = [p for p in sig.parameters.values() if p.name != 'self']
16     names = {p.name for p in params}
17     if 'antibiotic' not in names:
18         raise TypeError(
19             f"Adapter {cls.__name__} must accept an 'antibiotic' parameter in __init__"
20         )
21
22 def register_model(name: str = None):
23     def _decorator(cls):
24         _validate_adapter_init(cls)
25         key = (name or cls.__name__).lower()
26         base_dir = Path(__file__).resolve().parents[1]
27         pesos_dir = base_dir / 'ai_models' / key / 'pesos'
28         MODEL_REGISTRY[key] = cls
29         MODEL_ANTIOTICS[key] = _compute_model_supported_antibiotics(pesos_dir)
30         return cls
31     return _decorator

```

Código 6.23: Registro de modelos de predicción

```

1 def get_model_adapter_class(name: str):
2     key = name.lower() if name else ''
3     if key not in MODEL_REGISTRY:
4         raise ValueError(f"Model '{name}' not found in
5         registry.")
6     return MODEL_REGISTRY[key]

```

Código 6.24: Registro de modelos de predicción

Además, para asegurar la integridad de las clases y los métodos de los adaptadores se definió un validador que se ejecuta durante el registro de cada modelo, verificando que la clase implementa correctamente la interfaz definida en `ModelInterface`, que los métodos necesarios están presentes y proporciona el nombre de la clase como identificador para el modelo si no se ha definido uno en el decorador (Código 6.23-9).

Por otro lado, inspirado en el patrón *Strategy* se definió un mecanismo de selección dinámica del modelo concreto a utilizar en función de la configuración del proceso de predicción. Esto permite intercambiar distintos modelos de predicción simplemente llamando a una función con distintos parámetros. El código se puede ver en Código 6.25, donde se define la función `get_prediction` que recibe como parámetro el nombre del modelo a utilizar, el antibiótico y el archivo de subida asociado al proceso de predicción, resolviendo dinámicamente la clase del modelo a utilizar a través del mecanismo de registro y ejecutando las funciones de carga (`load()`) y predicción (`predict()`) de forma transparente para el usuario.

```

1 def get_prediction(model_name: str, antibiotic: str,
2     file_upload) -> float:
3     model_cls = get_model_adapter_class(model_name)
4     if not model_cls:
5         raise ValueError(f'Model {model_name} not found in
6         registry.')
7
8     adapter = model_cls(antibiotic=antibiotic)
9
10    adapter.load()
11    return adapter.predict(file_upload)

```

Código 6.25: Llamada única de predicción

Instrucciones de integración de nuevos modelos

Para la administración centralizada de los modelos se desarrolló el módulo `ai_models`. Este módulo actúa como punto centralizado para la definición de modelos de predicción, sus adaptadores y utilidades relacionadas.

La organización interna del módulo sigue una estructura modular que permite aislar cada modelo en su propia carpeta, facilitando la gestión de archivos, pesos y código asociado a cada modelo. La estructura estándar es la mostrada en **Código 6.26**

```
1 ai_models/<model_name>/
2   pesos/
3     <antibiotic>.pt
4   model_classes.py
```

Código 6.26: Estructura de archivos recomendada para la integración de nuevos modelos de predicción dentro del módulo ``ai_models``

Donde `model_classes.py` debe exponer la arquitectura del modelo y la clase adaptadora que implementa la interfaz de predicción, registrada mediante el decorador `@register_model("alias")`. En la carpeta de pesos deberán aparecer una lista de archivos de pesos en formato `.pt` nombrados por el antibiótico al que pertenecen.

Integración en el flujo de procesamiento

Finalmente, los modelos implementados inicialmente se adaptaron moviendo los métodos definidos a los adaptadores correspondientes.

Además, se integró la llamada a los modelos de predicción dentro del flujo de procesamiento del sistema web, permitiendo que una vez generadas las características a partir de los resultados de anotación, se pueda realizar la predicción de resistencia a antibióticos utilizando el modelo seleccionado. Adicionalmente, se implementó una función de predicción para múltiples antibióticos que hace uso de la definida previamente `get_prediction`.

El método `get_prediction_matrix(models, antibiotics, file_upload)` recibe una lista de modelos, una lista de antibióticos y un archivo de subida asociado a un proceso de anotación, y devuelve una matriz de predicciones con la probabilidad de resistencia a cada antibiótico para cada modelo. La función itera sobre cada combinación de modelo y antibiótico, llamando a la función `get_prediction` para obtener la predicción correspondiente y almacenándola en una estructura de diccionario anidado que se devuelve al finalizar el proceso. Preparando los resultados

```

1  def get_prediction_matrix(model_names: list[str],
    antibiotics: list[str], file_upload) -> dict:
2      data = {}
3      for antibiotic in antibiotics:
4          row = {}
5          for model_name in model_names:
6              try:
7                  row[model_name] = get_prediction(model_name,
                                                    antibiotic, file_upload)
8              except Exception:
9                  row[model_name] = None
10         data[antibiotic] = row
11
12     return data

```

 Python

Código 6.27: Función de predicción para múltiples antibióticos

para su visualización según el mockups planteado durante el diseño del sistema en la [Figura 5.8](#).

6.9.3. Resultados

En este sprint se consolidó la arquitectura modular de modelos de predicción, estableciendo un sistema de registro automático de modelos con validación de integridad. Además se ha definido una interfaz de adaptadores y se ha ofrecido una estructura estándar para la incorporación de nuevos modelos que minimiza la configuración manual. Finalmente, se ha asegurado la correcta implementación e integración de dos modelos de predicción completamente funcionales (base_bakta_50 y base_bakta_90) listos para uso en el sistema final.

6.10. Sprint 8 - Interfaz de usuario y notificaciones

6.10.1. Objetivos

6.10.2. Detalles de implementación

6.10.3. Resultados

6.11. Sprint 9 - Integración del sistema, pruebas y validación

6.11.1. Objetivos

6.11.2. Detalles de implementación

6.11.3. Resultados

6.12. Sprint 10 - Memoria, ajustes finales y puesta en producción

6.12.1. Objetivos

6.12.2. Detalles de implementación

6.12.3. Resultados

6.13. Conclusiones

Conclusiones

1. trabajo futuro

Bibliografía

- [1] Z. Sadeghi *et al.*, «A review of Explainable Artificial Intelligence in healthcare», *Computers and Electrical Engineering*, vol. 118, p. 109370, 2024, doi: <https://doi.org/10.1016/j.compeleceng.2024.109370>.